Alt Seviye Programlama
BLM2021

Öğr. Grv. Furkan ÇAKMAK

Ders İşleyiş Bilgilendirmesi

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

- Teorik Ders Saatleri: **Perşembe 09:00 - 11:50**
- Derslerin tamamı online ortamda olacak.
- Duyurular için: https://avesis.yildiz.edu.tr/fcakmak/dokumanlar
  - UBIS Toplu Mesaj Sistemi
- Google Classroom Sınıf Kodu: **wtrrzdi**
  - Dersle ilgili sorularınızı Classroom üzerinden duvar paylaşımı yaparak sorabilirsiniz.
- Zoom Personal Room:
  - https://us04web.zoom.us/j/3752287039?pwd=TTFKNittZWJTUEhHREovckl0VTVYUT09
  - Meeting ID: 375 228 7039
  - Passcode: 5AXMNd
- Dersle ilgili soru ve danışmalarınız için EN AZ 1 (BİR) GÜN ÖNCEDEN linkten randevu alınız.
  - https://fcakmak.simplybook.it/v2/
  - Mail ile randevu verilmeyecektir.
  - Alınan randevu saatinde lütfen yukarıda verilen Zoom odasında bulununuz.

Ders İşleyiş Bilgilendirmesi - 2

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

- Öğrenciler ders esnasındaki soruları yazarak DEĞİL sesli olarak sormalıdırlar.
- Soru sormak isteyen öğrenci muhakkak kendini **görünebilir** kılmalıdır.
- Öğrenciler söz aldığı durumlar hariç ders dinlerken mikrofonlarını KAPALI tutmalılar.
- Olası uzun süreli (>1dk) bağlantı kopması durumlarında dersin devamıyla ilgili duyuru anında Google Classroom sayfasından yapılacaktır.
- Dileyen öğrenciler dersi anlatımını harici olarak kendi imkanları ile kaydedebilirler.
  - Bunun haricinde ders sunuları paylaşılMAyacaktır.

Ders Tanıtım Formu ve Konular

BLM20
Alt Seviye Programlama

Hafta 1

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2021	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

Ders Tanıtım Formu - Ders Kitabı

BLM20
Alt Seviye
Programlama

Hafta 1

	Kitaplar
1	80x86 Assembly Dili, Tasarım ve Çevre Birimleri, A.Tevfik İNAN, E. USLU, F. ÇAKMAK - Seçkin Yayıncılık
2	IBM PC Assembler Language and Programming, Peter Abel - Prentice Hall
3	The 80386, 80486 and Pentium Processor, Walter A.Triebel - Prentice Hall
4	THE INTEL MICROPROCESSORS Architecture, Programming, and Interfacing, Barry Brey - Prentice Hall

Ders Tanıtım Formu - Değerlendirme

BLM20 Furkan Ç
Alt Seviye Programlama

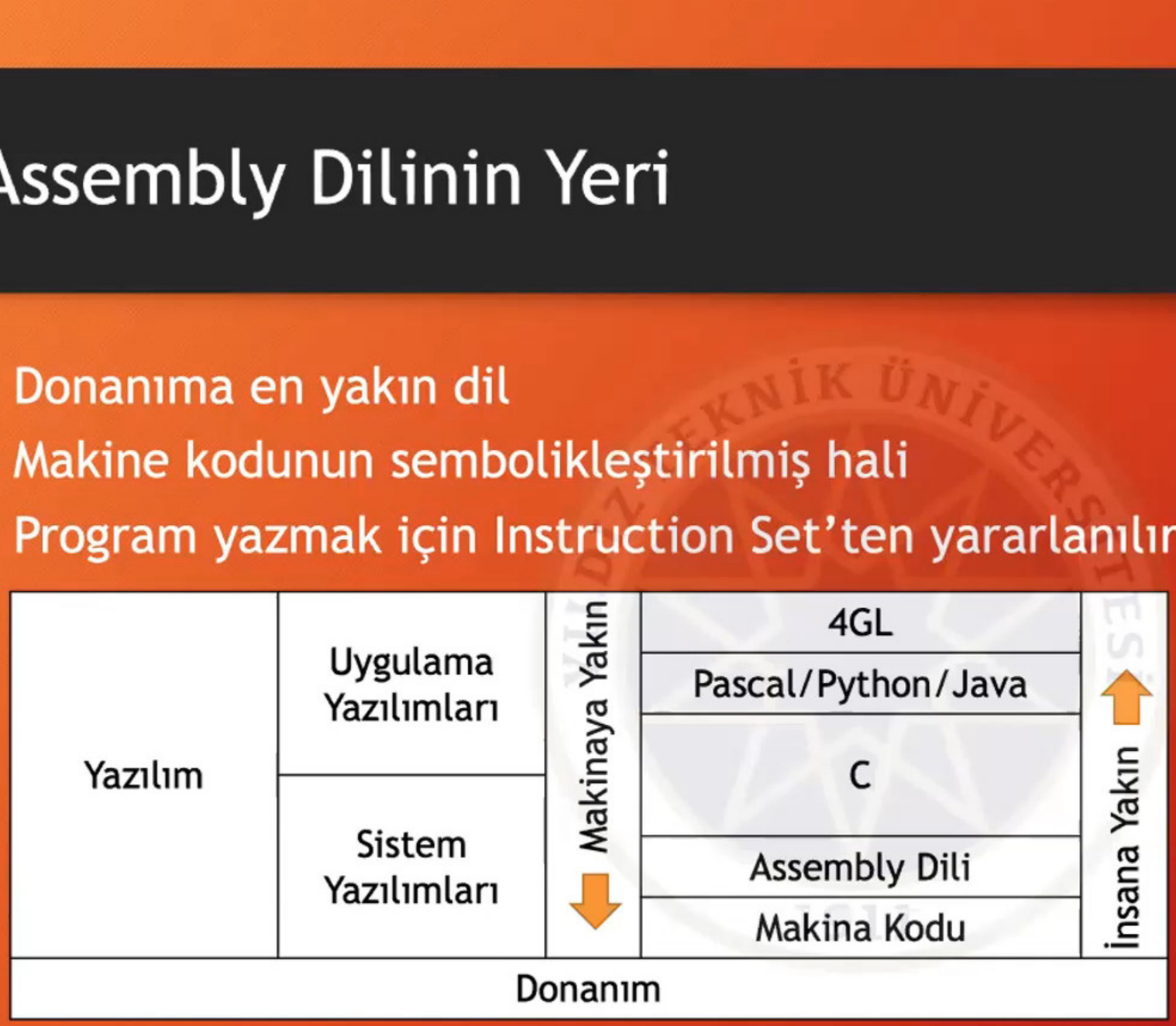
Hafta 1

	Yöntem	Adedi	Etki Oranı (%)
Başarı Değerlendirme Sistemi	Ara Sınavlar	-	-
	Kısa Sınavlar	-	-
	Ödevler	3	60
	Projeler	-	-
	Dönem Ödevi	-	-
	Laboratuvar	-	-
	Diğer	-	-
	Final Sınavı	1	40

Assembly Dilinin Yeri

Furkan Ç

- Donanıma en yakın dil
- Makine kodunun sembolikleştirilmiş hali
- Program yazmak için Instruction Set'ten yararlanılır



Assembly Dili Hakkında Yanlışlar*

BLM20 Alt Seviye Programlama

Hafta 1

- Öğrenmek zordur.
- Okumak ve anlamak güçtür.
- Hatayı bulmak zordur.
- Yazılmış programların bakımı zordur.
- Program yazmak zordur.
- Programlama uzun zaman almaktadır.
- Gelişen derleyici teknolojileri bu dile ihtiyacı kaldırmıştır.
- Bilgisayarlar o kadar hızlanmıştır ki, hız için assembly diline olan ihtiyaç ortadan kalmıştır.
- Uygulamanın hızlanması için assembly dili yerine daha iyi bir algoritma kullanılmalıdır.
- Günümüzde bilgisayarların bellekleri arttı ve ucuzladı. Yerden (bellek) kazanmak için assembly diline gerek kalmadı.
- Assembly dili taşınabilir (portable) değildir.

* Randall Hyde, 1996, «The Art of Assembly Language»

Assembly Dili Hakkında Doğrular*

BLM20 Alt Seviye Programlama

Hafta 1

- Hızlıdır.
- Bellek üzerinde az yer kaplar.
- Yetenekleri fazladır.
- Bilgi birikimi ile yüksek seviyeli dillerde hareket kabiliyeti artar.

* Randall Hyde, 1996, «The Art of Assembly Language»

Sayı Sistemleri

BLM20
Alt Seviye
Programlama

Hafta 1

- Bilgisayar teknolojisinin temelleri transistörlere dayanır.
  - Kapalı devre – 0
  - Açık devre – 1
  - İkili (Binary) sayı sistemi
- Gösterim kolaylaştırmak için;
  - Sekizli (Octal)
  - On altını (Hexadecimal)

İkili (Binary) Sayı Düzeni

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

- Her bir digit 1 biti temsil eder.
- 8 bit = 1 byte
- En anlamlı bit (MSB – Most Significant Bit) -> en soldaki bit
- En az anlamlı bit (LSB – Least Significant Bit) -> en sağdaki bit
- Program yazarken B harfi ile tanımlanır.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
↓	↓	↓	↓	↓	↓	↓	↓	
128	64	32	16	8	4	2	1	B
1	0	1	1	0	1	0	1	
↓	↓	↓	↓	↓	↓	↓	↓	
128	0	32	16	0	4	0	1	$= 181$

Binary İşlemler (Toplama, Çıkartma)

Toplama

1 1 1 1

73 → 0 1 0 0 1 0 0 1 B

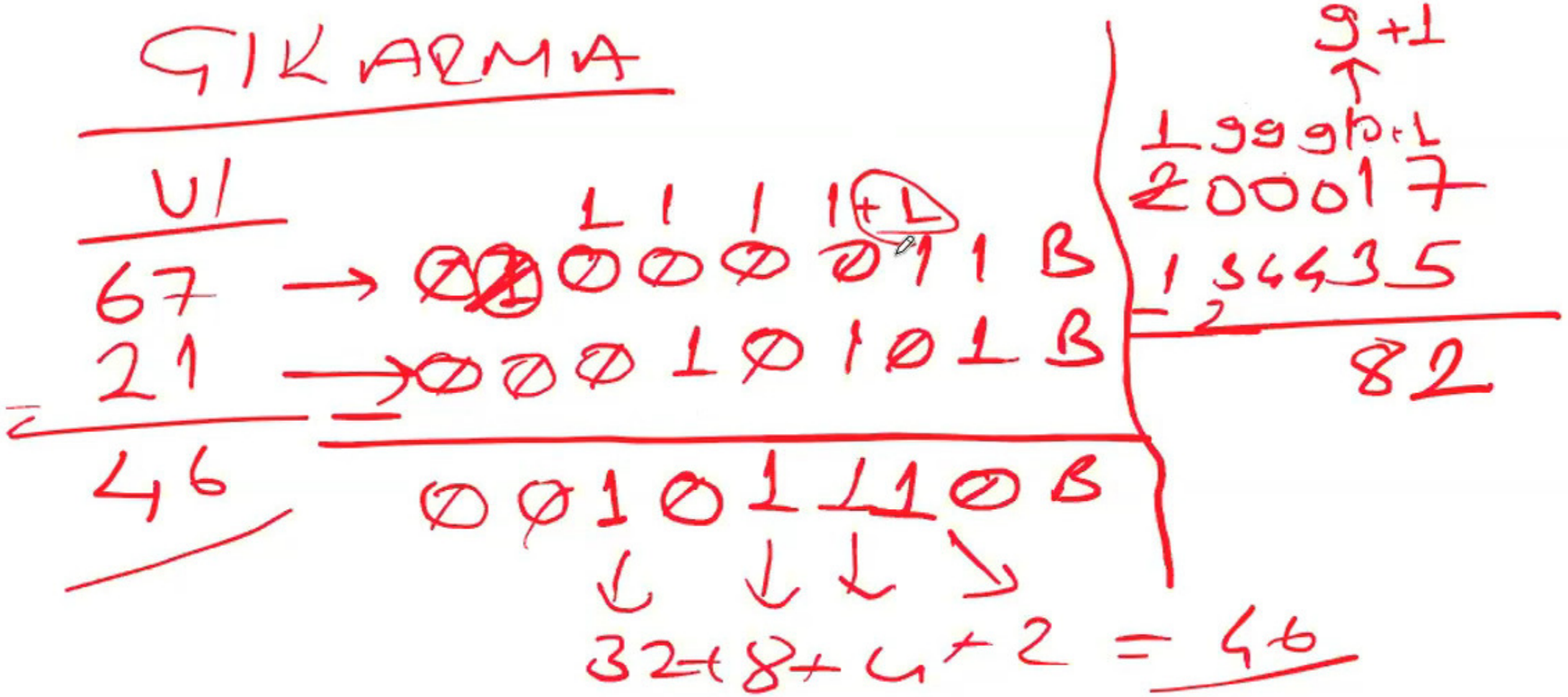
62 → 0 0 1 1 1 1 1 0 B

+

135 → 1 0 0 0 0 1 1 1

128 + 4 + 2 + 1 = 135

Binary İşlemler (Toplama, Çıkartma)



Binary İşlemler (Toplama, Çıkartma)

V2

67 → 01000011B

21 → 00010101B → 21

-21 → 11101010B → 21' (Tümleyen)

+1B

11101011B → 21'' → (-21)

67

-21

+

01000011B

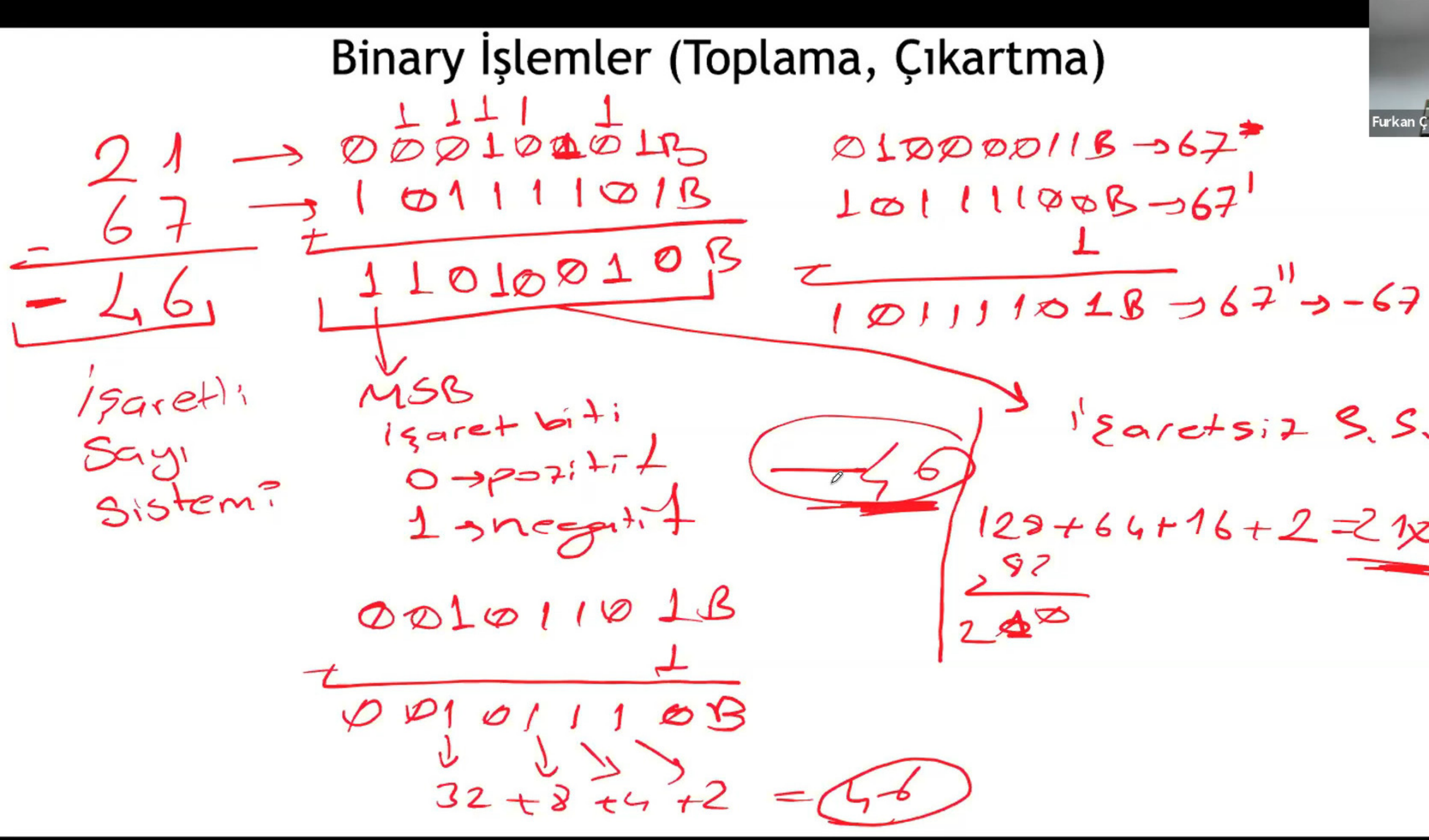
00101110

↓ ↓

32 + 8 + 4 + 2 = 46

Taşma 1 ↙

Binary İşlemler (Toplama, Çıkartma)



Octal ve Hexadecimal Sayı Düzeni

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

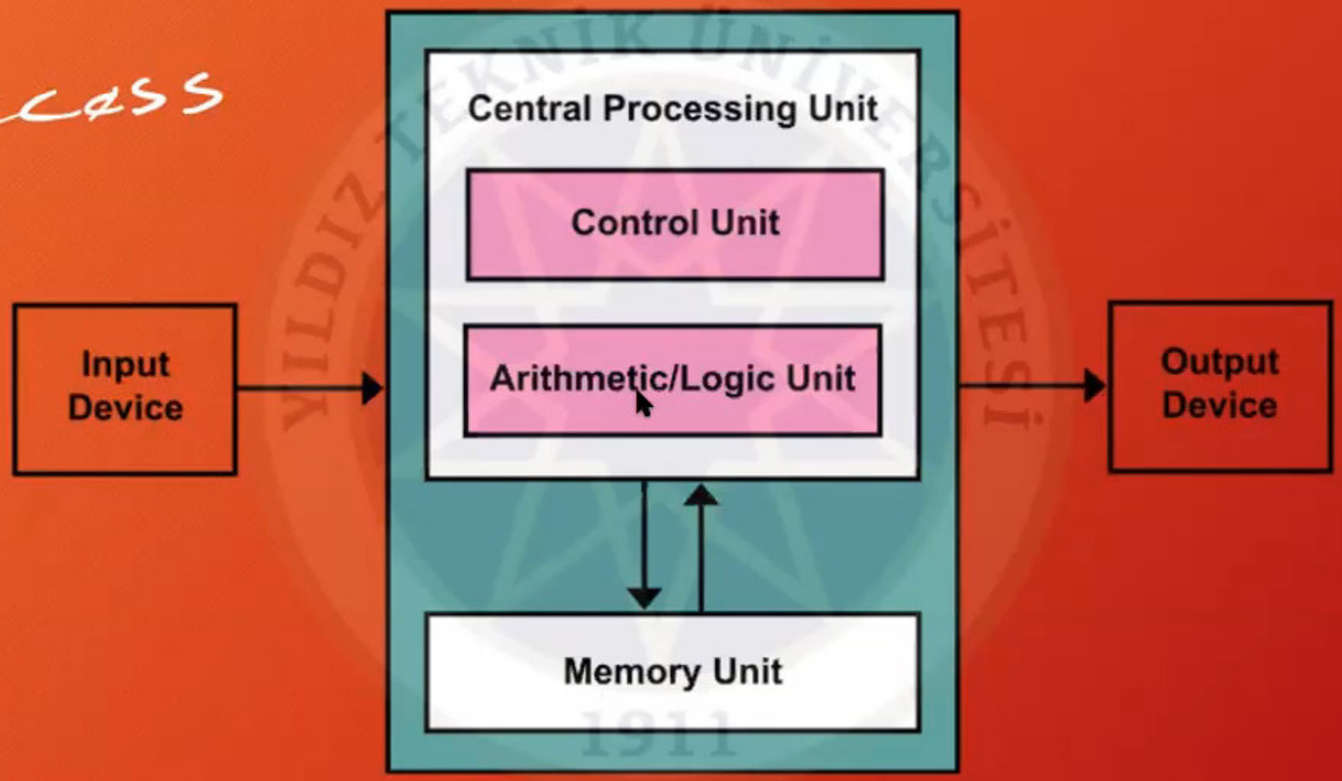
Decimal	*Binary*	*Octal*	*Hex.*	*Decimal*	*Binary*	*Octal*	*Hex.*
0	0000B	01*O*	00*H*	8	1000*B*	10*O*	08*H*
1	0001*B*	02*O*	01*H*	9	1001*B*	11*O*	09*H*
2	0010*B*	03*O*	02*H*	10	1010*B*	12*O*	0A*H*
3	0011*B*	04*O*	03*H*	11	1011*B*	13*O*	0B*H*
4	0100*B*	05*O*	04*H*	12	1100*B*	14*O*	0C*H*
5	0101*B*	06*O*	05*H*	13	1101*B*	15*O*	0D*H*
6	0110*B*	06*O*	06*H*	14	1110*B*	16*O*	0E*H*
7	0111*B*	07*O*	07*H*	15	1111*B*	17*O*	0F*H*

Sistem Organizasyonu (Von Neuman - 80x86)

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

RAM
Random Access Memory



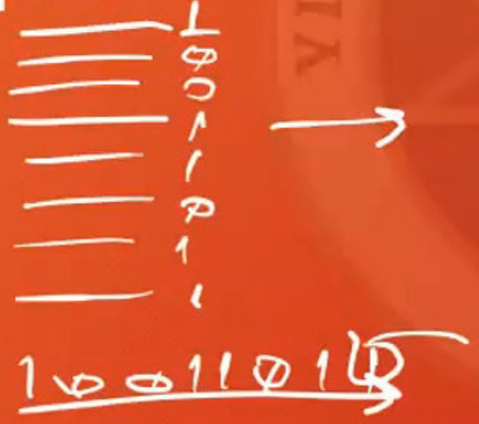
R1

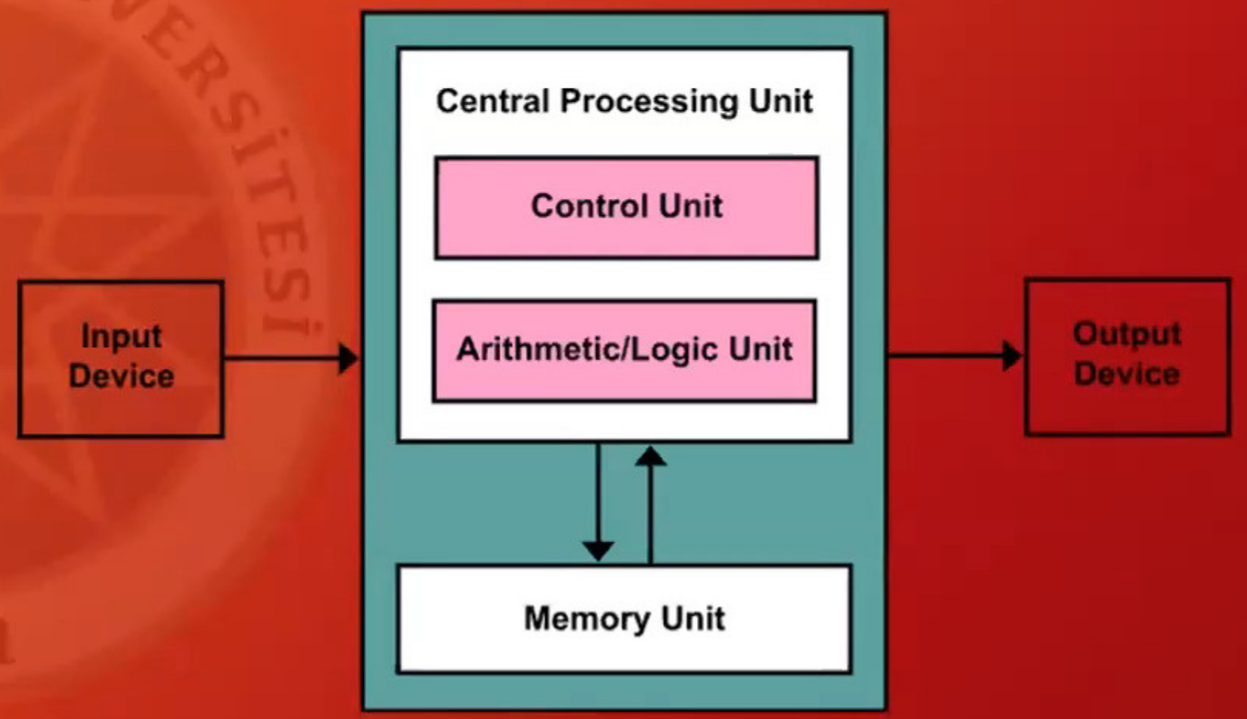
Sistem Organizasyonu (Von Neuman - 80x86)

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

- Birimlerin bağlantıları yollar (bus) ile sağlanır.
  - Address bus (Adres yolu)
  - Data bus (Veri yolu)
  - Control bus (Kontrol yolu)
- Yolların tipleri
  - Paralel
  - Seri





Yol (Bus) Çeşitleri

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1

- FSB (Front Side Bus)
  - İşlemci ile anakart arasındaki bağlantı
- Bellek Bağlantı Yolları
  - SDRAM (Synchronous Dynamic RAM)
  - DDR-SDRAM (Dual Data Rate SDRAM)
  - DDR2-SDRAM
  - DDR3-SDRAM
  - DDR4-SDRAM
  - RDRAM (Rambus Dynamic RAM)
- Grafik Birim Bağlantı Yolları
  - AGP (Advanced Graphic Port)
  - PCIe (Peripheral Component Interconnect Express)
- İkincil Bellek Birim Bağlantı Yolları
  - ATA (Advanced Technology Attachment)
  - SATA (Serial ATA)
- Diğer Çevre Birim Bağlantı Yolları
  - IEEE 1394
  - Apple -> Firewire
  - Sony -> i.Link
- USB

80x86 Ailesi İşlemciler

BLM20 Furkan Ç
Alt Seviye Programlama

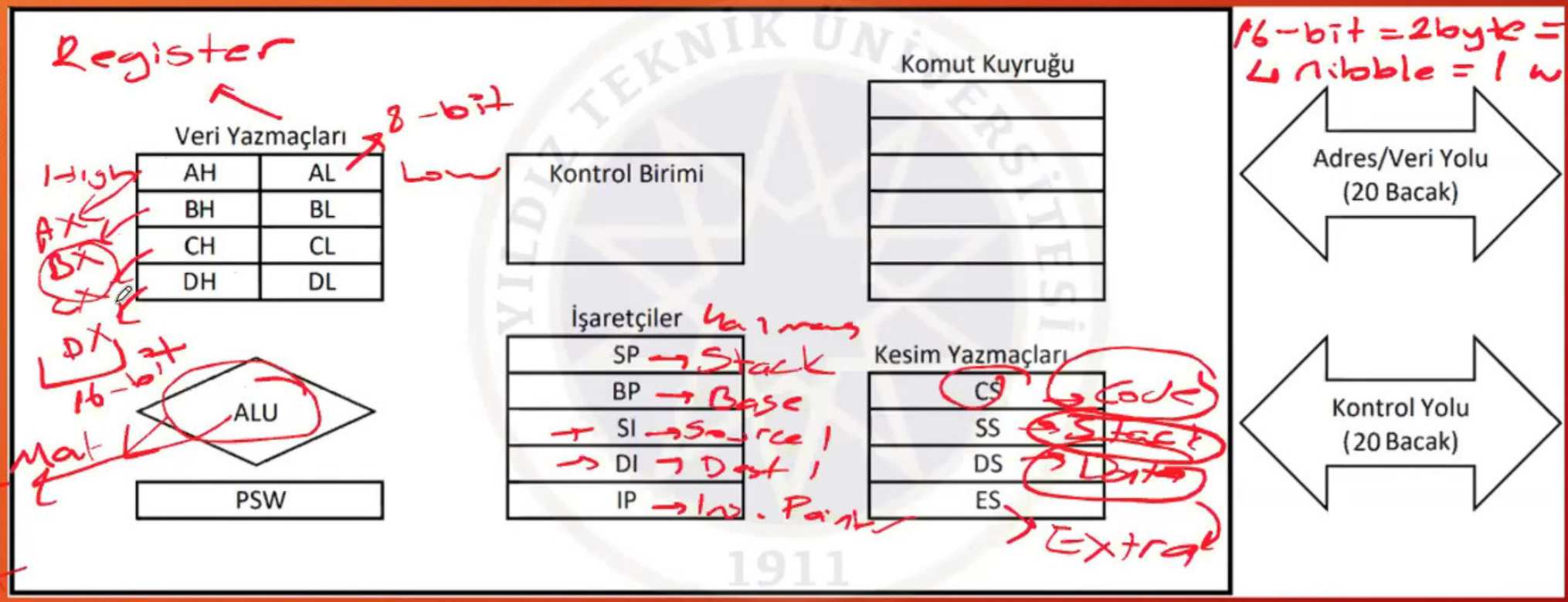
Hafta 1

- CISC (Complex Instruction Set Computers)
- İşlem hacmi -> 8, 16, 32, 64
- Intel, AMD, Cyrix, IBM, TI, NexGen, vb.
- Backward Code Compatibility (Geriye Yönelik Kod Uyumluluğu)
- Matematik İşlemciler -> 8086 için 8087 vb.

8086 İşlemcinin İç Yapısı

BLM20 Furkan Ç
Alt Seviye Programlama

Hafta 1



Bellekler, Yazmaçlar ve Fiziksel Adres Hesapları

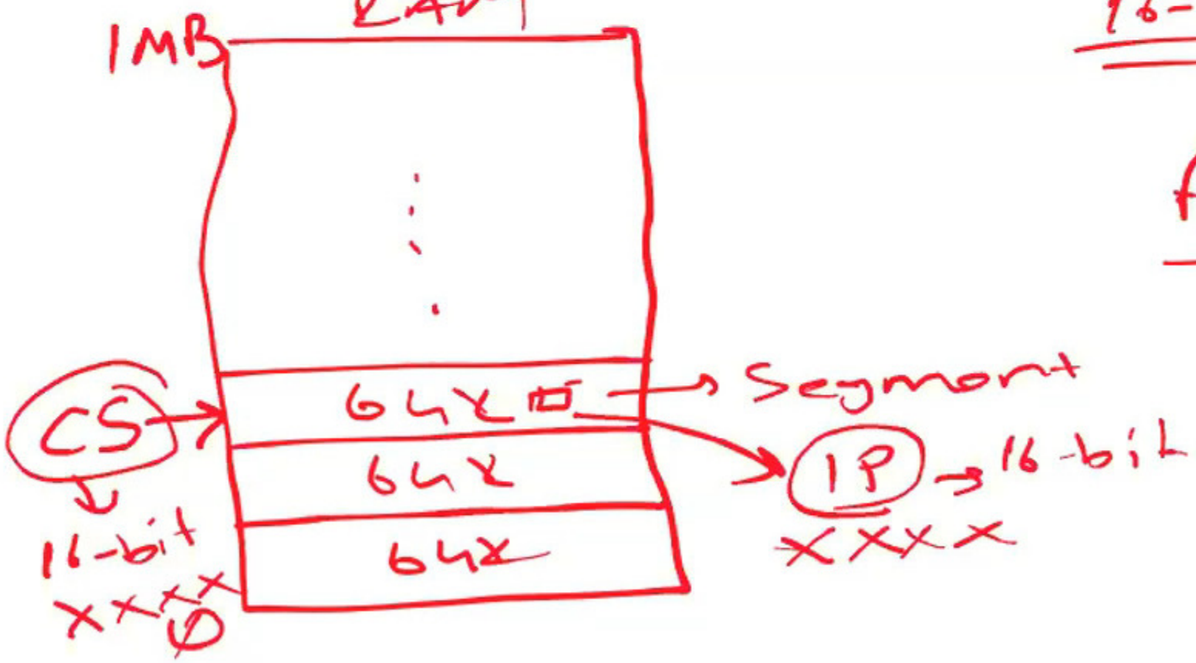
$2^{16} \rightarrow 65536 \rightarrow 64K \times 1000 = 64000$ (segment)

$2^6 \times 2^{10} \rightarrow 2^{20}$ (1024) $\rightarrow 2^{30}$

K M G

$2^{20} = 1MB$

RAM



int a[1000];

16-bit

92

CS : 2C50H

IP : 0093H

2C500H

0093H

2C593H

Fiziksel Adres Hesabı

CS x 16 + IP

CS : 2C58H

IP : 0013H

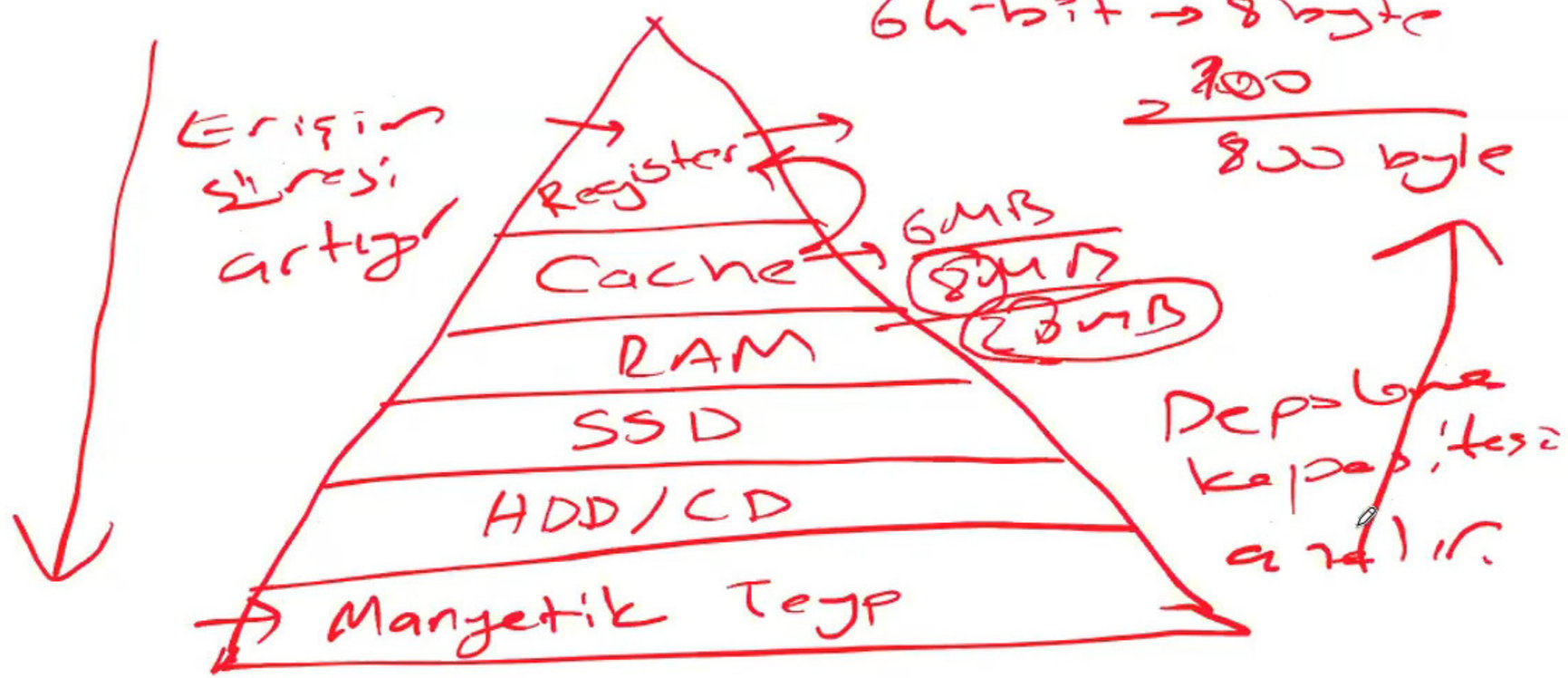
2C580H

0013H

2C593H

FA

Bellekler, Yazmaçlar ve Fiziksel Adres Hesapları

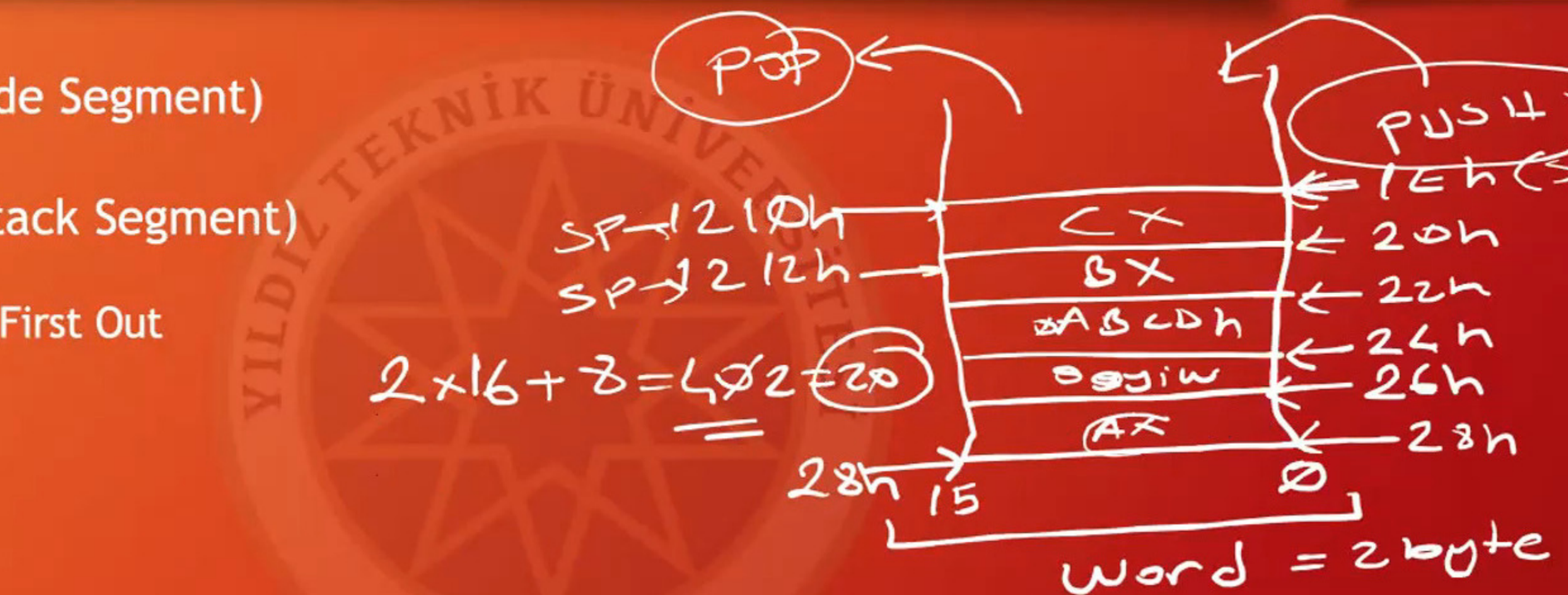


Segments / Kesimler

BLM20
Alt Seviye Programlama

Hafta 1

- Kod Kesimi (Code Segment)
  - CS:IP
- Yığın Kesimi (Stack Segment)
  - SS:[SP-BP]
  - LIPO: Last in First Out



Segments / Kesimler

BLM20
Alt Seviye Programlama

Hafta 1

Furkan ÇAK

- Kod Kesimi (Code Segment)
  - CS:IP
- Yığın Kesimi (Stack Segment)
  - SS:[SP-BP]
  - LIPO: Last in First Out
  - PUSH - POP
- Veri Kesimi (Data Segment)
  - DS:[SI, DI, BX]
- Ek Kesim (Extra Segment)
  - ES:[SI, DI, BX]

	Kesim	*Göreli Konum*
Komut Adresleme	*CS*	*IP*
Yığın Adresleme	*SS*	*SP*
	SS	*BP*
Veri Adresleme	*DS*	*SI, DI, BX*
	ES	*DI, SI, BX*

Bayraklar (Flags - PSW) → 16-bit, 2-byte

BLM20
Alt Seviye Programlama

Hafta 1

Furkan ÇAK

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
–	–	–	–	OF	DF	IF	TP	SF	ZF	–	AF	–	PF	–	CF

Bayrak	Görevi (Set –> 1, Clear, Reset –> 0)
CF: Carry Flag	Elde ödünç durumlarında CF set olur.
PF: Parity Flag	Even parity –> Set, Odd Parity –> Clear (Even: Çift)
AF: Auxilary CF	8 – bit işlemde 4'den 5'e; 16 – bit işlemde 8'den 9'a elde/ödünç aktarımı
ZF: Zero Flag	İşlem sonucu 0 ise Set, 1 ise Clear
SF: Sign Flag	İşlem sonucu negatif ise Set, pozitif ise Clear
TP: Trap Flag	Adım bayrağı. Set ise her komuttan sonra kesme üretilerek prog. izleniyor
IF: Interrupt Flag	Maskelenebilir (Maskable) kesmelerin kontrolü için kullanılır. 1 ise izin verilir.
DF: Direction Flag	Dizgi (String) işlemlerinde işlemin yönünü belirlemek için kullanılır.
OF: Overflow Flag	Aritmetik taşma durumunda Set, aksi durumda Clear.

Belleğe Erişim

Furkan ÇAK

- İşlemcinin adresleyebileceği bellek adres bacağı ile değişir.
  - $20\ bacak\ varsa;\ 2^{20} = 1\ MB$
- Fiziksel adres hesabı

Gerçek Kip	*Korumalı Kip*
• 16-bitlik kesim yazmaçları sayesinde 64K'lık bloklara erişilebilir. Toplam alan 1MB'dır. • Programlar birbirlerinin bellek alanına erişebilir. Multitasking (Çoklu görev) yoktur. • Yazılım ile BIOS kodlarına veya donanıma doğrudan erişmek mümkündür.	• Bellek koruması vardır. Programlar birbirlerinin alanına giremez. Çoklu görev desteklenir. • Donanım olarak, programın çalışma sıralarını değiştirme hakkı vardır (Preemptive Multitasking) • Korumalı kipte yazmaçlar 32-bitlik tir. • Günümüz işletim sistemleri açılışı sırasında işlemciyi korumalı kipe geçirerek çalışır.

İşlemcinin Komutları Adım Adım Çalıştırması

BLM20
Alt Seviye Programlama

Hafta 1

Furkan ÇAK

1. Komutu belirleyen byte'ın bellek üzerindeki kod alanına alınması (Instruction fetch)
2. IP yazmacının bir sonraki byte'ı gösterecek şekilde değiştirilmesi
3. Alınan komutun ne komutu olduğunun ve ne tür parametrelerle çalışacağının belirlenmesi (Instruction Decode)
4. Gerekli olması durumunda kullanılacak parametrelerin bellek alanından alınması (Operand Fetch)
5. IP yazmacının bir sonraki komutu gösterecek şekilde ayarlanması
6. İşlemin gerçekleştirilmesi (Execute)
7. Elde edilen sonucun gerekli olan yere yerleştirilmesi (Store)

Ders Tanıtım Formu ve Konular

BLM20
Alt Seviye Programlama

Hafta 2

Furkan ÇAK

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

Mnemonic (Assembly Komutları)

BLM20
Alt Seviye Programlama

Hafta 2

Furkan ÇAK

operand

{Etiket:} Mnemonic {{işlenen1,}, işlenen2} ;açıklamalar

```
MOV   AX, 2
ADD   Ax, BX
SUB   cx, 7
INC   BP
NOP   → 1c
```

Veri Aktarım Komutları

BLM20
Alt Seviye Programlama

Hafta 2

Veri Aktarım Komutları				
MOV	LEA	LDS	LES	XCHG

Öğr. Grv. Furkan ÇAKMAK

Veri Aktarım Komutları - MOV

BLM20 Alt Seviye Programlama

Hafta 2

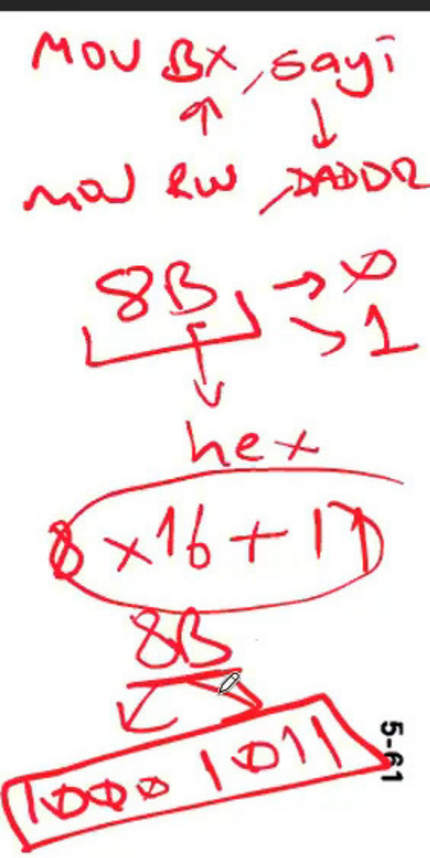
Furkan ÇAK

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses O	D	I	T	S	Z	A	P	C	Operation Performed
Memory Reference (Continued)	MOV	RW,DADDR	8B aadddbbb [DISP][DISP]	8+EA										[RW] ← [EA] Load 16 bits of data from the data memory word addressed by DADDR to register RW
	MOV	DADDR,RB	88 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RB] Store the data byte from register RB in the memory byte addressed by DADDR
	MOV	DADDR,RW	89 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RW] Store the 16-bit data word from register RW in the memory word addressed by DADDR
	MOV	AL,LABEL	A0 PPQQ	10										[AL] ← [EA] Load the data memory byte directly addressed by LABEL into register AL
	MOV	AX,LABEL	A1 PPQQ	10										[AX] ← [EA] Load the 16-bit data memory word directly addressed by LABEL into register AX
	MOV	LABEL,AL	A2 PPQQ	10										[EA] ← [AL] Store the 8-bit contents of register AL into the data memory byte directly addressed by LABEL
	MOV	LABEL,AX	A3 PPQQ	10										[EA] ← [AX] Store the 16-bit contents of register AX into the data memory word directly addressed by LABEL
	MOV	SR,DADDR	8E aa0rrbbb [DISP][DISP]	8+EA										[SR] ← [EA] Load into Segment register SR the contents of the 16-bit memory word addressed by DADDR
	MOV	DADDR,SR	8C aa0rrbbb [DISP][DISP]	9+EA										[EA] ← [SR] Store the contents of Segment register SR in the 16-bit memory location addresed by DADDR

Veri Aktarım Komutları - MOV

BLM20 Alt Seviye Programlama

Hafta 2



Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses O	D	I	T	S	Z	A	P	C	Operation Performed
Memory Reference (Continued)	MOV	RW,DADDR	8B adddbbb [DISP][DISP]	8+EA										[RW] ← [EA] Load 16 bits of data from the data memory word addressed by DADDR to register RW
	MOV	DADDR,RB	88 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RB] Store the data byte from register RB in the memory byte addressed by DADDR
	MOV	DADDR,RW	89 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RW] Store the 16-bit data word from register RW in the memory word addressed by DADDR
	MOV	AL,LABEL	A0 PPQQ	10										[AL] ← [EA] Load the data memory byte directly addressed by LABEL into register AL
	MOV	AX,LABEL	A1 PPQQ	10										[AX] ← [EA] Load the 16-bit data memory word directly addressed by LABEL into register AX
	MOV	LABEL,AL	A2 PPQQ	10										[EA] ← [AL] Store the 8-bit contents of register AL into the data memory byte directly addressed by LABEL
	MOV	LABEL,AX	A3 PPQQ	10										[EA] ← [AX] Store the 16-bit contents of register AX into the data memory word directly addressed by LABEL
	MOV	SR,DADDR	8E aa0rrbbb [DISP][DISP]	8+EA										[SR] ← [EA] Load into Segment register SR the contents of the 16-bit memory word addressed by DADDR
	MOV	DADDR,SR	8C aa0rrbbb [DISP][DISP]	9+EA										[EA] ← [SR] Store the contents of Segment register SR in the 16-bit memory location addressed by DADDR

5-61

Veri Aktarım Komutları - MOV

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses O	D	I	T	S	Z	A	P	C	Operation Performed
Memory Reference (Continued)	MOV	RW,DADDR	8B aadddbbb [DISP][DISP]	8+EA										[RW] ← [EA] Load 16 bits of data from the data memory word addressed by DADDR to register RW
	MOV	DADDR,RB	88 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RB] Store the data byte from register RB in the memory byte addressed by DADDR
	MOV	DADDR,RW	89 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RW] Store the 16-bit data word from register RW in the memory word addressed by DADDR
	MOV	AL,LABEL	A0 PPQQ	10										[AL] ← [EA] Load the data memory byte directly addressed by LABEL into register AL
	MOV	AX,LABEL	A1 PPQQ	10										[AX] ← [EA] Load the 16-bit data memory word directly addressed by LABEL into register AX
	MOV	LABEL,AL	A2 PPQQ	10										[EA] ← [AL] Store the 8-bit contents of register AL into the data memory byte directly addressed by LABEL
	MOV	LABEL,AX	A3 PPQQ	10										[EA] ← [AX] Store the 16-bit contents of register AX into the data memory word directly addressed by LABEL
	MOV	SR,DADDR	8E aa0rrbbb [DISP][DISP]	8+EA										[SR] ← [EA] Load into Segment register SR the contents of the 16-bit memory word addressed by DADDR
	MOV	DADDR,SR	8C aa0rrbbb [DISP][DISP]	9+EA										[EA] ← [SR] Store the contents of Segment register SR in the 16-bit memory location addresed by DADDR

5-61

Veri Aktarım Komutları - MOV

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses O	D	I	T	S	Z	A	P	C	Operation Performed
Memory Refrence (Continued)	MOV	RW,DADDR	8B aadddbbb [DISP][DISP]	8+EA										[RW] ← [EA] Load 16 bits of data from the data memory word addressed by DADDR to register RW
	MOV	DADDR,RB	88 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RB] Store the data byte from register RB in the memory byte addressed by DADDR
	MOV	DADDR,RW	89 aasssbbb [DISP][DISP]	9+EA										[EA] ← [RW] Store the 16-bit data word from register RW in the memory word addressed by DADDR
	MOV	AL,LABEL	A0 PPQQ	10										[AL] ← [EA] Load the data memory byte directly addressed by LABEL into register AL
	MOV	AX,LABEL	A1 PPQQ	10										[AX] ← [EA] Load the 16-bit data memory word directly addressed by LABEL into register AX
	MOV	LABEL,AL	A2 PPQQ	10										[EA] ← [AL] Store the 8-bit contents of register AL into the data memory byte directly addressed by LABEL
	MOV	LABEL,AX	A3 PPQQ	10										[EA] ← [AX] Store the 16-bit contents of register AX into the data memory word directly addressed by LABEL
	MOV	SR,DADDR	8E aa0rrbbb [DISP][DISP]	8+EA										[SR] ← [EA] Load into Segment register SR the contents of the 16-bit memory word addressed by DADDR
	MOV	DADDR,SR	8C aa0rrbbb [DISP][DISP]	9+EA										[EA] ← [SR] Store the contents of Segment register SR in the 16-bit memory location addresed by DADDR

5-61

MOV AX,L1 ←

MOV L1,AL ← 1 byte

MOV L1,AX → 2 byte

MOV CS,tm? ←

CS, DS, SS, ES

Veri Aktarım Komutları - MOV (Devam)

Immediate	MOV	DADDR, DATA8	C6 aa000bbb [DISP][DISP] YY	10+EA									[EA] ← DATA8 Load the immediate data byte DATA8 into the data memory byte addressed by DADDR
	MOV	DADDR, DATA16	C7 aa000bbb [DISP][DISP] YYYY	10+EA									[EA] ← DATA16 Load the immediate 16-bit data word DATA16 into the data memory word addressed by DADDR
	MOV	RB,DATA8	10110ddd YY	4*									[RB] ← DATA8 Load the immediate data byte DATA8 into 8-bit register RB
	MOV	RW,DATA16	10111ddd YYYY	4*									[RW] ← DATA16 Load the immediate 16-bit data word DATA16 into 16-bit register RW
Register Move	MOV	RBD,RBS	8A11dddsss	2*									[RBD] ← [RBS] Move the contents of any RB register to any RB register
	MOV	RWD,RWS	8B 11dddsss	2*									[RWD] ← [RWS] Move the contents of any RW register to any RW register
	MOV	SR,RW	8E 110rrsss	2*									[SR] ← [RWS] Move the contents of any RW register to any Segment register
	MOV	RW,SR	8C 110rrddd	2*									[RWD] ← [SR] Move the contents of any Segment register to any RW register

Veri Aktarım Komutları - MOV (Devam)

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

Immediate	MOV	DADDR, DATA8	C6 aa000bbb [DISP][DISP] YY	10+EA											[EA] ← DATA8 Load the immediate data byte DATA8 into the data memory byte addressed by DADDR
	MOV	DADDR, DATA16	C7 aa000bbb [DISP][DISP] YYYY	10+EA											[EA] ← DATA16 Load the immediate 16-bit data word DATA16 into the data memory word addressed by DADDR
	MOV	RB,DATA8	10110ddd YY	4*											[RB] ← DATA8 Load the immediate data byte DATA8 into 8-bit register RB
	MOV	RW,DATA16	10111ddd YYYY	4*											[RW] ← DATA16 Load the immediate 16-bit data word DATA16 into 16-bit register RW
Register Move	MOV	RBD,RBS	8A11dddsss	2*											[RBD] ← [RBS] Move the contents of any RB register to any RB register
	MOV	RWD,RWS	8B 11dddsss	2*											[RWD] ← [RWS] Move the contents of any RW register to any RW register
	MOV	SR,RW	8E 110rrsss	2*											[SR] ← [RWS] Move the contents of any RW register to any Segment register
	MOV	RW,SR	8C 110rrddd	2*											[RWD] ← [SR] Move the contents of any Segment register to any RW register

Veri Aktarım Komutları - MOV (Devam)

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

Immediate	MOV	DADDR, DATA8	C6 aa000bbb [DISP][DISP] YY	10+EA										[EA] ← DATA8 Load the immediate data byte DATA8 into the data memory byte addressed by DADDR
	MOV	DADDR, DATA16	C7 aa000bbb [DISP][DISP] YYYY	10+EA										[EA] ← DATA16 Load the immediate 16-bit data word DATA16 into the data memory word addressed by DADDR
	MOV	RB,DATA8	10110ddd YY	4*										[RB] ← DATA8 Load the immediate data byte DATA8 into 8-bit register RB
	MOV	RW,DATA16	10111ddd YYYY	4*										[RW] ← DATA16 Load the immediate 16-bit data word DATA16 into 16-bit register RW
Register Move	MOV	RBD,RBS	8A11dddsss	2*										[RBD] ← [RBS] Move the contents of any RB register to any RB register
	MOV	RWD,RWS	8B 11dddsss	2*										[RWD] ← [RWS] Move the contents of any RW register to any RW register
	MOV	SR,RW	8E 110rrsss	2*										[SR] ← [RWS] Move the contents of any RW register to any Segment register
	MOV	RW,SR	8C 110rrddd	2*										[RWD] ← [SR] Move the contents of any Segment register to any RW register

Veri Aktarım Komutları - LEA, LDS, LES

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

LDS	RW,DADDR	C5 aasssbbb [DISP][DISP]	16+EA											
LEA	RW,DADDR	8D aasssbbb [DISP][DISP]	2+EA											
LES	RW,DADDR	C4 aasssbbb [DISP][DISP]	16+EA											

[RW] ← [EA], [DS] ← [EA+2]

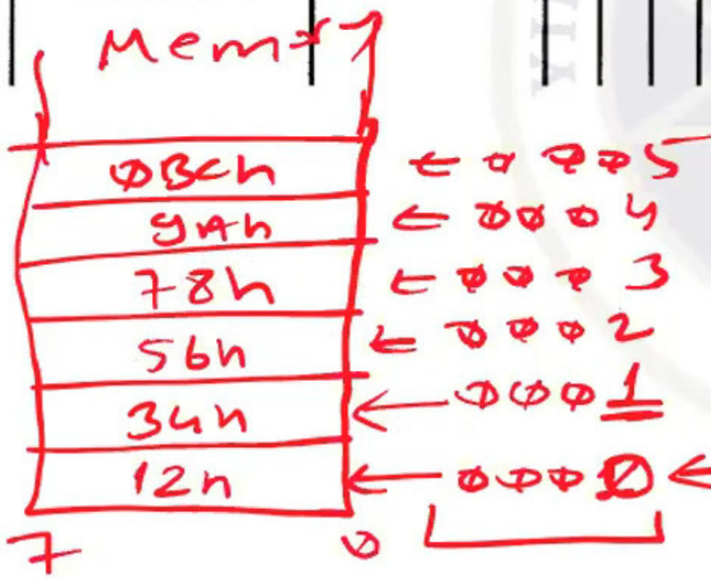
Load 16 bits of data from the memory word addressed by DADDR into register RW. Load 16 bits of data from the next sequential memory word into the DS register

[RW] ← OEA

Load into RW the 16-bit address displacement which, when added to the segment register contents, creates the effective data memory address

[RW] ← [EA], [ES] ← [EA+2]

Load 16 bits of data from the memory word addressed by DADDR into register RW. Load 16 bits of data from the next sequential memory word into the ES register




```
LDS AX, my_var
; AX ← 3412h
; DS ← 7856h
```

Veri Aktarım Komutları - LEA, LDS, LES

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

LDS	RW,DADDR	C5 aasssbbb [DISP][DISP]	16+EA										[RW] ← [EA], [DS] ← [EA+2] Load 16 bits of data from the memory word addressed by DADDR into register RW. Load 16 bits of data from the next sequential memory word into the DS register
LEA	RW,DADDR	8D aasssbbb [DISP][DISP]	2+EA										[RW] ← OEA Load into RW the 16-bit address displacement which, when added to the segment register contents, creates the effective data memory address
LES	RW,DADDR	C4 aasssbbb [DISP][DISP]	16+EA										[RW] ← [EA], [ES] ← [EA+2] Load 16 bits of data from the memory word addressed by DADDR into register RW. Load 16 bits of data from the next sequential memory word into the ES register

LEA SI, dizi → 7

dizi → 0ABCDH

MOV SI, dizi

; SI ← ABCD

Veri Aktarım Komutları - XCHG

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

XCHG	RB,DADDR	86 aaregbbb [DISP][DISP]	17+EA	[RB] ⟷ [EA] Exchange a byte of data between register RB and the data memory location addressed by DADDR
XCHG	RW,DADDR	87 aaregbbb [DISP][DISP]	17+EA	[RW] ⟷ [EA] Exchange 16 bits of data between register RW and the data memory location addressed by DADDR
XCHG	AX,RW	10010reg	3*	[AX] ⟷ [RW] Exchange the contents of AX and any RW register
XCHG	RB,RB	86 11regreg	4*	[RB] ⟷ [RB] Exchange the contents of any two RB registers
XCHG	RW,RW	87 11regreg	4*	[RW] ⟷ [RW] Exchange the contents of any two RW registers

A B (swap)

7 15

15 7

int a=7, b=15;

int tmp;

tmp = a

a = b

b = tmp

XCHG a,b

DADDR, DADDR

Veri Aktarım Komutları - XCHG

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

XCHG	RB,DADDR	86 aaregbbb [DISP][DISP]	17+EA	[RB] ⟷ [EA] Exchange a byte of data between register RB and the data memory location addressed by DADDR
XCHG	RW,DADDR	87 aaregbbb [DISP][DISP]	17+EA	[RW] ⟷ [EA] Exchange 16 bits of data between register RW and the data memory location addressed by DADDR
XCHG	AX,RW	10010reg	3*	[AX] ⟷ [RW] Exchange the contents of AX and any RW register
XCHG	RB,RB	86 11regreg	4*	[RB] ⟷ [RB] Exchange the contents of any two RB registers
XCHG	RW,RW	87 11regreg	4*	[RW] ⟷ [RW] Exchange the contents of any two RW registers

```
MOV AX, 5
MOV BX, 6
XCHG BX, AX   :)
```

Aritmetik Komutlar

BLM20
Alt Seviye
Programlama

Hafta 2

Furkan ÇAK

Aritmetik Komutlar		
ADD	INC	MUL
ADC	DEC	IMUL
SUB	NEG	DIV
SBB	CMP	IDIV

Aritmetik Komutlar- ADD

BLM20
Alt Seviye Programlama

Hafta 2

Furkan ÇAK

ADD	RB,DADDR	02 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RB] ← [EA] + [RB] Add the contents of the data byte addressed by DADDR to register RB
ADD	RW,DADDR	03 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RW] ← [EA] + [RW] Add the contents of the 16-bit word addressed by DADDR to register RW
ADD	DADDR,RB	00 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RB] Add the 8-bit contents of register RB to the data memory byte addressed by DADDR
ADD	DADDR,RW	01 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RW] Add the 16-bit contents of register RW to the data memory word addressed
ADD	AL,DATA8	04 YY	4*	X				X	X	X	X	X	[AL] ← [AL] + DATA8 Add 8-bit immediate data to the AL register
ADD	AX,DATA16	05 YYYY	4*	X				X	X	X	X	X	[AX] ← [AX] + DATA16 Add 16-bit immediate data to the AX register
ADD	RB,DATA8	80 11000ddd YY	4*	X				X	X	X	X	X	[RB] ← [RB] + DATA8 Add 8-bit immediate data to the RB register
ADD	RW,DATA16	81 11000ddd YYYY	4*	X				X	X	X	X	X	[RW] ← [RW] + DATA16 Add 16-bit immediate data to the RW register
ADD	DADDR, DATA8	80 aa000bbb [DISP][DISP] YY	17+EA	X				X	X	X	X	X	[EA] ← [EA] + DATA8 Add 8-bit immediate data to the data memory byte addressed by DADDR
ADD	DADDR, DATA16	81 aa000bbb [DISP][DISP] YYYY	17+EA	X				X	X	X	X	X	[EA] ← [EA] + DATA16 Add 16-bit immediate data to the data memory word addressed by DADDR
ADD	RBD,RBS	02 11dddsss	3*	X				X	X	X	X	X	[RBD] ← [RBD] + [RBS] Add the 8-bit contents of register RBS to register RBD
ADD	RWD,RWS	03 11dddsss	3*	X				X	X	X	X	X	[RWD] ← [RWD] + [RWS]

Aritmetik Komutlar- ADC

Furkan ÇAK

AX ← 17
BX ← 4
CF ← set

ADC AX, BX
↙
AX ← AX + BX + CF

ADC	RB,DADDR	12 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RB] ← [EA] + [RB] + [C] Add the contents of the data byte addressed by DADDR, plus the Carry status, to register RB
ADC	RW,DADDR	13 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RW] ← [EA] + [RW] + [C] Add the contents of the 16-bit data word addressed by DADDR, plus the Carry status, to register RW
ADC	DADDR,RB	10 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RB] + [C] Add the 8-bit contents of register RB, plus the Carry status, to the data memory byte addressed by DADDR
ADC	DADDR,RW	11 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RW] + [C] Add the 16-bit contents of register RW, plus the Carry status, to the data word addressed by DADDR
ADC	AL,DATA8	14 YY	4*	X				X	X	X	X	X	[AL] ← [AL] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the AL register
ADC	AX,DATA16	15 YYYY	4*	X				X	X	X	X	X	[AX] ← [AX] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the AX register
ADC	B,DATA8	80 11010ddd YY	4*	X				X	X	X	X	X	[RB] ← [RB] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the RB register
ADC	RW,DATA16	81 11010ddd YYYY	4*	X				X	X	X	X	X	[RW] ← [RW] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the RW register
ADC	DADDR, DATA8	80 aa010bbb [DISP][DISP] YY	17+EA	X				X	X	X	X	X	[EA] ← [EA] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the data memory byte addressed by DADDR
ADC	DADDR, DATA16	81 aa010bbb [DISP][DISP] YYYY	17+EA	X				X		X	X	X	[EA] ← [EA] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the data memory word addressed by DADDR
ADC	RBD,RBS	12 11dddsss	3*	X				X	X	X	X	X	[RBD] ← [RBD] + [RBS] + [C] Add the 8-bit contents of register RBS, plus the Carry status, to register RBD
ADC	RWD,RWS	13 11dddsss	3*	X				X	X	X	X	X	[RWD] ← [RWD] + [RWS] + [C]

Aritmetik Komutlar- ADC

Furkan ÇAK

MOV BL, 0FAh
MOV AL, 64h

ADD BL, AL
↓
5E

CF 1 FA
64
5E

ADC	RB,DADDR	12 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RB] ← [EA] + [RB] + [C] Add the contents of the data byte addressed by DADDR, plus the Carry status, to register RB
ADC	RW,DADDR	13 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RW] ← [EA] + [RW] + [C] Add the contents of the 16-bit data word addressed by DADDR, plus the Carry status, to register RW
ADC	DADDR,RB	10 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RB] + [C] Add the 8-bit contents of register RB, plus the Carry status, to the data memory byte addressed by DADDR
ADC	DADDR,RW	11 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RW] + [C] Add the 16-bit contents of register RW, plus the Carry status, to the data word addressed by DADDR
ADC	AL,DATA8	14 YY	4*	X				X	X	X	X	X	[AL] ← [AL] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the AL register
ADC	AX,DATA16	15 YYYY	4*	X				X	X	X	X	X	[AX] ← [AX] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the AX register
ADC	B,DATA8	80 11010ddd YY	4*	X				X	X	X	X	X	[RB] ← [RB] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the RB register
ADC	RW,DATA16	81 11010ddd YYYY	4*	X				X	X	X	X	X	[RW] ← [RW] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the RW register
ADC	DADDR, DATA8	80 aa010bbb [DISP][DISP] YY	17+EA	X				X	X	X	X	X	[EA] ← [EA] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the data memory byte addressed by DADDR
ADC	DADDR, DATA16	81 aa010bbb [DISP][DISP] YYYY	17+EA	X				X		X	X	X	[EA] ← [EA] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the data memory word addressed by DADDR
ADC	RBD,RBS	12 11dddsss	3*	X				X	X	X	X	X	[RBD] ← [RBD] + [RBS] + [C] Add the 8-bit contents of register RBS, plus the Carry status, to register RBD
ADC	RWD,RWS	13 11dddsss	3*	X				X	X	X	X	X	[RWD] ← [RWD] + [RWS] + [C]

Aritmetik Komutlar- ADC

BLM20 Furkan ÇAK
Alt Seviye Programlama

Hafta 2

ADC	RB,DADDR	12 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RB] ← [EA] + [RB] + [C] Add the contents of the data byte addressed by DADDR, plus the Carry status, to register RB
ADC	RW,DADDR	13 aadddbbb [DISP][DISP]	9+EA	X				X	X	X	X	X	[RW] ← [EA] + [RW] + [C] Add the contents of the 16-bit data word addressed by DADDR, plus the Carry status, to register RW
ADC	DADDR,RB	10 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RB] + [C] Add the 8-bit contents of register RB, plus the Carry status, to the data memory byte addressed by DADDR
ADC	DADDR,RW	11 aasssbbb [DISP][DISP]	16+EA	X				X	X	X	X	X	[EA] ← [EA] + [RW] + [C] Add the 16-bit contents of register RW, plus the Carry status, to the data word addressed by DADDR
ADC	AL,DATA8	14 YY	4*	X				X	X	X	X	X	[AL] ← [AL] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the AL register
ADC	AX,DATA16	15 YYYY	4*	X				X	X	X	X	X	[AX] ← [AX] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the AX register
ADC	B,DATA8	80 11010ddd YY	4*	X				X	X	X	X	X	[RB] ← [RB] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the RB register
ADC	RW,DATA16	81 11010ddd YYYY	4*	X				X	X	X	X	X	[RW] ← [RW] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the RW register
ADC	DADDR, DATA8	80 aa010bbb [DISP][DISP] YY	17+EA	X				X	X	X	X	X	[EA] ← [EA] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the data memory byte addressed by DADDR
ADC	DADDR, DATA16	81 aa010bbb [DISP][DISP] YYYY	17+EA	X				X		X	X	X	[EA] ← [EA] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the data memory word addressed by DADDR
ADC	RBD,RBS	12 11dddsss	3*	X				X	X	X	X	X	[RBD] ← [RBD] + [RBS] + [C] Add the 8-bit contents of register RBS, plus the Carry status, to register RBD
ADC	RWD,RWS	13 11dddsss	3*	X				X	X	X	X	X	[RWD] ← [RWD] + [RWS] + [C]

MOV BL, 0FAh
MOV AL, 64h
ADD BL, AL
5E
MOV BH, 0
ADC BH, 0
BH ← BH + 0 + CF
1
BX
BH BL
01 5E

Aritmetik Komutlar- SUB

BLM20 Alt Seviye Programlama

Hafta 2

Furkan ÇAK

SUB → Op1 , Op2

Op1 ← Op1 - Op2

MOV AX, 7

MOV Saji, 9

SUB AX, Saji

↓

; AX ← -2

SUB	RB,DADDR	2A aadddbbb [DISP][DISP]	9+EA	X			X	X	X	X	X	[RB] ← [RB] – [EA] Subtract the contents of the data memory byte addressed by DADDR from the contents of 8-bit register RB, using twos complement arithmetic
SUB	RW,DADDR	2B aadddbbb [DISP][DISP]	9+EA	X			X	X	X	X	X	[RW] ← [RW] – [EA] Subtract the contents of the 16-bit data memory word addressed by DADDR from the contents of 16-bit register RW, using twos complement arithmetic
SUB	DADDR,RB	28 aasssbbb [DISP][DISP]	16+EA	X			X	X	X	X	X	[EA] ← [EA] – [RB] Subtract the contents of 8-bit register RB from the data memory byte addressed by DADDR, using twos complement arithmetic
SUB	DADDR,RW	29 aasssbbb [DISP][DISP]	16+EA	X			X	X	X	X	X	[EA] ← [EA] – [RW] Subtract the contents of 16-bit register RW from the 16-bit data memory word addressed by DADDR, using twos complement arithmetic
SUB	AL,DATA8	2C YY	4*	X			X	X	X	X	X	[AL] ← [AL] – DATA8 Subtract the 8-bit immediate signed binary data from AL register contents using twos complement arithmetic
SUB	AX,DATA16	2D YYYY	4*	X			X	X	X	X	X	[AX] ← [AX] – DATA16 Subtract the 16-bit immediate signed binary data from AX register contents using twos complement arithmetic
SUB	RB,DATA8	80 11101ddd YY	4*	X			X	X	X	X	X	[RB] ← [RB] – DATA8 Subtract the 8-bit immediate signed binary data from RB register contents using twos complement arithmetic
SUB	RW,DATA16	81 11101ddd YYYY	4*	X			X	X	X	X	X	[RW] ← [RW] – DATA16 Subtract the 16-bit immediate signed binary data from RW register contents using twos complement arithmetic
SUB	DADDR, DATA8	80 aa101bbb [DISP][DISP] YY	17+EA	X			X	X	X	X	X	[EA] ← [EA] – DATA8 Subtract the 8-bit immediate signed binary data from the contents of the data memory byte addressed by DADDR using twos complement arithmetic
SUB	DADDR, DATA16	100000a1 aa101bbb [DISP][DISP]YY[YY]	17+EA	X		0	X	X	X	X	X	[EA] ← [EA] – DATA16 Subtract the 16-bit immediate signed binary data from the contents of the 16-bit data memory word addressed by DADDR using twos complement arithmetic
SUB	RBD,RBS	2A 11dddsss	3*	X			X	X	X	X	X	[RBD] ← [RBD] – [RBS] Subtract the 8-bit contents of register RBS from RBD using twos complement arithmetic
SUB	RWD,RWS	2B 11dddsss	3*	X			X	X	X	X	X	[RWD] ← [RWD] – [RWS] Subtract the 16-bit contents of register RWS from RWD using twos complement arithmetic

Aritmetik Komutlar- SBB

BLM20 Furkan ÇAK
Alt Seviye Programlama

Hafta 2

ADC

SBB

MOV AL, 7

MOV Sayi, -5

SUB AL, Sayi

SBB AH, 0

AH ← AH - 0 - CF

SBB	RB,DADDR	1A aaddd bbb [DISP][DISP]	9+EA	X			X	X	X	X	X	[RB] ← [RB] – [EA] – [C] Subtract the contents of the data byte addressed by DADDR from the contents of 8-bit register RB, using twos complement arithmetic. Decrement the result in RB if the Carry status was initially set
SBB	RW,DADDR	1B aadddbbb [DISP][DISP]	9+EA	X			X	X	X	X	X	[RW] ← [RW] – [EA] – [C] Subtract the contents of the 16-bit data word addressed by DADDR from the contents of the 16-bit register RW, using twos complement arithmetic. Decrement the result in RW if the Carry status was initially set
SBB	DADDR,RB	18 aasssbbb [DISP][DISP]	16+EA	X			X	X	X	X	X	[EA] ← [EA] – [RB] – [C] Subtract the contents of 8-bit register RB from the data byte addressed by DADDR, using twos complement arithmetic. Decrement the result in data memory if the Carry status was initially set
SBB	DADDR,RW	19 aasssbbb [DISP][DISP]	16+EA	X			X	X	X	X	X	[EA] ← [EA] – [RW] – [C] Subtract the contents of 16-bit register RW from the 16-bit data word addressed by DADDR, using twos complement arithmetic. Decrement the result in data memory if the Carry status was initially set
SBB	AL,DATA8	1C YY	4*	X			X	X	X	X	X	[AL] ← [AL] – DATA8 – [C] Subtract 8-bit immediate signed binary data from AL register contents using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	AX,DATA16	1D YYYY	4*	X			X	X	X	X	X	[AX] ← [AX] – DATA16 – [C] Subtract 16-bit immediate signed binary data from AX register contents using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	RB,DATA8	80 11011ddd YY	4*	X			X	X	X	X	X	[RB] ← [RB] – DATA8 – [C] Subtract 8-bit immediate signed binary data from RB register contents using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	RW,DATA16	100000a1 11011ddd YY [YY]	4*	X			X	X	X	X	X	[RW] ← [RW] – DATA16 – [C] Subtract 16-bit immediate signed binary data from RW register contents using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	DADDR, DATA8	80 aa011bbb [DISP][DISP] YY	17+EA	X			X	X	X	X	X	[EA] ← [EA] – DATA8 – [C] Subtract 8-bit immediate signed binary data from contents of data memory byte addressed by DADDR using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	DADDR, DATA16	100000a1 aa011bbb [DISP][DISP] YY [YY]	17+EA	X			X	X	X	X	X	[EA] ← [EA] – DATA16 – [C] Subtract 16-bit immediate signed binary data from contents of 16-bit data memory word addressed by DADDR using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	RBD,RBS	1A 11dddsss	3*	X			X	X	X	X	X	[RBD] ← [RBD] – [RBS] – [C] Subtract the 8-bit contents of register RBS from RBD using twos complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	RWD,RWS	1B 11dddsss	3*	X			X	X	X	X	X	[RWD] ← [RWD] – [RWS] – [C] Subtract the 16-bit contents of register RWS from RWD using twos complement arithmetic. If the Carry status was originally 1 decrement the result

Aritmetik Komutlar- INC, DEC, NEG

BLM20
Alt Seviye Programlama

Hafta 2

Furkan ÇAK

MOV AX, 17h

INC AX → i++ → 17h + 1 = 18h

ADD AX, 1 → i = i + 1

DEC AX 17 - 1 = 16h

Aritmetik Komutlar- INC, DEC, NEG

BLM20
Alt Seviye Programlama

Hafta 2

Furkan ÇAK

NEG → Two's complement

6Eh → 6 E

MOV DH, 01101110b

NEG DH 10010001b → First C.

1

; DH = 92h 10010010b

9 2

Aritmetik Komutlar- CMP

Furkan ÇAK

CMP AX, BX → flags → JXXX
→ SMP

SUB AX, BX ← AX ← AX - BX

Aritmetik Komutlar

Alt Seviye Programlama

Hafta 2

Aritmetik Komutlar		
ADD	INC	MUL
ADC	DEC	IMUL
SUB	NEG	DIV
SBB	CMP	IDIV

Aritmetik Komutlar- MUL, DIV, IMUL, IDIV

MUL Op1
Byte
AX ← AL x Op1

MOV AL,3
MOV DL,5
MUL DL ;
; AX ← 15

MUL Op
word
DX:AX ← AX * Op

MUL AX,BX

AL*DL → AX
AX ← AX*BX
DX : AX
High Low
0FFFFh
0FFDDh

MOV DX, 17h ?
PUSH DX
MOV AX, 5
MOV BX, 2
MUL BX
POP DX
; BX→2, AX→10
; DX→0

DX:12h
15 0

Alt Seviye Programlama

Hafta 2

Aritmetik Komutlar- MUL, DIV, IMUL, IDIV

MOV AX, 0ffffh

MOV BX, 1000h

BX: 0001 0000 0000 0000b

AX: 1111 1111 1111 1111b

IMUL

Aritmetik Komutlar- MUL, DIV, IMUL, IDIV

DIV Op

Byte

Bölünen → AX | Op → Bölen

AL → Bölüm

AH → Kalan

```
MOV AX,11h
MOV AL,0AAh
MOV CL, 2h
DIV CL
```

Ax:11AA | 2

AL

AH

DIV Op

word

DX:AX | Op

AX

DX

Aritmetik Komutlar- MUL, DIV, IMUL, IDIV

Alt Seviye Programlama

Hafta 2

DIV Op

Byte

Bölünen → AX | Op → Bölen

AL → Bölüm

AH → Kalan

MOV CX, 0100h → 256

SUB DX, DX

MOV AX, 1000h → ?

DIV CX

1000h | 256

16

DX : 0

DIV Op

word

0100h → 256

DX:AX | Op

AX

DX

Alt Seviye Programlama

Hafta 2

Aritmetik Komutlar- MUL, DIV, IMUL, IDIV

sayi → 1234ABCDh

High: DX — Low: AX

DX:AX → #ABCDh

```
MOV AX, WORD PTR [sayi]
MOV DX, WORD PTR [sayi+2]
```

DX → 1234h

Alt Seviye Programlama

Hafta 2

Dallanma Komutları

- Programın akışını değiştiren komutlardır.
- IP yazmacının değerini değiştirirler.
- Koşullara bağlı veya herhangi bir koşul olmaksızın dallanma gerçekleştirilebilir.
- Koşullu dallanma komutları genellikle CMP ile birlikte kullanılır.
- Farklı isimlerdeki komutlar (mnemonic) aynı anlama gelebilmektedir.

MOV AX, 2
MOV BX, 2

CMP AX, BX

AX - BX

ZF E

Alt Seviye Programlama

Hafta 2

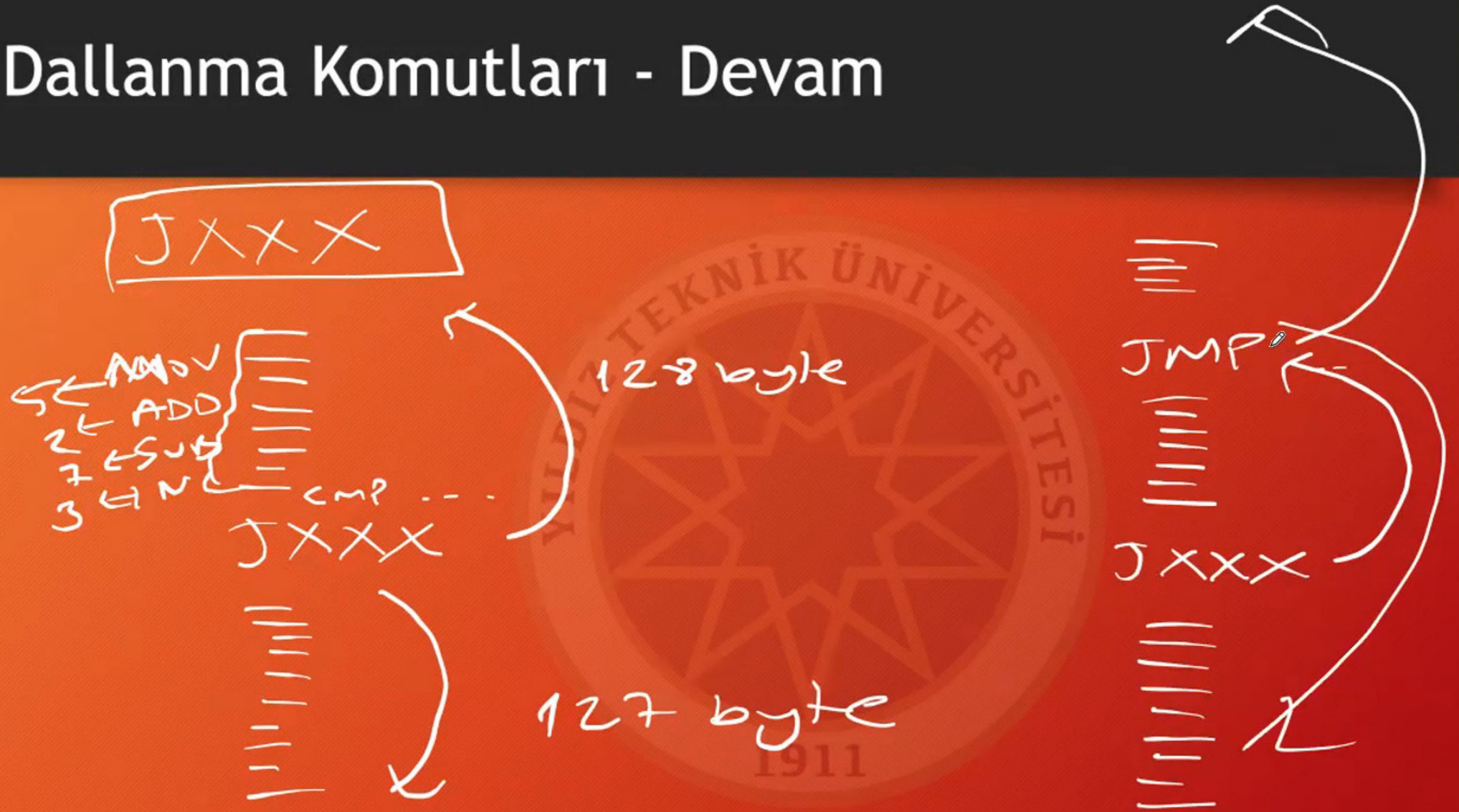
Dallanma Komutları

- Programın akışını değiştiren komutlardır.
- IP yazmacının değerini değiştirirler.
- Koşullara bağlı veya herhangi bir koşul olmaksızın dallanma gerçekleştirilebilir.
- Koşullu dallanma komutları genellikle CMP ile birlikte kullanılır.
- Farklı isimlerdeki komutlar (mnemonic) aynı anlama gelebilmektedir.
- Koşullu dallanam komutlarının aralığı 8-bit ile sınırlıdır.
  - 128 byte geri veya 127 byte ileri zıplanabilir.)

Alt Seviye Programlama

Hafta 2

Dallanma Komutları - Devam



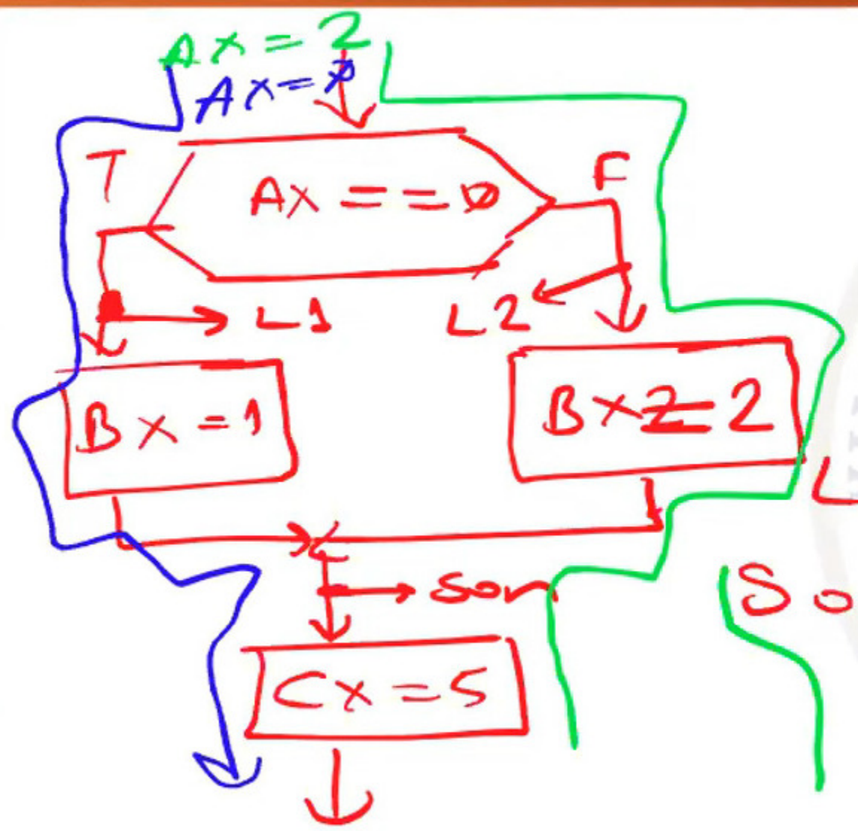
Alt Seviye Programlama

Hafta 2

Dallanma Komutları - Devam

- Dallanma Komutları
  - Koşulsuz Dallanma Komutları
    - JMP
  - Koşullu Dallanma Komutları
    - Basit Koşullu Dallanma Komutları
      - JZ/JE
      - JNZ/JNE
      - JS
      - JNS
      - JO
      - JNO
      - JP/JPE
      - JNP/JPO
    - Birden Fazla Bayrağın Durumuna Göre Dallanan Komutlar
      - İşaretsiz Sayılar için Dallanma Komutları (**A**bove/**B**elow)
        - JB/JNAE/JC
        - JA/JNBE
        - JAE/JNB/JNC
        - JBE/JNA
      - İşaretli Sayılar için Dallanma Komutları (**G**reater/**L**ess)
        - JL/JNGE
        - JNL/JGE
        - JLE/JNG
        - JG/JNLE

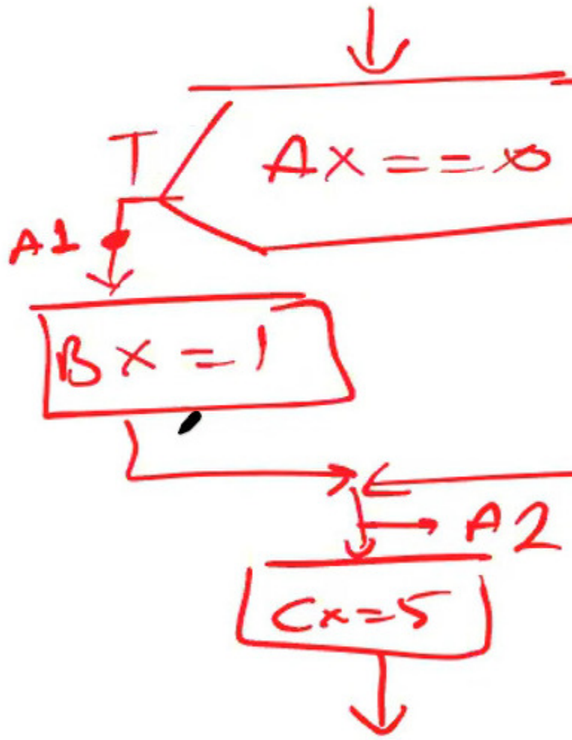
Dallanma Örnekleri JZ




```
CMP AX, 0
JE L1
MOV BX, 2
JMP Son
L1: MOV BX, 1
Son: MOV CX, 5
```

Furkan ÇAKMAK
Alt Seviye Programlama
Hafta 2

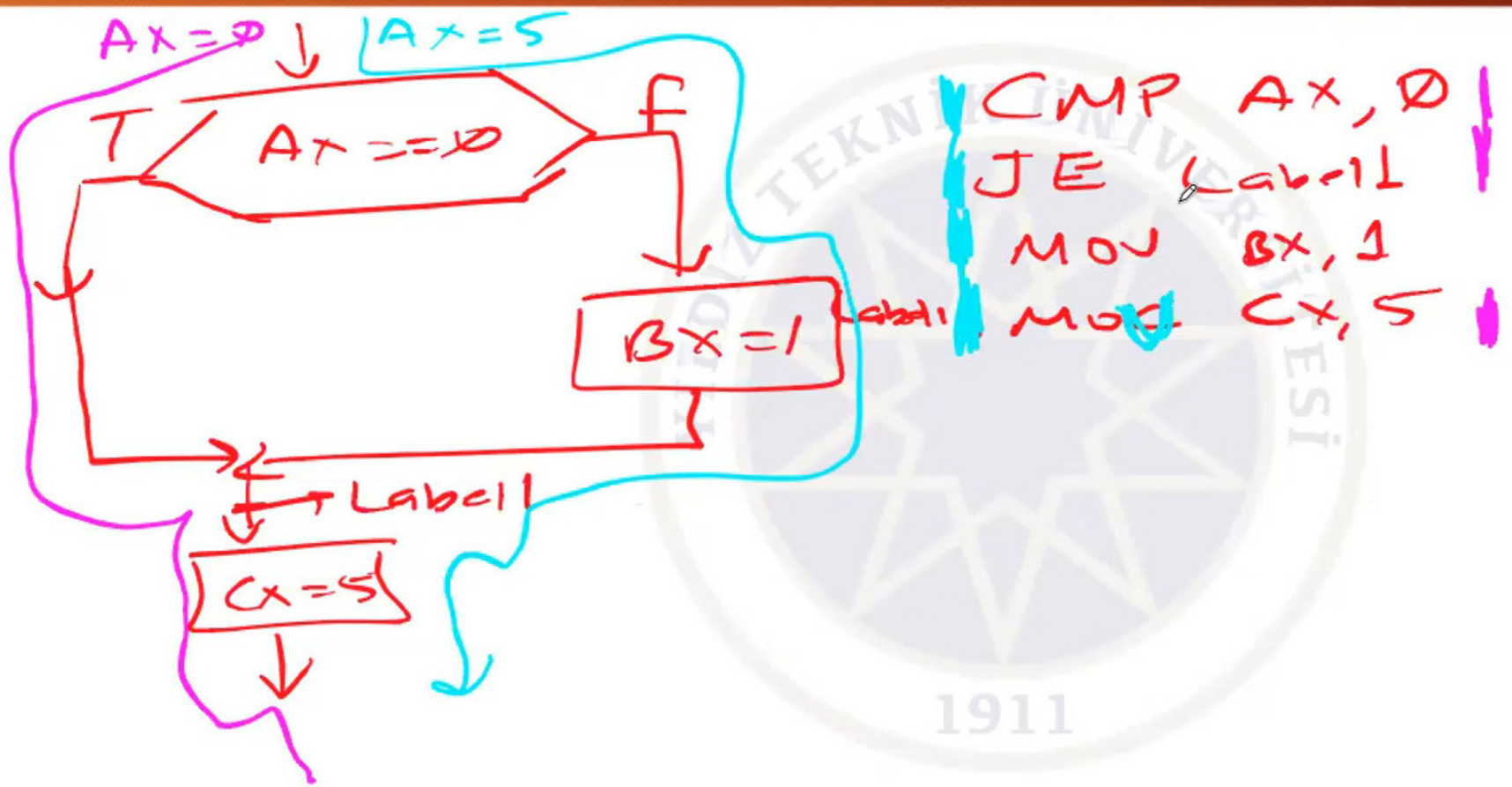
Dallanma Örnekleri




```
CMP AX,0
JE A1
JMP A2
A1: MOV BX,1
A2: MOV CX,5
```

```
CMP AX,0
JNE A2
MOV BX,1
A2: MOV CX,5
```

Dallanma Örnekleri



Alt Seviye Programlama

Hafta 3

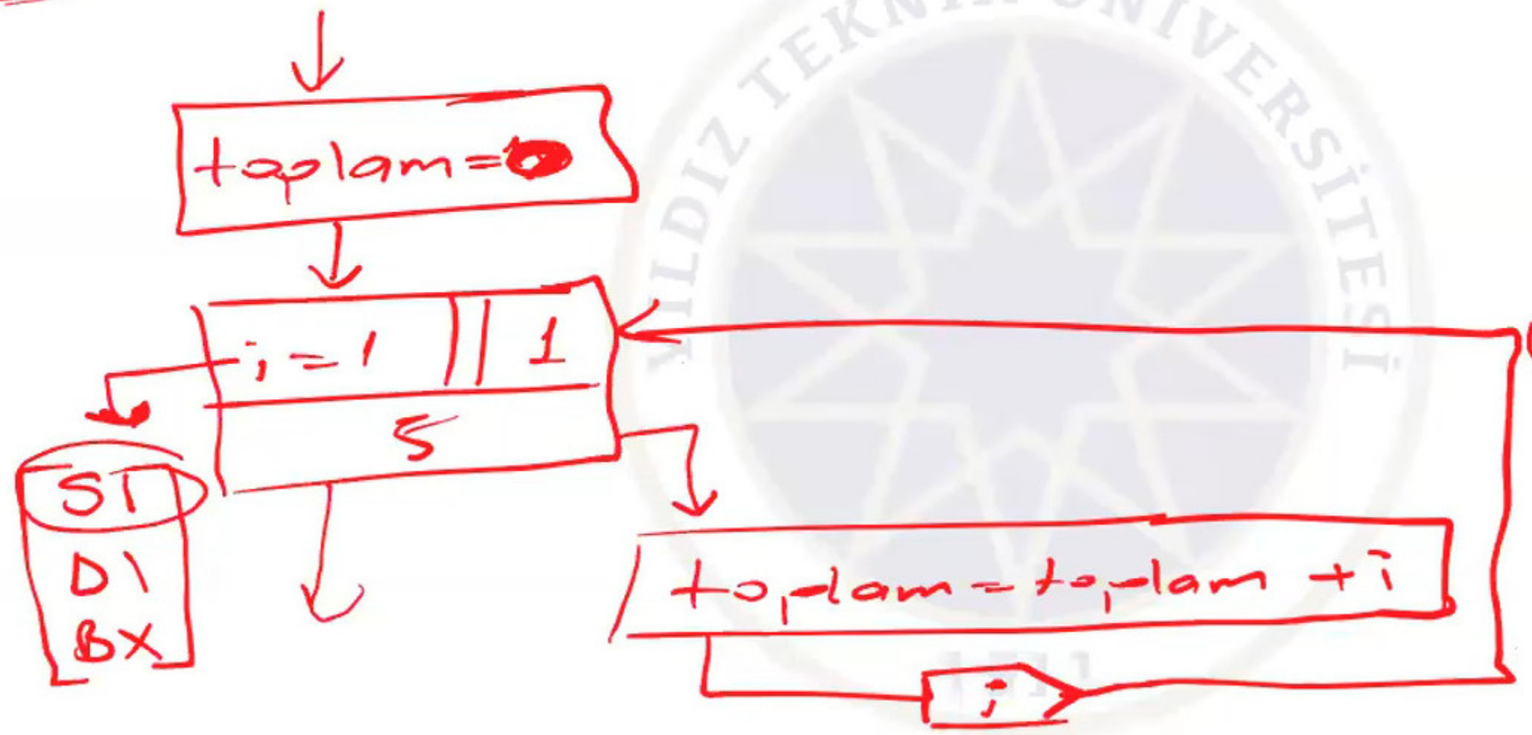
Ders Tanıtım Formu ve Konular

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

Döngü Komutu - LOOP

MOV DADDR, idata

CX ← 10




```
        MOV toplam, 0
        MOV CX, 5
        MOV SI, 1
L1:     ADD toplam, SI
        INC SI
        LOOP L1
```

Alt Seviye Programlama

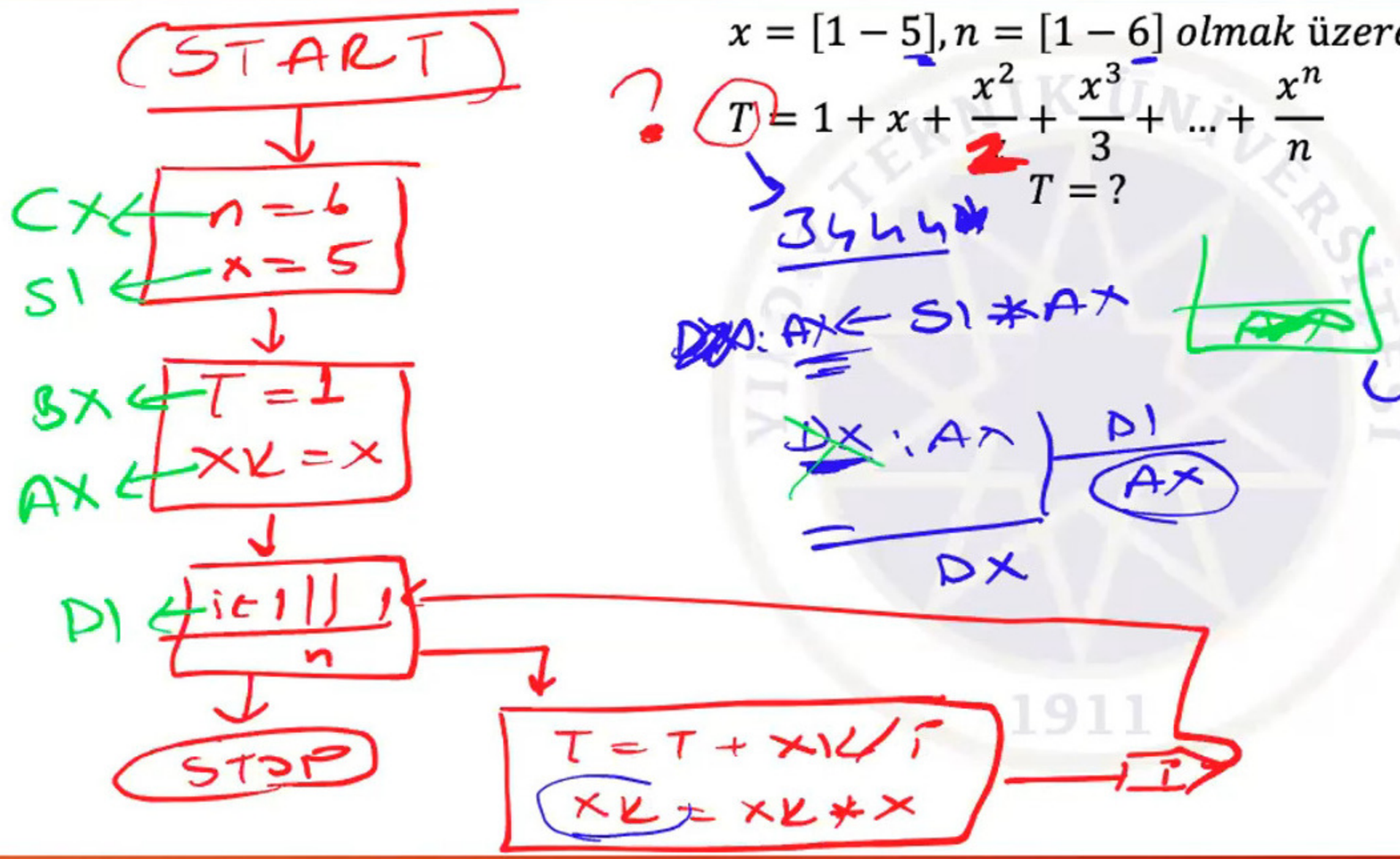
Hafta 3

LOOP Örneği 1

$x = [1-5], n = [1-6]$ *olmak üzere;*

$$T = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \ldots + \frac{x^n}{n}$$

$T = ?$

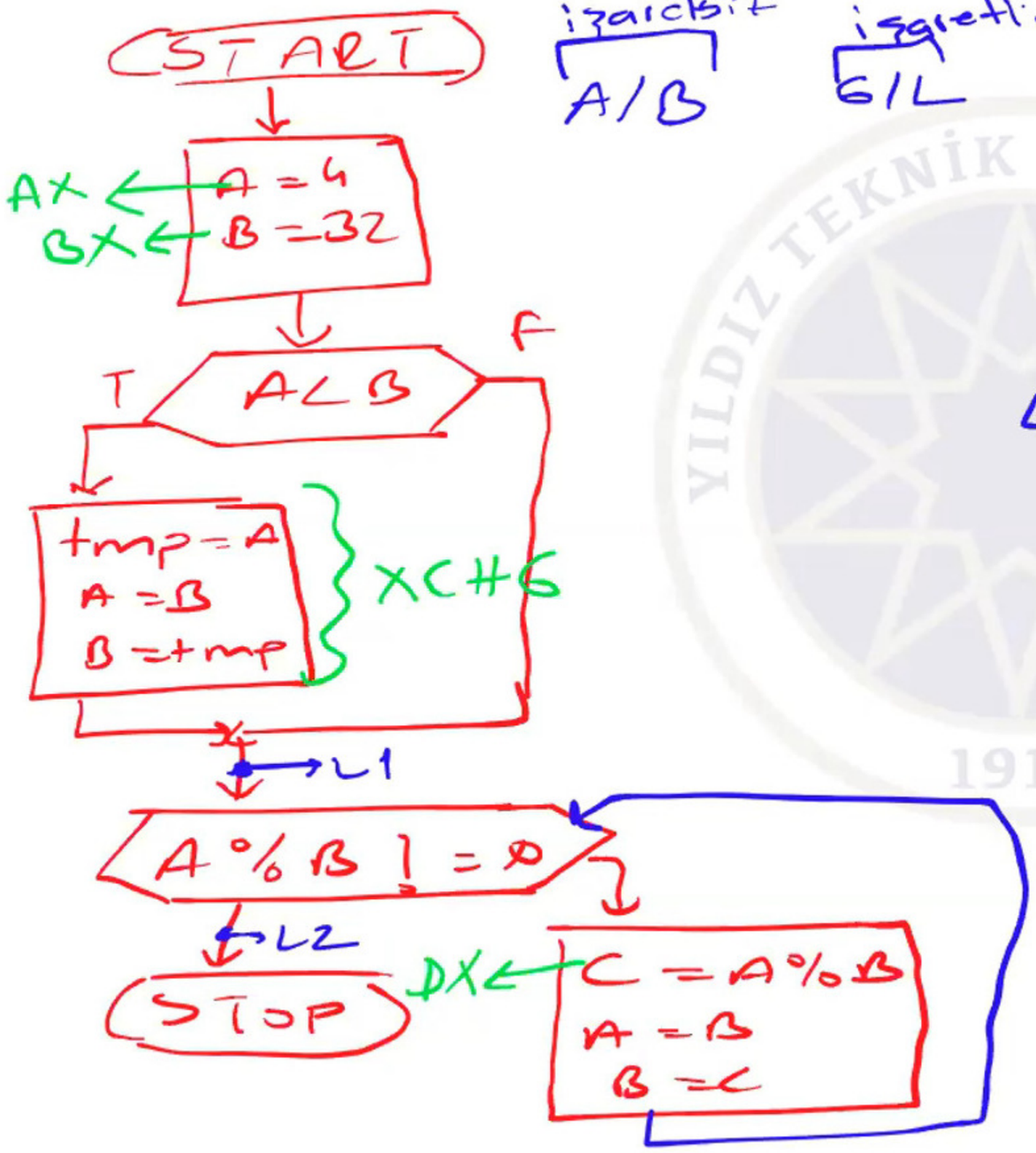



```
MOV CX,6
MOV SI,5
MOV BX,1
MOV AX,SI
MOV DX,0
L1: PUSH AX
DIV DI
ADD BX,AX
POP AX
MUL SI
INC DI
LOOP L1
```

$$A\ ve\ B = [0-500] \rightarrow EBOB(A,B) = ?$$

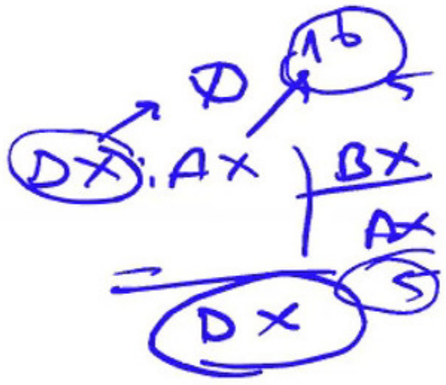
işaretsiz
A/B

işaretli
6/L




```
MOV AX, 4
MOV BX, 32
CMP AX, BX
JAE L1
XCH6 AX, BX
L1: MOV DX, 0
    DIV BX
    CMP DX, 0
    JE L2
    MOV AX, BX
    MOV BX, DX
    JMP L1
L2: ; sonuc BX'te
```

?



YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Alt Seviye Programlama

Hafta 3

Bayraklarla İlgili Komutlar

Bayraklarla İlgili Komutlar		
CLC - Clear CF	**STD** - Set DF	**SAHF** - Store AH in Flag
CMC - Complement CF	**STI** - Set IF	
STC - Set CF	**CLI** - Clear IF	
CLD - Clear DF	**LAHF** - Load AH with Flag	

Mantıksal Komutlar - NOT, OR, AND, XOR, TEST

Alt Seviye Programlama

Hafta 3

NOT	OR	AND	XOR	TEST
MOV AX, 00FFh NOT AX ; AX ← 0FF00h FF00h mov ax ff00h				

Alt Seviye Programlama

Hafta 3

Mantıksal Komutlar - NOT, OR, AND, XOR, TEST

NOT	OR	AND	XOR	TEST

0000100110b

01000000b

10010011b

93h

A	B	OR S1	AND S2	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

MOV AL, 12h

AND AL, 80h

AL ← 0

0000100110b

10000000b

AND

00000000b

MOV AL, 12h

OR AL, 80h

AL → 0000100110b

80h → 10000000b

OR

10010011b

AL ← 93h

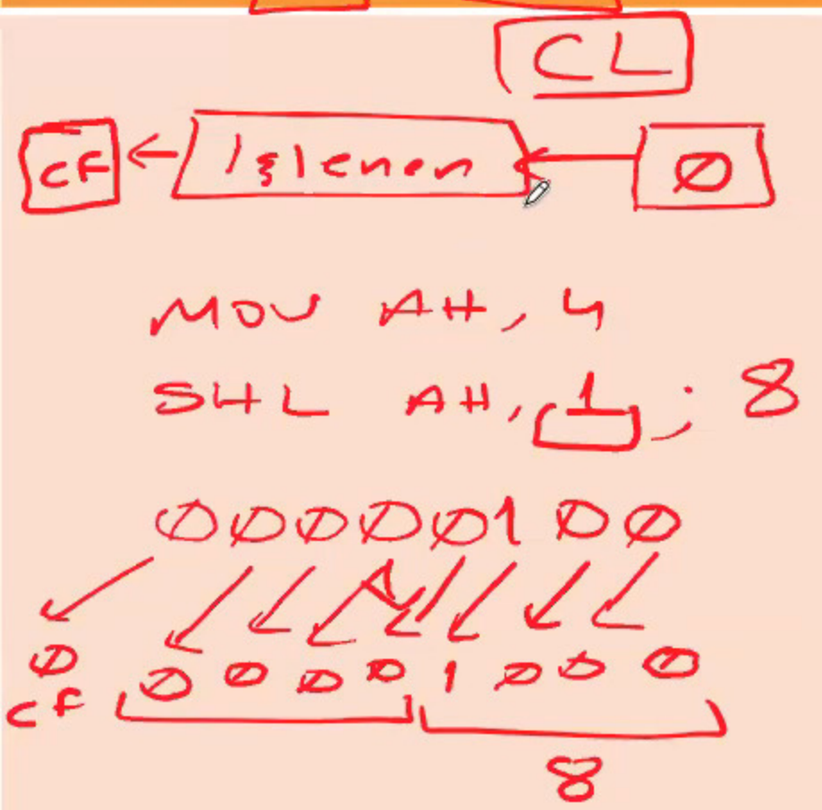
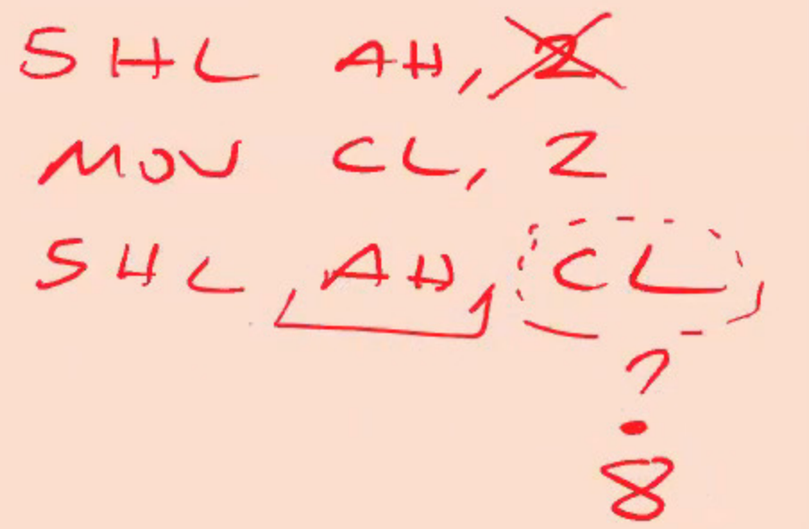
Mantıksal Komutlar - NOT, OR, AND, XOR, TEST

Alt Seviye Programlama

Hafta 3

NOT	OR	AND	XOR	TEST
				TEST Op1, Op2 Flags ← PSW ← Op1 ⊗ Op2 AND Op1, Op2 Op1 ← Op1 ⊗ Op2

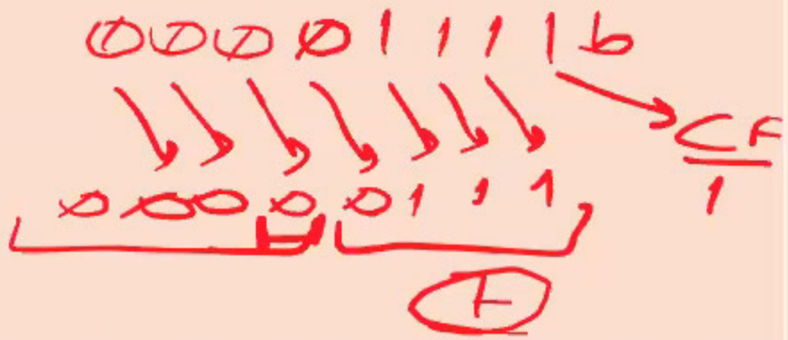
Öteleme Komutları - SHL, SHR, SAR, SAL

SHL ve SAL	SHR	SAR
		

Öteleme Komutları - SHL, SHR, SAR, SAL

SHL ve SAL	SHR	SAR

```
MOV AX, 15
SHR AX, 1 ; 7
```



Öteleme Komutları - SHL, SHR, SAR, SAL

BLM2021
Alt Seviye Programlama

Hafta 3

SHL ve SAL	SHR	SAR

"sayi" → 13 → 6 → 3 → 1 → 0
→ 17 → 8 → 4 → 2 → 1 → 0

2^1 (1) → tek

0 → çift

SHL sayi, 1
JCF tek
MOV AX, 1
JMP sonuç
tek: MOV AX, 0
sonuç:

Cf
0 → 1
C → T

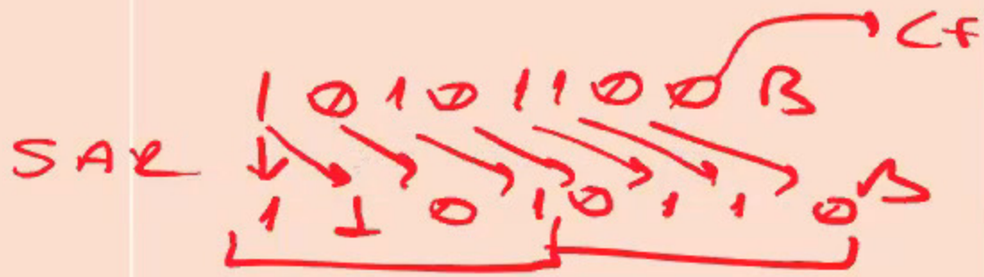
SHR DADDR, idata (1)

{MOV AX, sayi
SHR AX, 1

Öteleme Komutları - SHL, SHR, SAR, SAL

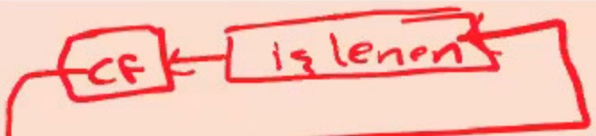
SHL ve SAL	SHR	SAR


```
MOV AH, 10101100B
SAR AH, 1
```



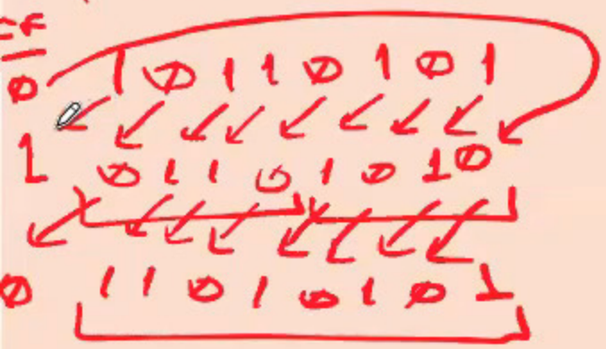
Döndürme Komutları – RCL, RCR, ROL, ROR

RCL Rotate through Carry Left	RCR Rotate Right through Carry	ROL Rotate Left	ROR Rotate Right



AH ← 10110101, CF ← 0

RCL AH, 1

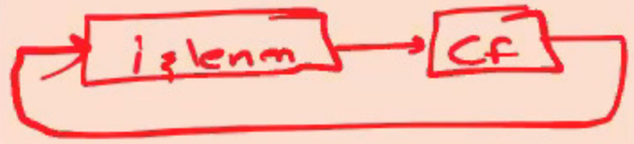


Döndürme Komutları – RCL, RCR, ROL, ROR

BLM2021
Alt Seviye Programlama

Hafta 3

RCL Rotate through Carry Left	RCR Rotate Right through Carry	ROL Rotate Left	ROR Rotate Right



STC ; CF:1

MOV BL, 11100111B

MOV CL, 3

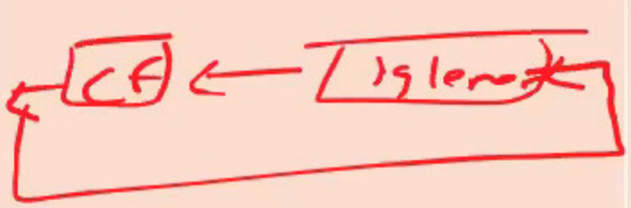
RCR BL, CL

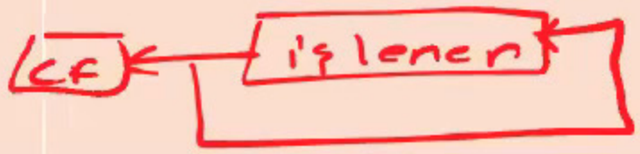
Döndürme Komutları – RCL, RCR, ROL, ROR

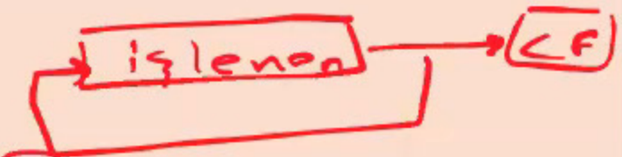
BLM2021
Alt Seviye Programlama

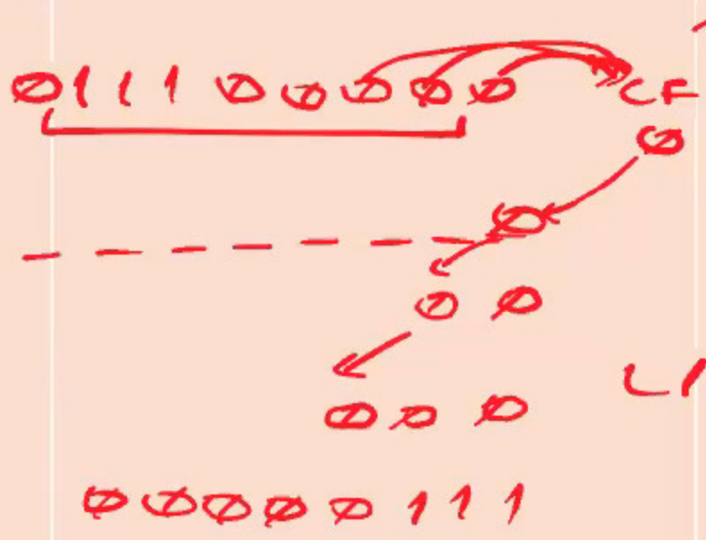
Hafta 3

RCL Rotate through Carry Left	RCR Rotate Right through Carry	ROL Rotate Left	ROR Rotate Right









```
MOV AL, a
MOV CX, 8
L1: SHR AL, 1
RCL AH, 1
LOOP L1
MOV a, AH
```

Ders Tanıtım Formu ve Konular

BLM2021
Alt Seviye Programlama

Hafta 5

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

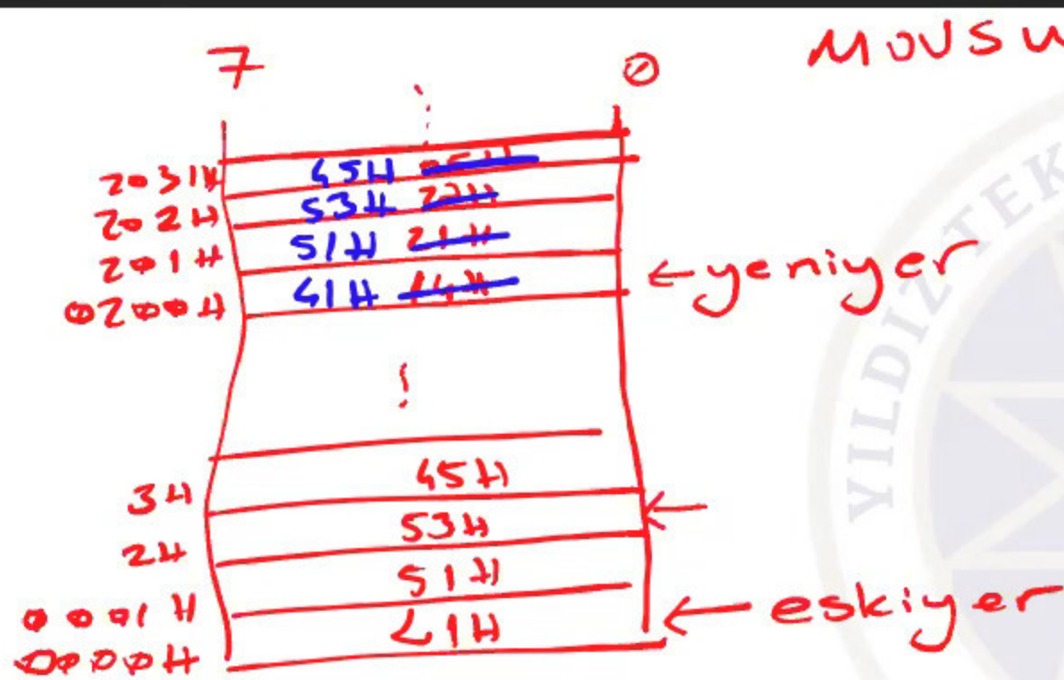
Dizgi (String), Yığın (Stack) Komutları

BLM2021
Alt Seviye Programlama

Hafta 5

- MOVSB
- CMPSB
- SCASB
- LODSB
- STOSB
- CBW
- CWD
- POP
- PUSH
- PUSHF
- POPF
- LAHF
- SAHF

MOVSB (Move String Byte)



MOVSW

DS

[ES:DI] ← [DS:SI]

CLD

DF → 0 → adres artacak

STD

1 ⇒ adres azalacak

```
LEA SI, eskiyer
LEA DI, yeniyer
MOV CX, 4
CLD
REP MOVSB
```

CMPSB (Compare String Byte)

CMPSW

[DS:SI] – [ES:DI]

DF → 0 → ↑
DF → 1 → ↓

```
LEA SI, dizi1      → 'assembly12'
LEA DI, dizi2      → 'assamb123'
MOV CX, 10
CLD
REPE CMPSB
```

SCASB (Scan String Byte)

SCASW

AL - [ES:DI] ⇒ bayraklar

	Bellek
020CH	6DH
020BH	6FH
020AH	63H
0209H	2EH
0208H	43H
0207H	46H
0206H	40H
0205H	61H
0204H	6DH
0203H	75H
0202H	79H
0201H	55H ← email

7 0

@ ← 6DH

```
LEA DI, email
MOV CX, 12
CLD
MOV AL, '@'
REPNE SCASB,
```

LODSB, STOSB, CBW, CWD

sw sw

LODSB
AL ← [DS:SI]

STOSB
[ES:DI] ← AL

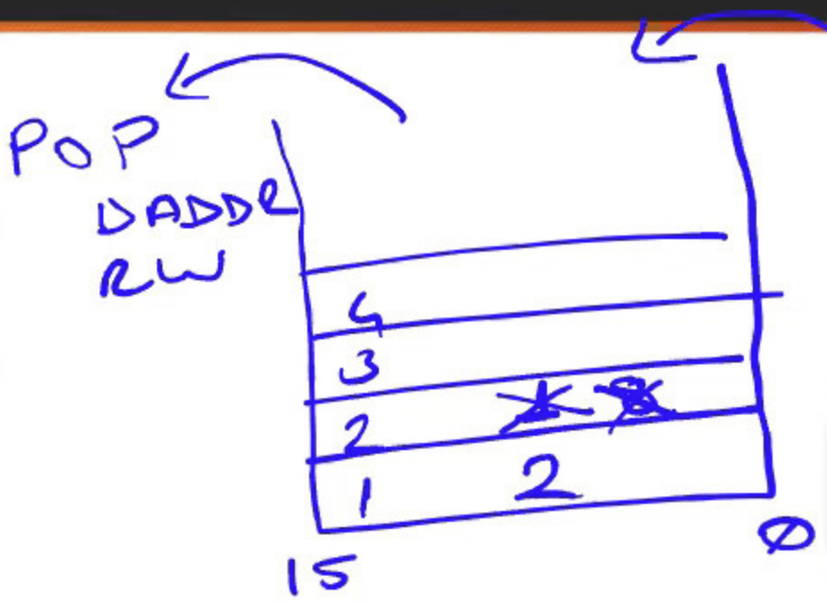
CWD
DX, AX ← AX

CBW (Convert Byte to Word)
AX ← AL

~~CBW BL ; BX ← BL~~

CBW
TEST
AL ← 1000 1111 B
AX ← 1111 1111 1000 1111 B
F F

Yığın (Stack) Komutları



DADDR
regW → RW

SR

```
MOV AX, 2
PUSH AX
SHR AX, 1
PUSH AX
POP BX ← 1
ADD BX, 7
PUSH BX
POP sayi ← 8
```

dizi[]: 1, 2, 3, 4
4 3 2 1

```
XOR SI, SI
MOV CX, 4
L1: PUSH dizi[SI]
    INC SI
    LOOP L1

    XOR SI, SI
    MOV CX, 4
L2: POP dizi[SI]
    INC SI
    LOOP L2
```

Yığın (Stack) Komutları

PUSH F → PSW

↓
Flags

POPF

LAHF → Load AH with Flags

AH

SF	ZF	?	AF	?	PF	?	CF

SAHF → Store AH in Flags

Yığın (Stack) Komutları

Ör/ Diğer bayraklara dokunmadan ZF değerinin Complementini alalım.

0 → 1
1 → 0

```
      ↑
  1 0 0 1 1 0 1 0
+ 0 1 0 0 0 0 0 0  → 40H
-----------------
  1 1 0 1 1 0 1 0
```

- PUSHF → 9 flags
- LAHF
- XOR AH, 40H
- POPF
- SAHF

MOV BL, 0
SUB BL, BL

Örnek 2

0 1 0 1 0 1 0 1 b

0 0 0 0 0 0 0 1 b
1 3 5 7

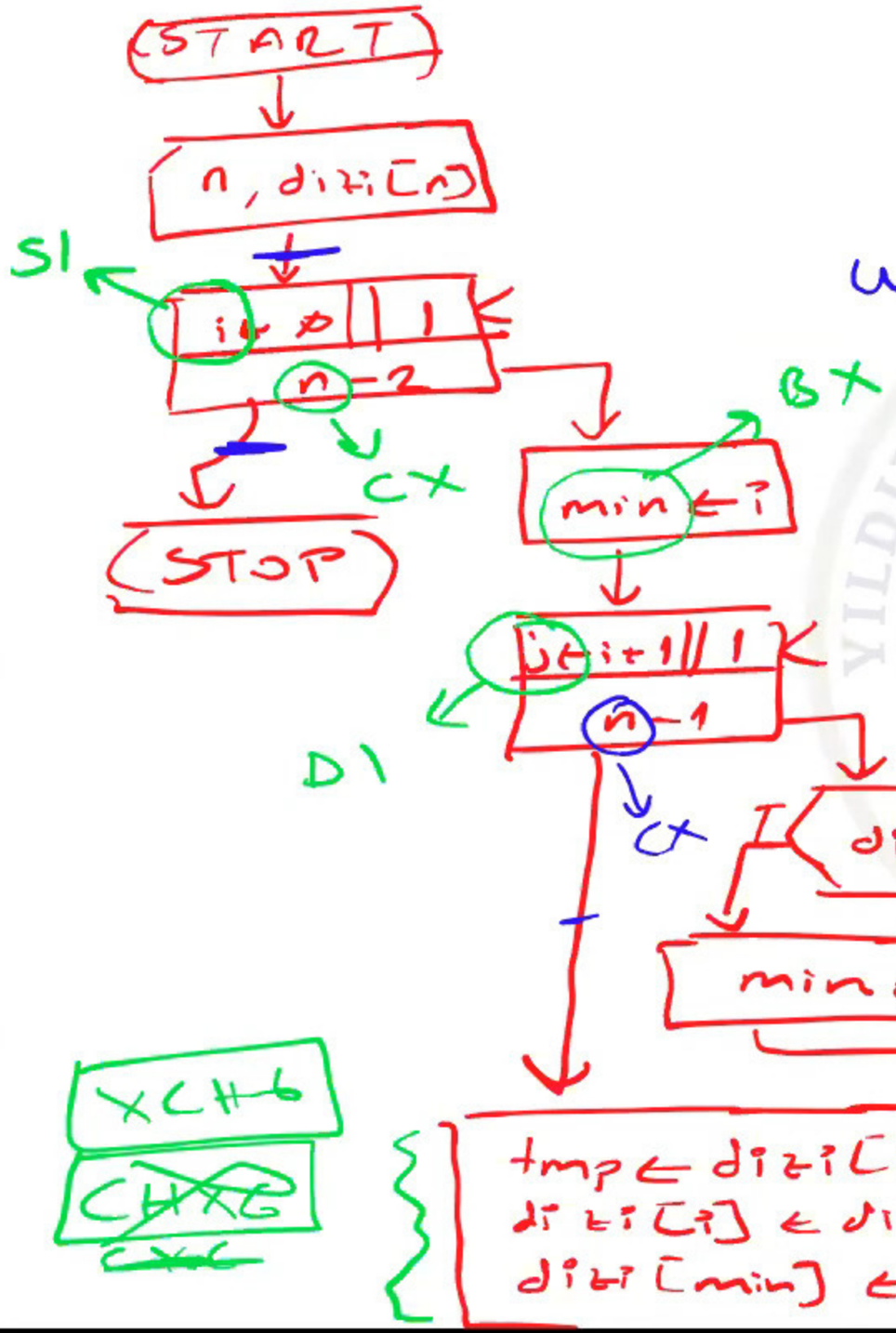
XOR BL, BL
MOV CX, 8
MOV AH, 55h
L1: SHL AH, 1
JNC Devam
INC BL
Devam: LOOP L1

Verilen 1 byte'taki bitleri sayan ASM kodunu yazalım.

```
XOR  BL, BL
MOV  CX, 8
MOV  AH, 55H
L1:  SHL  AH, 1
     ADC  BL, 0
     LOOP L1
```

```
XOR  BL, BL
MOV  AH, 55H
L1:  CMP  AH, 0
     JE   cikis
     SHL  AH, 1
     ADC  BL, 0
     JMP  L1
cikis:
```

Selection Sort



Word, işaretli:

dış-dön:
```
XOR SI,SI
MOV CX,n
DEC CX
MOV BX,SI
PUSH CX
MOV DI,SI
ADD DI,2
MOV DX,SI
SHR DX,1
MOV CX,n
SUB CX,DX
```

iç-dön:
```
MOV AX,dizi[DI]
CMP AX,dizi[BX]
JGE fakedal
MOV BX,DI
ADD DI,2
LOOP iç-dön
POP CX
MOV AX,dizi[SI]
XCHG AX,dizi[BX]
MOV dizi[SI],AX
```

Ders Tanıtım Formu ve Konular

BLM2021
Alt Seviye
Programlama

Hafta 6

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

Adresleme Kipleri

BLM2021
Alt Seviye Programlama

Hafta 6

- Anlık (Immediate) Adresleme
- Yazmaç (Register) Adresleme
- Doğrudan (Direct) Adresleme
- Yazmaç Dolaylı (Register Indirect) Adresleme
- Baz Göreli (Base Ralative) Adresleme
- Doğrudan İndisli (Direct Index) Adresleme
- Dizi (String) Adresleme
- İskele (Port) Adresleme

YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Anlık (Immediate) Adresleme

- *MOV reg, idata*
- *MOV mem, idata*

- MOV AL, 16
- MOV CX, 0ABCDh

Yazmaç (Register) Adresleme

BLM2021
Alt Seviye Programlama

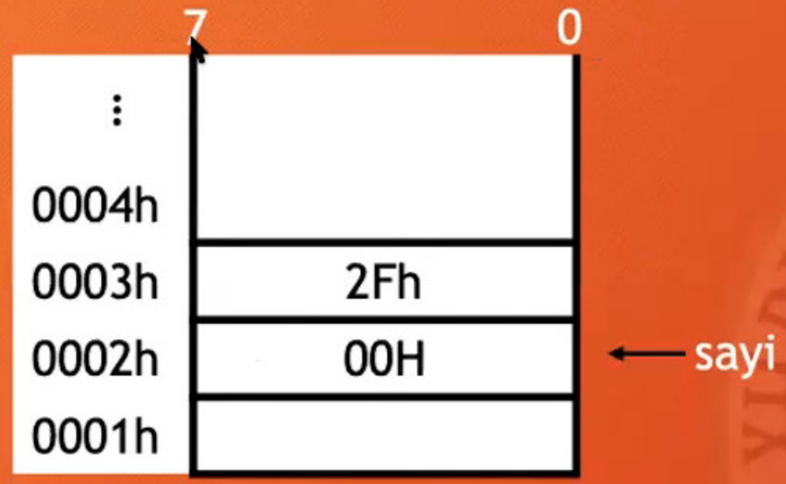
Hafta 6

- *MOV reg,reg*

- MOV DS, AX
- MOV DL, CH

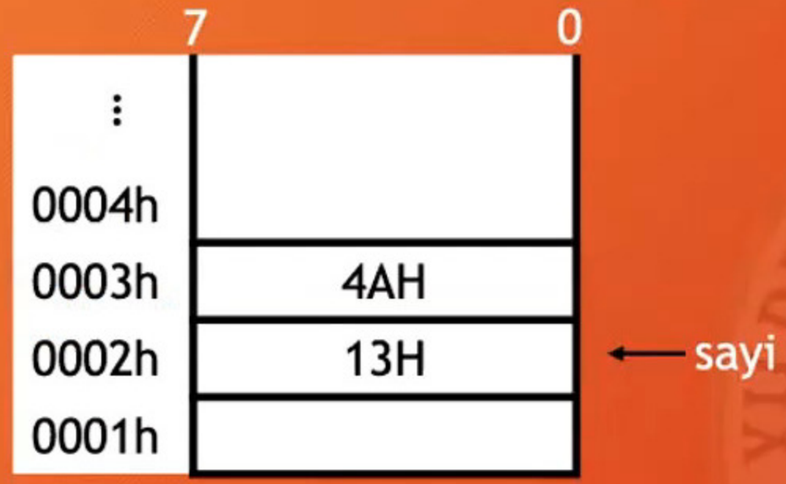
YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Doğrudan (Direct) Adresleme



- sayi → Word ise;
  - $MOV\ AX, sayi \implies AX: 2F00h$
- sayi → Byte ise;
  - $MOV\ AL, sayi \implies AL: 00h$

Yazmaç Dolaylı (Register Indirect) Adresleme



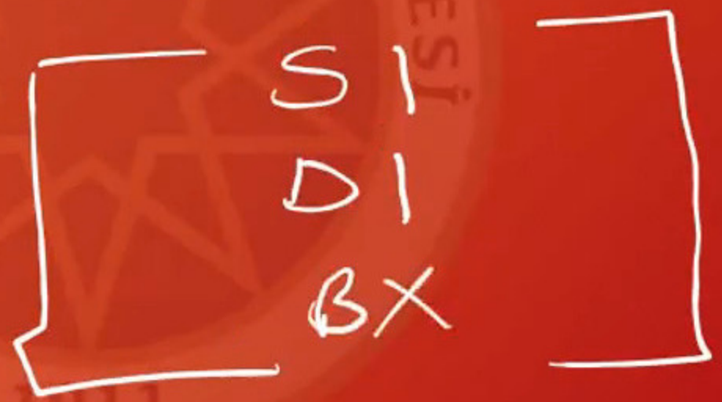
- $LEA\ SI, sayi \Longrightarrow SI: 0002h$
- $MOV\ AX, [SI] \Longrightarrow AX: 4A13h$

- $MOV\ DI, OFFSET\ sayi \Longrightarrow DI: 0002h$
- $MOV\ AX, [DI] \Longrightarrow AX: 4A13h$

Baz Göreli (Base Ralative) Adresleme

	7 … 0	
⋮		
0018h		
0017h	59h	
0016h	4Ch	
0015h	42h	
0014h	4Dh	
0013h	45h	
0012h	53h	
0011h	53h	
0010h	41h	← msg

- ⊛ $msg\ db\ 'ASSEMBLY'$
- $LEA\ BX, msg \Longrightarrow BX: 0010h$
- $MOV\ AL, [BX+4] \Longrightarrow \mathrm{AL}: 4\mathrm{Dh}$
- $MOV\ AX, [BX+5] \Longrightarrow AX: 4C42h$

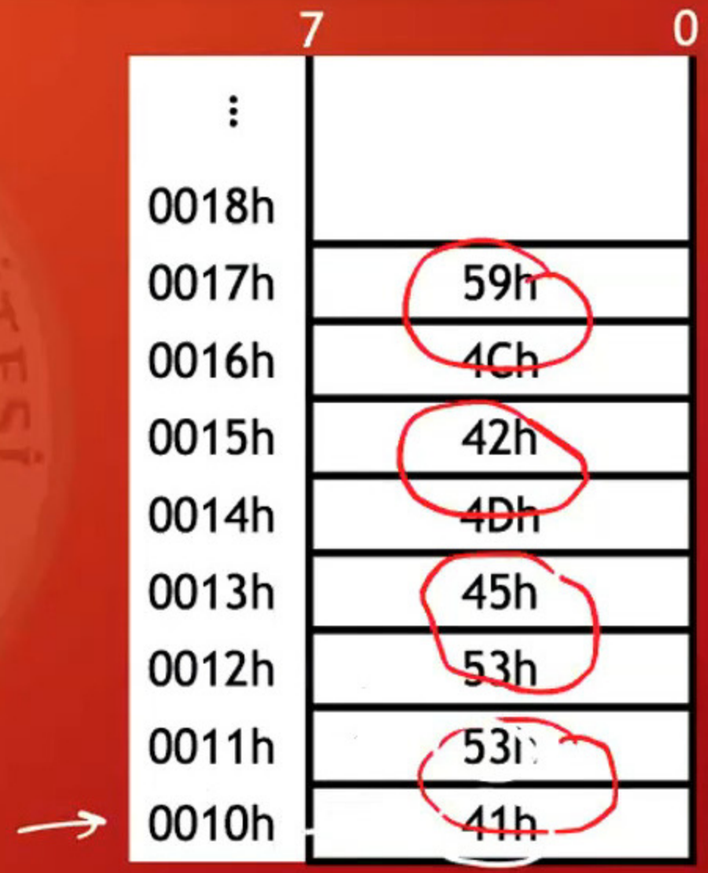


Doğrudan İndisli (Direct Index) Adresleme

- Yüksek seviyeli dillerdeki dizi erişimlerine benziyor
- DİZİLERİN TİPİ ÇOK ÖNEMLİ

- $\circledast$ $dizi$ **DB** $41h, 53h, 53h, 45h, 4Dh, 42h, 4Ch, 59h$
- $XOR\ SI, SI$ → 0
- $MOV\ AL, dizi[SI] \Rightarrow AL: ?$ 41h
- $INC\ SI$ → 1
- $MOV\ AL, dizi[SI] \Rightarrow AL: ?$ 53h

- $\circledast$ $dizi2$ **DW** $5341h,\ 4553h,\ 424Dh,\ 594Ch$
- $XOR\ SI, SI$ → 0
- $MOV\ AX, dizi2[SI] \Rightarrow AX: ?$ 5341h



Doğrudan İndisli (Direct Index) Adresleme

- Yüksek seviyeli dillerdeki dizi erişimlerine benziyor
- DİZİLERİN TİPİ ÇOK ÖNEMLİ

- ⊛ $dizi$ **DB** $41h, 53h, 53h, 45h, 4Dh, 42h, 4Ch, 59h$
- $XOR\ SI, SI$
- $MOV\ AL, dizi[SI] \Longrightarrow AL: ?$
- $INC\ SI$
- $MOV\ AL, dizi[SI] \Longrightarrow AL: ?$

- ⊛ $dizi2$ **DW** $5341h,\ 4553h,\ 424Dh,\ 594Ch$
- $XOR\ SI, SI$
- $MOV\ AX, dizi2[SI] \Longrightarrow AX: ?$ — 5341h
- $INC\ SI$ → 1
- $MOV\ AX, dizi2[SI] \Longrightarrow AX: ?$ 5353h

ADD SI, 2

	7 … 0
⋮	
0018h	
0017h	59h
0016h	4Ch
0015h	42h
0014h	4Dh
0013h	45h
0012h	53h
0011h	53h
0010h	41h

Örnek 1

2 dizinin yerini farklı adreslemeyle değiştirelim.

d1, d2: byte tanımlı

n: dizilerin boyutu

```
MOV CX, n
XOR SI,SI      ; d1 → SI ← 0
LEA DI, d2     ; DI → 1FA2h
don: MOV AL, d1[SI]
     XCHG AL, [DI]      ; XCHG [DI], d1[SI]
     MOV d1[SI], AL     XCHG DADDE, DADDE2
     INC DI
     INC SI
     LOOP don
```

A şirketinde çalışanların doğum yılı d1 dizisinde, B şirketinde çalışanların yaşları d2 dizisinde tutulmaktadır. Önce A şirketinde çalışan a1 kişinin, sonra B şirketinde çalışan b2 kişinin bu bilgileri boyutu a1 + b1 olan d3 dizisine aktarmak için gerekli ASM kodunu yazalım.

word

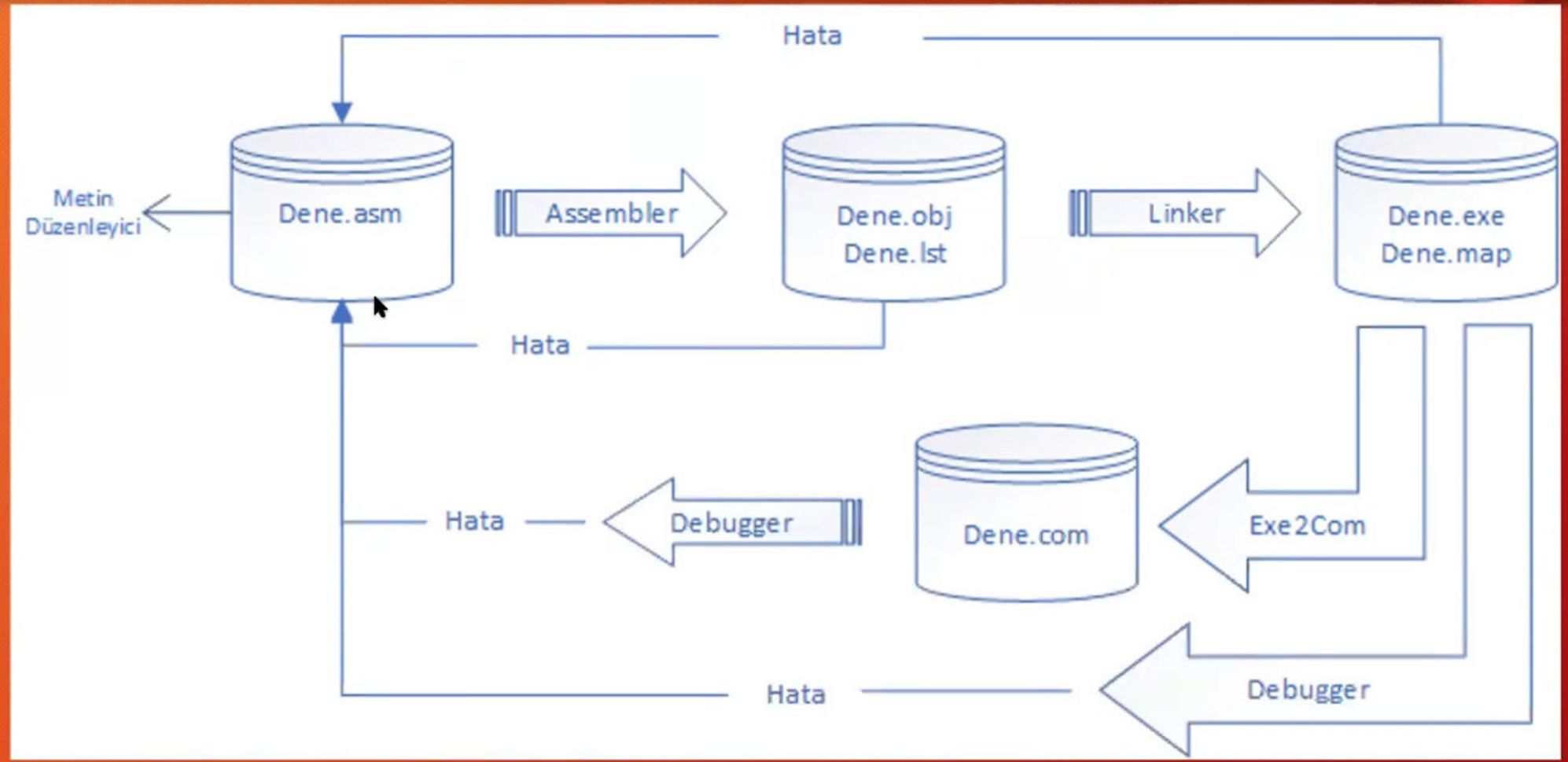
BYTE

WORD

```
    LEA DI,d1
    LEA SI,d2
    XOR BX,BX
    MOV CX,a1
L1: MOV AX,[DI]
    MOV d3[BX],AX
    ADD DI,2
    ADD BX,2
    LOOP L1
    MOV CX,b1
L2: MOV AL,[SI]
    CBW
    MOV d3[BX],AX
    ADD BX,2
    INC SI
    LOOP L2
```

DF AX ① XOR AH AH

Assembly Program Derleme ve Çalıştırma



Sözde (Pseudo) Komutlar

BLM2021
Alt Seviye Programlama

Hafta 6

- Makine kodu karşılığı yoktur.
- Bayraklar üzerinde etkisi yoktur.
- Debug ortamında kullanılamıyorlar.
- LST dosya düzeni
  - Program kesim düzeni
- Veri tanımlamaları için kullanılırlar.
- Derleyici tarafından anlamlandırılırlar.

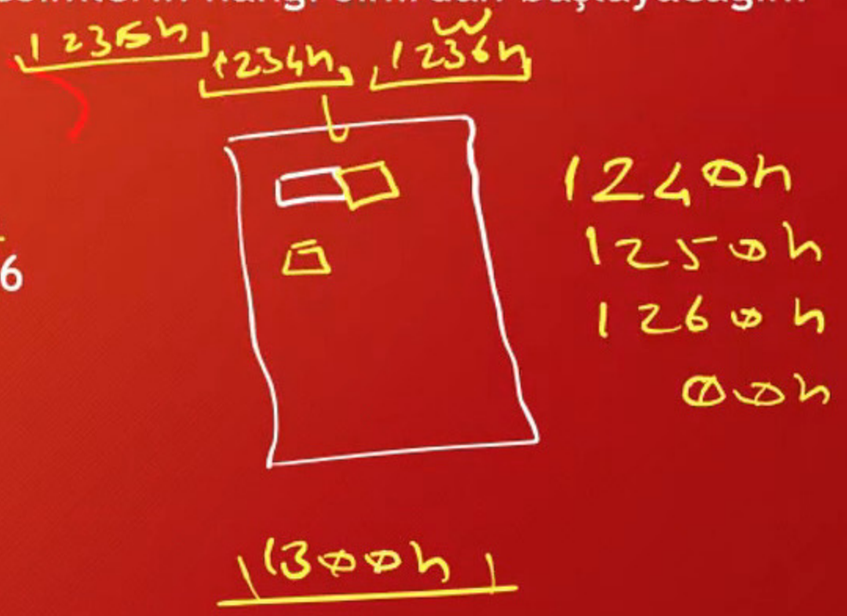
YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Sözde Komutlar – SEGMENT/ENDS


```
kesim_ismi      SEGMENT {seçenekler}
                ..............
                ..............
                ..............
kesim_ismi      ENDS
```

SEÇENEKLER

a) **Hizalama** (Alignment) Tipi:
- Tanımlanan kesimlerin hangi sınırdan başlayacağını belirler.
- **BYTE** /1
- **WORD** /2
- **PARA** /16
- **PAGE** /256

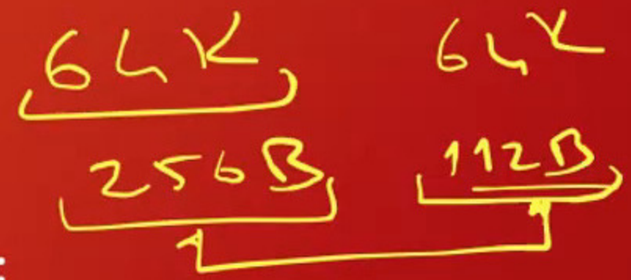


Sözde Komutlar – SEGMENT/ENDS


```
kesim_ismi      SEGMENT {seçenekler}
                ..............
                ..............
                ..............
kesim_ismi      ENDS
```

SEÇENEKLER

a) **Hizalama** (Alignment) Tipi:
- Tanımlanan kesimlerin hangi sınırdan başlayacağını belirler.
- **BYTE** /1
- **WORD** /2
- **PARA** /16
- **PAGE** /256



b) **Birleştirme** (Combine) Tipi:
- Link işleminden sonra aynı isim verilmiş kesimlerin birleştirilmesi için kullanılır.
- **PUBLIC**: Aynı isimli kesimler birbirinin devamı olur.
- **COMMON**: Kesimler birbirinin üzerine yazılır.
- **STACK**: LIFO sisteminde çalışır.
- **AT #####**: Belirlenen fiziksel adresten başlar.

Sözde Komutlar – SEGMENT/ENDS


```
kesim_ismi      SEGMENT {seçenekler}
                ..............
                ..............
                ..............
kesim_ismi      ENDS
```

SEÇENEKLER

a) **Hizalama** (Alignment) Tipi:
- Tanımlanan kesimlerin hangi sınırdan başlayacağını belirler.
- **BYTE** /1
- **WORD** /2
- **PARA** /16
- **PAGE** /256

b) **Birleştirme** (Combine) Tipi:
- Link işleminden sonra aynı isim verilmiş kesimlerin birleştirilmesi için kullanılır.
- **PUBLIC**: Aynı isimli kesimler birbirinin devamı olur.
- **COMMON**: Kesimler birbirinin üzerine yazılır.
- **STACK**: LIFO sisteminde çalışır.
- **AT #####**: Belirlenen fiziksel adresten başlar.

c. **Sınıf** (Class) Tipi:
- Birleştirme tipi PUBLIC olarak tanımlanmış kesimlerin link işlemi sırasında birbirinin peşi sıra gelmesi için kullanılır.
- Tırnak içinde yazılır.

Sintaks Örneği


```
stacksg     SEGMENT PARA STACK 'yigin'
            ..............
stacksg     ENDS
codesg      SEGMENT PARA 'kod'
            ..............
codesg      ENDS
datasg      SEGMENT PARA 'veri'
            ..............
datasg      ENDS
```

test SEGMENT BYTE AT 0120AH '+'

test ENDS

YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Sözde Komutlar – ASSUME, DB, DW, DD, DQ, DT

ASSUME SS:stacksg, DS:datasg, CS:codesg

- Segment atamaları için kullanılır.
- COM tipinde ise;
  - ASSUME SS:mysg, CS:mysg, DS:mysg

DB (Define Byte)

```
sayi1    DB 25
sayi2    DB 1Eh
sayi3    DB ?
sayi4    DB 1, 2, 3Fh, 0A4h, 5
strb     DB 'Assembly Dersi'
```

Sözde Komutlar – ASSUME, DB, DW, DD, DQ, DT

ASSUME SS:stacksg, DS:datasg, CS:codesg

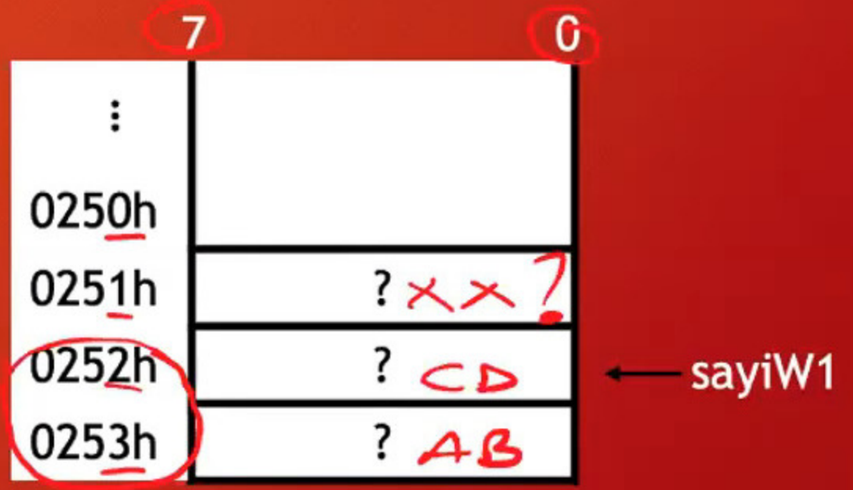
- Segment atamaları için kullanılır.
- COM tipinde ise;
  - ASSUME SS:mysg, CS:mysg, DS:mysg

DB (Define Byte)

```
sayi1   DB 25
sayi2   DB 1Eh
sayi3   DB ?
sayi4   DB 1, 2, 3Fh, 0A4h, 5
strb    DB 'Assembly Dersi'
```

DW (Define Word)

```
sayiW1   DW 0ABCDh
```



Sözde Komutlar – ASSUME, DB, DW, DD, DQ, DT

BLM2021
Alt Seviye Programlama

Hafta 6

ASSUME SS:stacksg, DS:datasg, CS:codesg

- Segment atamaları için kullanılır.
- COM tipinde ise;
  - ASSUME SS:mysg, CS:mysg, DS:mysg

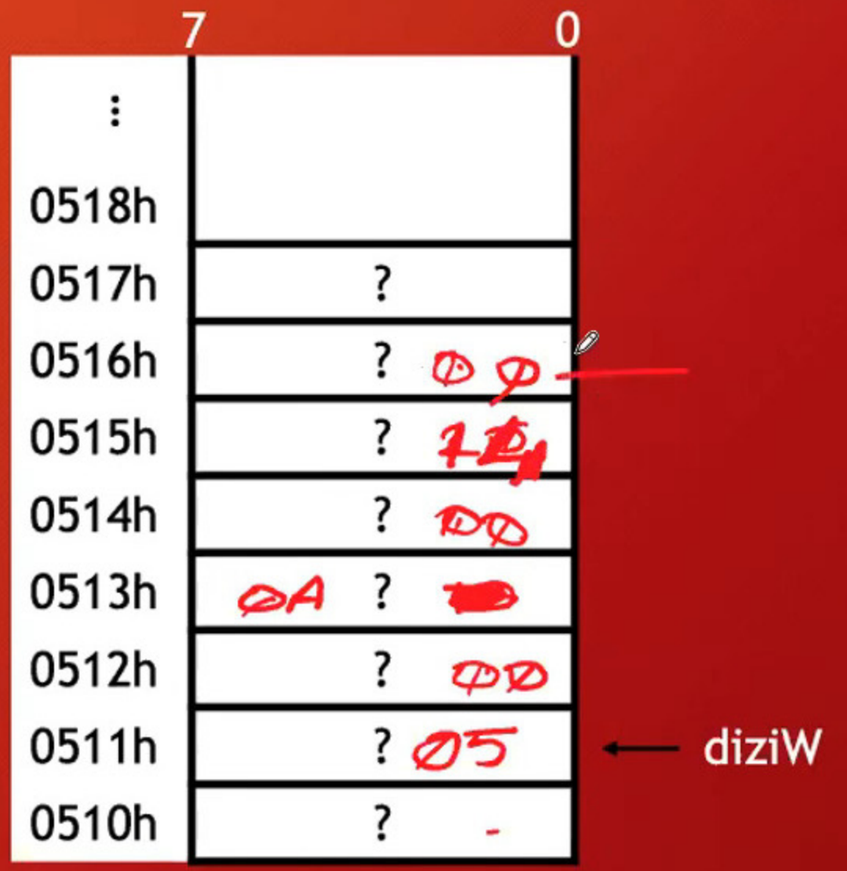
DB (Define Byte)

```
sayi1   DB 25
sayi2   DB 1Eh
sayi3   DB ?
sayi4   DB 1, 2, 3Fh, 0A4h, 5
strb    DB 'Assembly Dersi'
```

DW (Define Word)

```
sayiW1  DW 0ABCDh

diziW   DW 5, 10, 20
```



Sözde Komutlar – ASSUME, DB, DW, DD, DQ, DT

ASSUME SS:stacksg, DS:datasg, CS:codesg

- Segment atamaları için kullanılır.
- COM tipinde ise;
  - ASSUME SS:mysg, CS:mysg, DS:mysg

DB (Define Byte)

```
sayi1    DB 25
sayi2    DB 1Eh
sayi3    DB ?
sayi4    DB 1, 2, 3Fh, 0A4h, 5
strb     DB 'Assembly Dersi'
```

DW (Define Word)

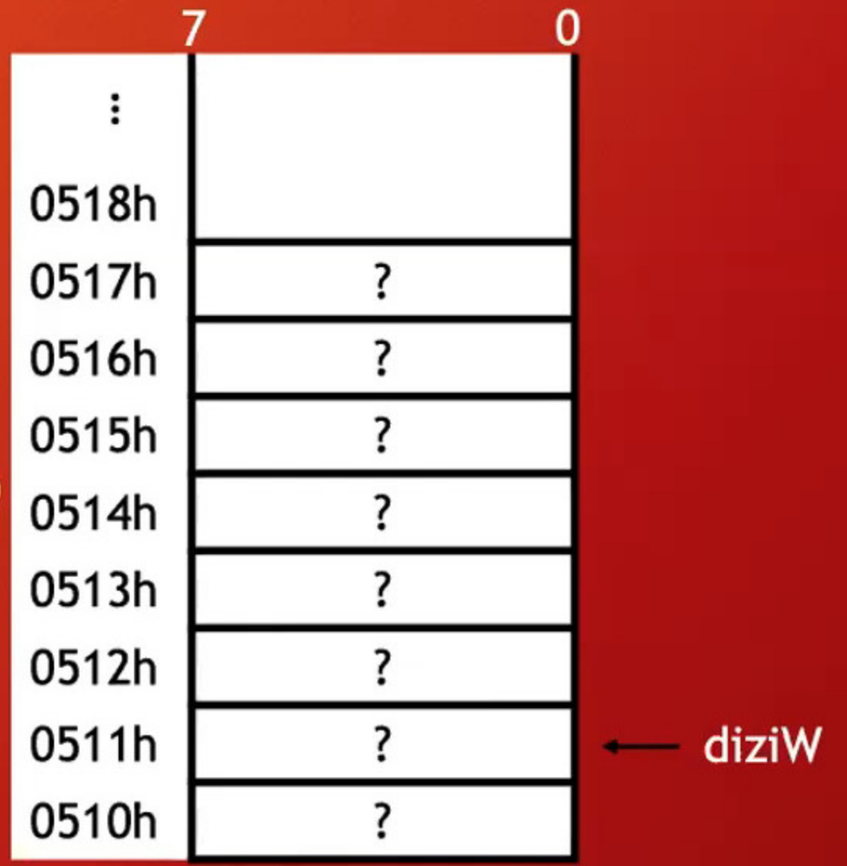
```
sayiW1   DW 0ABCDh

diziW    DW 5, 10, 20
```

DD (Define Double Word)

DQ (Define Quadro Word)

DT (Define Ten Bytes)



Sözde Komutlar – DUP, PTR

BLM2021
Alt Seviye Programlama

Hafta 6

DUP (Duplication Factor)

- Dizib2 DB 15 DUP(0)
- Mb1 DB 3 DUP(4 DUP(8))
  - 3x4 matirs (8)
- Diziw DW 10 DUP(?)

PTR (Pointer)

- Tip dönüşümü (casting)

- sayib DB 15h
- sayiw DW 3545h

- $MOV\ AX, WORD\ PTR\ sayib \Longrightarrow AX:?$ 2515h
- $MOV\ AH, BYTE\ PTR\ sayiw \Longrightarrow AH:?$ 45h
- $MOV\ AL, BYTE\ PTR\ sayiw + 1 \Longrightarrow AL:?$ 35h

Sözde Komutlar - LABEL

```
yeniB LABEL BYTE
sayiW DW 2535h
yeniW LABEL WORD
sayiBb DB 45h
sayiBc DB 55h

MOV AX, yeniW
AX : 5545h
MOV BL, yeniB
BL ← 35h
```

Sözde Komutlar - PROC/ENDP, SEG, END

BLM2021
Alt Seviye Programlama

Hafta 6

PROC (Procedure) / ENDP

```
yordam_ismi     PROC {NEAR/FAR}
                ..............
                ..............
                ..............
                RET/RETF
yordam_ismi     ENDP
```

OFFSET

- LEA aynı işlev

MOV AX, OFFSET Tablo
AX: ? 0713h

	7	0	
⋮			
0712h			
0713h	3Ch		← Tablo
0714h	7Fh		
0715h	89h		

END

- END baslangic_noktasi

END proc-name

YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Sözde Komutlar – PROC/ENDP, SEG, END

BLM2021
Alt Seviye Programlama

Hafta 6

PROC (Procedure) / ENDP

```
yordam_ismi     PROC {NEAR/FAR}
                ..............
                ..............
                ..............
                RET/RETF
yordam_ismi     ENDP
```

OFFSET

- LEA aynı işlev

$MOV\ AX, OFFSET\ Tablo$

$AX: ?$

	7	0	
⋮			
0712h			
0713h	3Ch		← Tablo
0714h	7Fh		
0715h	89h		

END

- END baslangic_noktasi

SEG (SEGMENT)

- $MOV\ ES, SEG\ sayiw$ → DS
  - $ES: ?$
- $MOV\ AX, SEG\ AramaProc$ → CS
  - $AX: ?$

Sözde Komutlar – LENGTH, TYPE, SIZE

LENGTH

- DUP'ten önceki sayı

- *Tablo DW 15 DUP(?)*
- *MOV AX, LENGTH Tablo*
  - *AX: ?*

TYPE

- Değişken tipinin kaç BYTE olduğunu bulmak için
- *sizeof()*

- *Tablo DW 15 DUP(0)*
- *MOV AX, TYPE Tablo*
  - *AX: ?*

- *Sayi DD 3412ABCDh*
- *MOV AX, TYPE Sayi*
  - *AX: ?*

SIZE

- LENGTH * TYPE

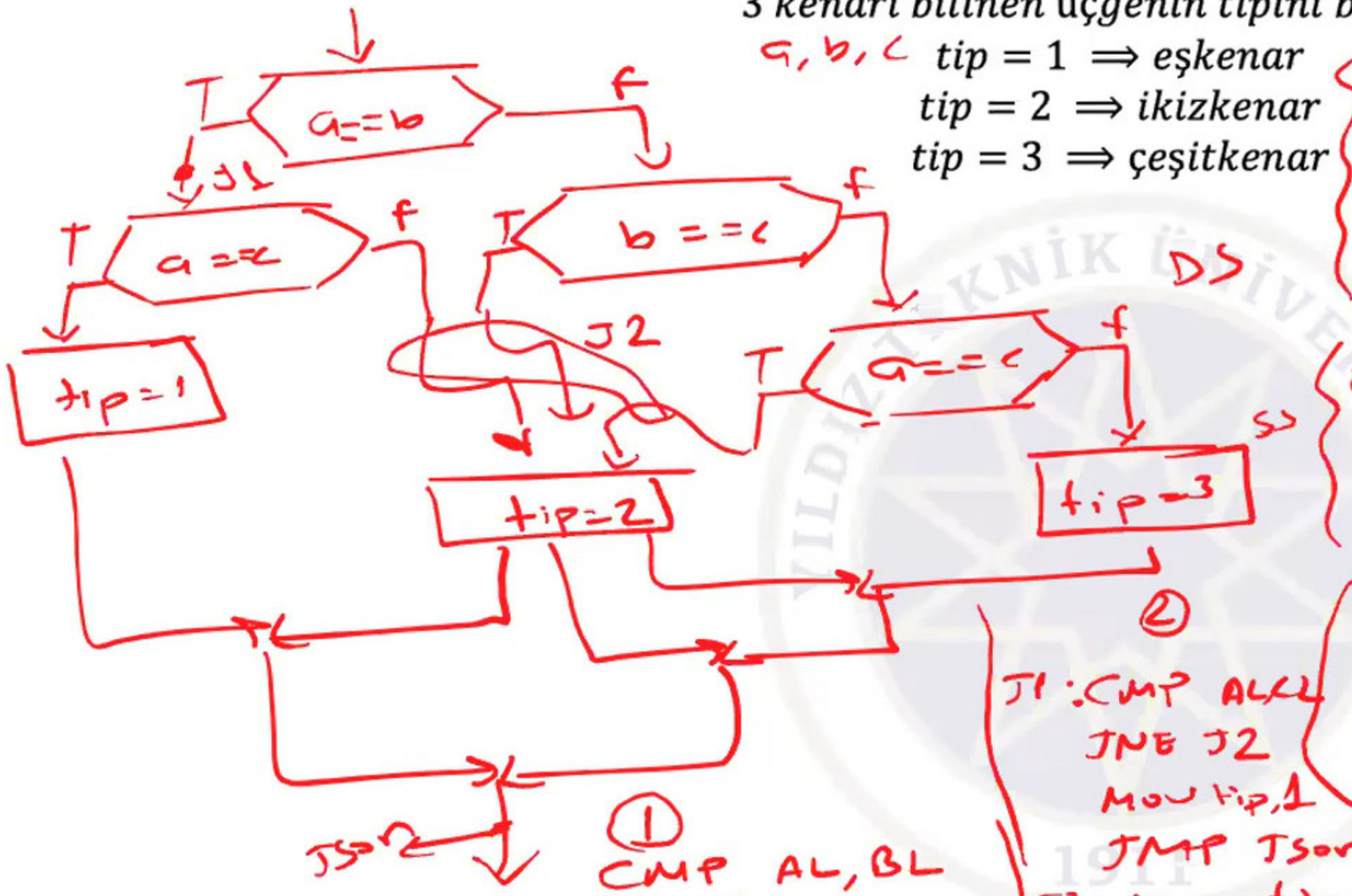
- *Tablo DW 20 DUP(?)*
- *MOV AX, SIZE Tablo*
  - *AX: ?h*

3 kenarı bilinen üçgenin tipini bulunuz.

a, b, c $tip = 1 \Rightarrow$ eşkenar

$tip = 2 \Rightarrow$ ikizkenar

$tip = 3 \Rightarrow$ çeşitkenar

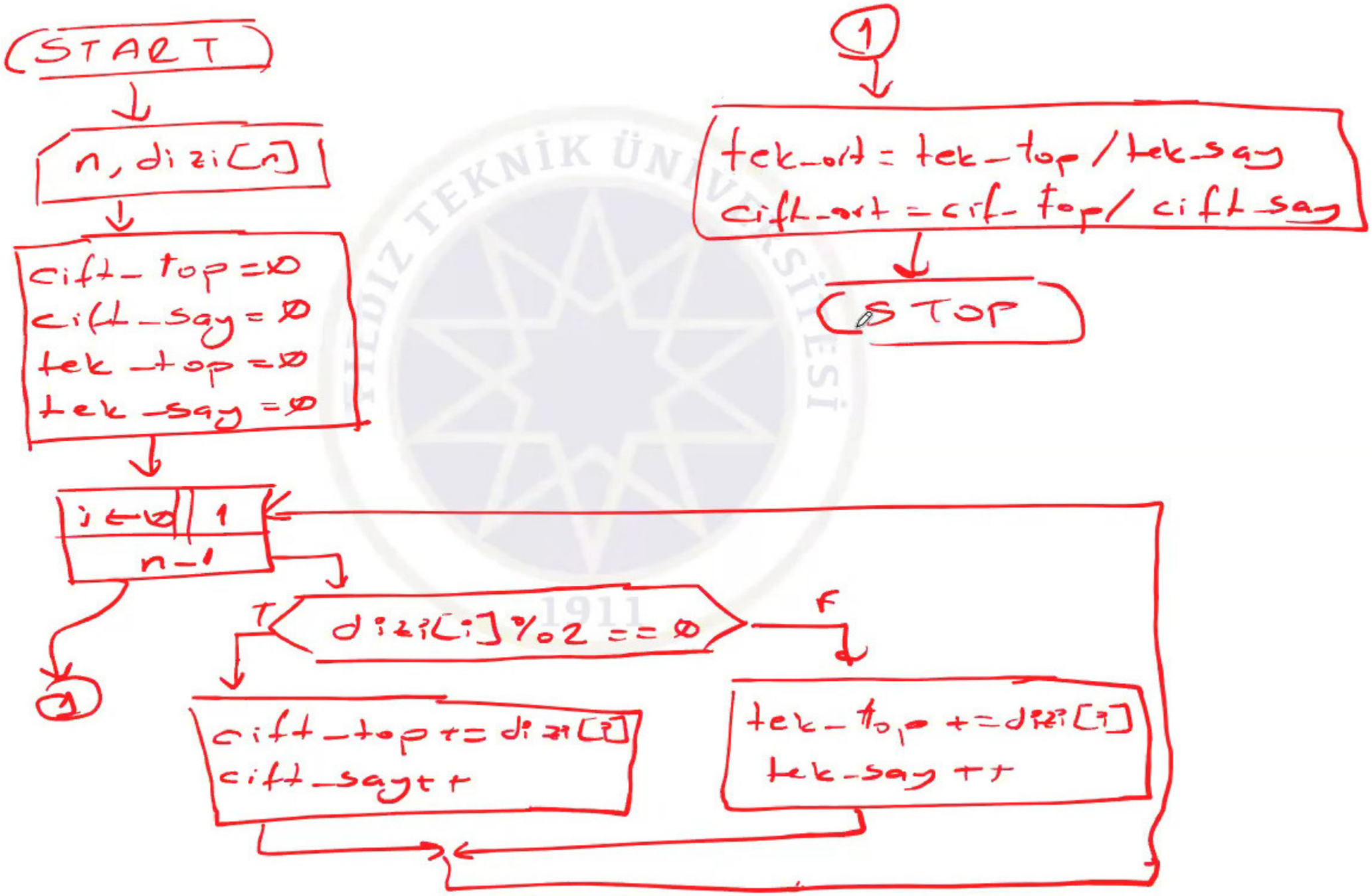



```
datasg  SEGMENT PARA 'veri'
tip     DB  ?
a       DB  12
b       DB  13
c       DB  12
datasg  ENDS
stacksg SEGMENT PARA STACK 'S'
        DW 15 DUP(?)
stack   ENDS
codesg  SEGMENT PARA 'kod'
        ASSUME CS:codesg,DS:datasg,
               SS:stacksg
ANA     PROC FAR
        PUSH DS
        XOR AX,AX
        PUSH AX
        MOV  AX,datasg
        MOV  DS,AX
        MOV  AL,a
        MOV  BL,b
        MOV  CL,c
        ①
```

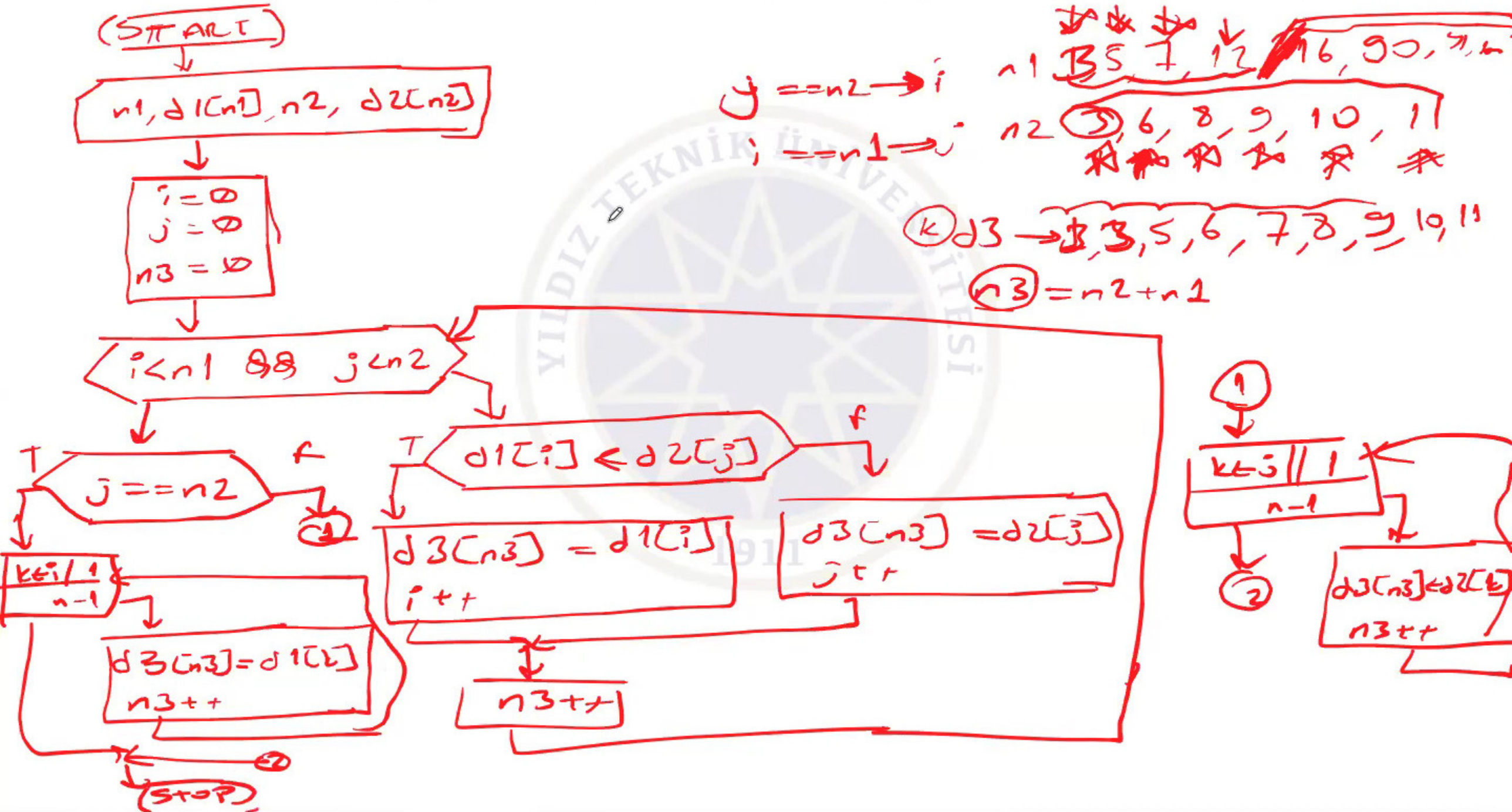
①
```
CMP AL,BL
JE J1
CMP BL,CL
JE J2
CMP AL,CL
JE J2
MOV tip,3
JMP Json
```

②
```
J1: CMP AL,CL
    JNE J2
    MOV tip,1
    JMP Json
J2: MOV tip,2
Json: RETF
ANA ENDP
codesg ENDS
       END ANA
```

Word tanımlı bir dizideki çiftlerin ve teklerin ortalamasını bulan EXE tipi ASM programını yazınız.



Verilen 2 tane küçükten büyüğe sıralı dizileri tek bir küçükten büyüğe sıralı diziye aktaran EXE tipi program.



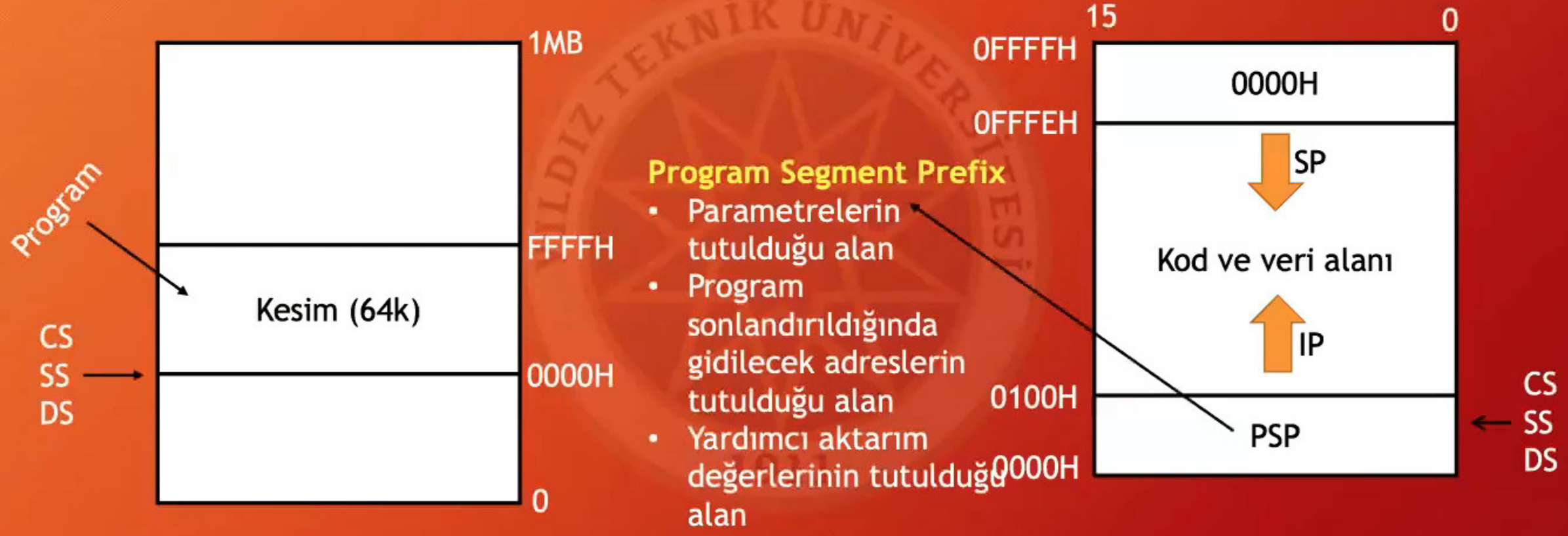
Ders Tanıtım Formu ve Konular

BLM2021
Alt Seviye
Programlama

Hafta 9-10

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

COM Tipi Program Yapısı



Program Segment Prefix

- Parametrelerin tutulduğu alan
- Program sonlandırıldığında gidilecek adreslerin tutulduğu alan
- Yardımcı aktarım değerlerinin tutulduğu alan

COM Tipi Program Yapısı - Devam

- Ana yordam NEAR olarak tanımlanır.
- Geri dönüş adresleri yığına yerleştiril**MEZ**.

COM Tipi - Sintaks 1

BLM2021
Alt Seviye
Programlama

Hafta 9-10

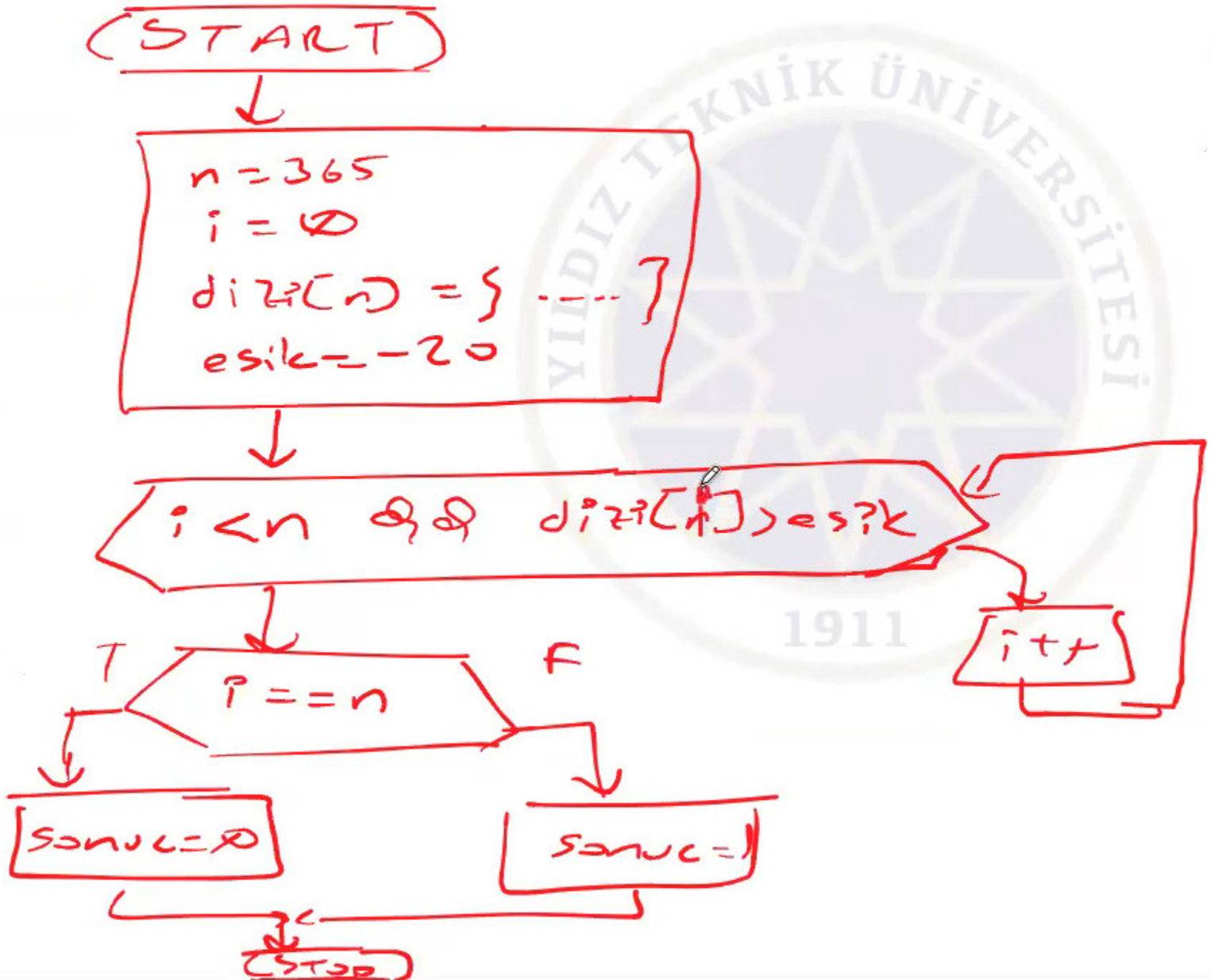
```
codesg  SEGMENT PARA 'kod'
                ORG 100h
                ASSUME CS: codesg, DS:codesg, SS:codesg
Bilgi:          JMP Basla
sayib           DB ?
sayiw           DW ?
                ....
BASLA           PROC NEAR
                ....
                RET
BASLA           ENDP
codesg          ENDS
                END Bilgi
```

1 yıl süresince günlük en düşük hava sıcaklığı bilgileri bulunan bir dizide −20'nin altında sıcaklık değeri olup olmadığını bulan COM tipi ASM programını yazınız.

max(en düşük) = -90

max(sıcaklık) = 57

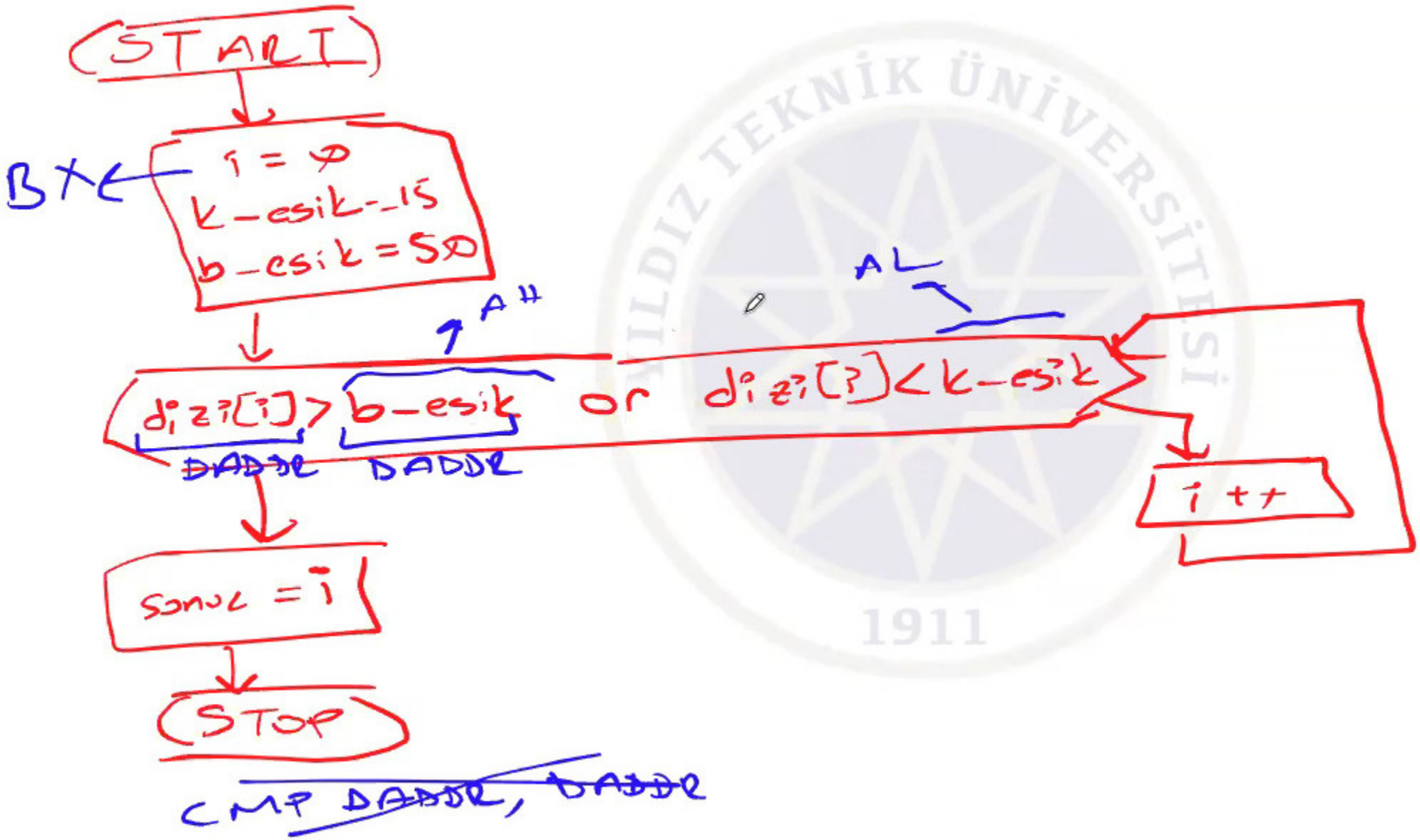
-21



COM Tipi - Sintaks 2


```
codeseg     SEGMENT PARA 'kod'
            ORG 100h
            ASSUME CS: codesg, DS:codesg, SS:codesg
ANA         PROC NEAR
            ....
            RET
ANA         ENDP
sayib       DB ?
sayiw       DW ?
            ....
codesg      ENDS
            END ANA
```

15 yaşından küçük ve 50 yaşından büyük 10 tane kişinin olduğu sıraya yaş koşulunu bozan 11. kişi karışmıştır. Sıradaki tüm kişilerin yaşları bilindiğine göre, sıraya sonradan karışan kişinin baştankaçıncı sırada olduğunu bulan COM tipi ASM programını yazınız.



EXE ve COM Tipi Program Karşılaştırması

Özellik	COM Program	EXE Program
Kullanılabilen bellek miktarı	64 KB	Boş bellek ile sınırlı
Başlangıçta CS yazmacının değeri	PSP'nin bulunduğu kesim adresi	END komutunu takip eden etiket ile belirlenen kodun bulunduğu kesim adresi
Başlangıçta IP yazmacının değeri	0100h	Program bitimini belirleyen END komutunu takip eden etiket ile belirlenen göreli konum adresi
Başlangıçta DS yazmacının değeri	PSP'nin bulunduğu kesim adresi	Dönüş için gerekli olan kesim adresi
Tanımlı değişkenlere erişmek için yapılması gerekenler	Yok	MOV AX, verikesimi_ismi MOV DS, AX
Değişkenlerin tanımlandığı yer	Kesim içinde JMP komutu ile ana yordam arasında veya ENDP komutundan sonra ENDS den önce olmak üzere 2 ayrı şekilde	Veri kesimi

EXE ve COM Tipi Program Karşılaştırması - Devam

Özellik	COM Program	EXE Program
Programı bitirmek için	RET veya INT 21h fonksiyon 4Ch	Eğer dönüş adresleri ana yordam içerisinde yığına atılmışsa RETF aksi hallerde INT 21h fonksiyon 4Ch
Dosyanın ikincil bellek üzerindeki büyüklüğü	Programın byte cinsinden karşılığı kadar	Programın boyutu + 512 Byte EXE Header
Çalışma önceliği	EXE'den önce	COM'dan sonra

Ders Tanıtım Formu ve Konular

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

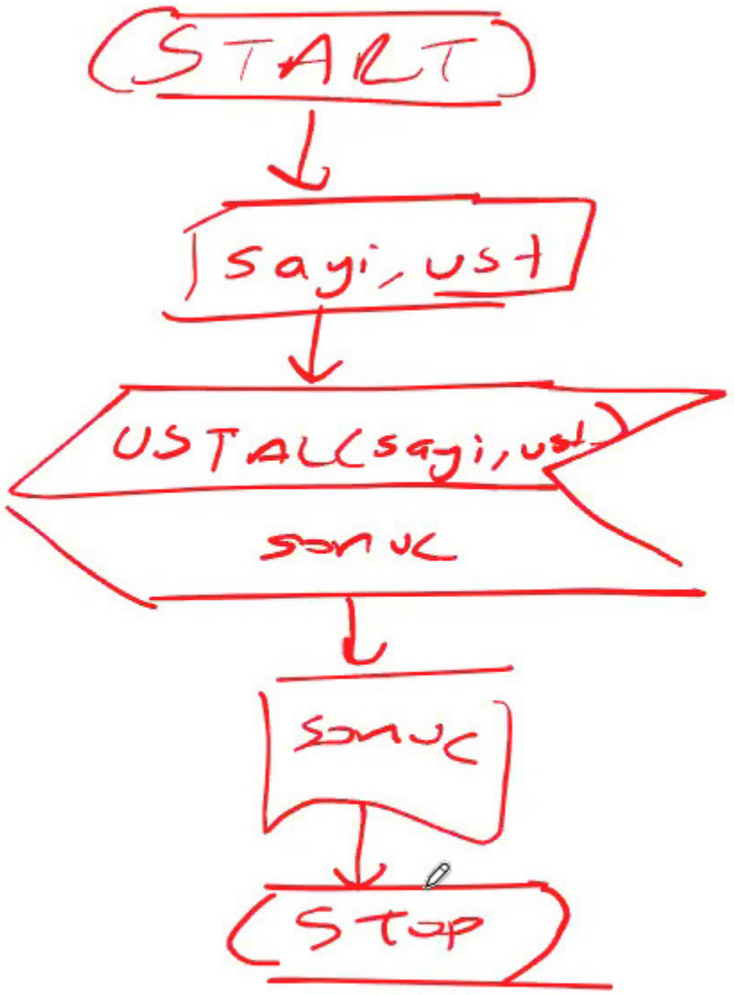
Yordam Tanımı (Syntax)


```
yordam_ismi     PROC {NEAR/FAR}
                ...
                ...        yordamın kodu
                ...
                RET
yordam_ismi     ENDP
```

Yordam Tanımı (Syntax) - Devam


```
codesg  SEGMENT PARA 'kod'
USTAL           PROC NEAR
                ...
                ...
                ...
                RET
                ENDP
ANA             PROC FAR
                ...
                ...
                ...
                CALL USTAL
                NOP
                ...
                RETF
                ENDP
ANA             ENDS
codesg          END ANA
```

Sayı ve üst değerini ana yordamdan USTAL yordamına yollayan ANA yordamı ve üst alma işlemi gerçekleştiren USTAL yordamını EXE tipinde yazınız.



YILDIZ TEKNİK ÜNİVERSİTESİ
1911

Örnek 2 – Inter-Segment Yordam

BL ve BH üzerinden aldığı 2 değeri toplayan ve AX üzerinden sonucu döndüren harici yordamı ve bu harici yordamı çağıran ana yordamı EXE tipinde yazınız.

Makro Tanımı (Syntax)


```
macro_ismi      MACRO {parametreler}
                LOCAL ...
                ASSUME ...
                ...
                ...        } macro kodu
                ...
                ENDM
```

Örnek 3 - Macro

n elemanlı bir dizinin elemanlarını 2'ye bölen MAIN yordamın içinde dizinin en küçük elemanını bulan MACRO'yu COM tipi program için yazınız.

Ders Tanıtım Formu ve Konular

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

Alt Seviye Programlama

Hafta 11

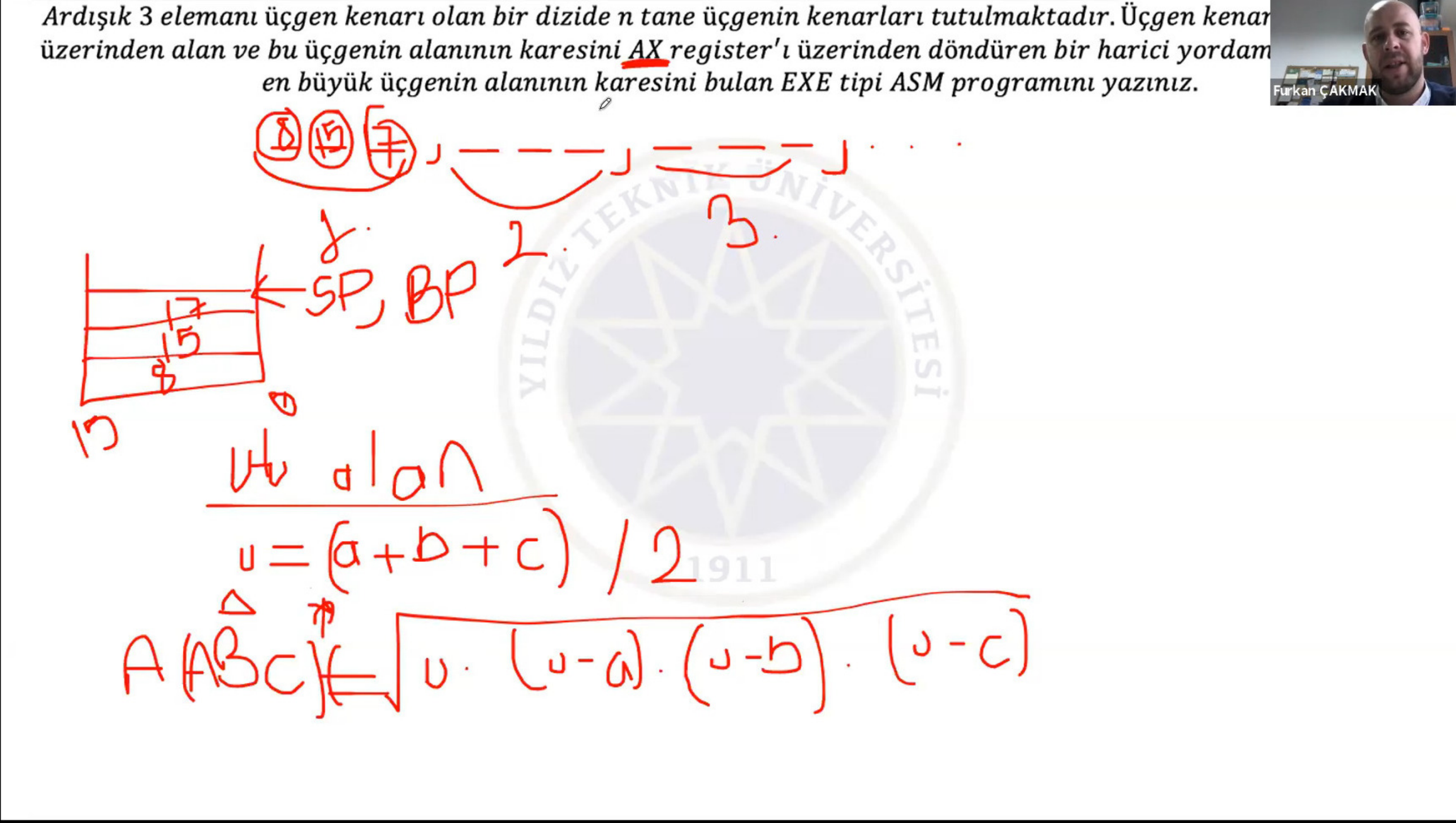
Parametre Aktarma Yöntemleri

- 3 adet yöntem var.
  - Yazmaç üzerinden parametre aktarımı
  - Yığın (stack) üzerinden parametre aktarımı
  - EXTRN/PUBLIC komutları ile parametre aktarımı

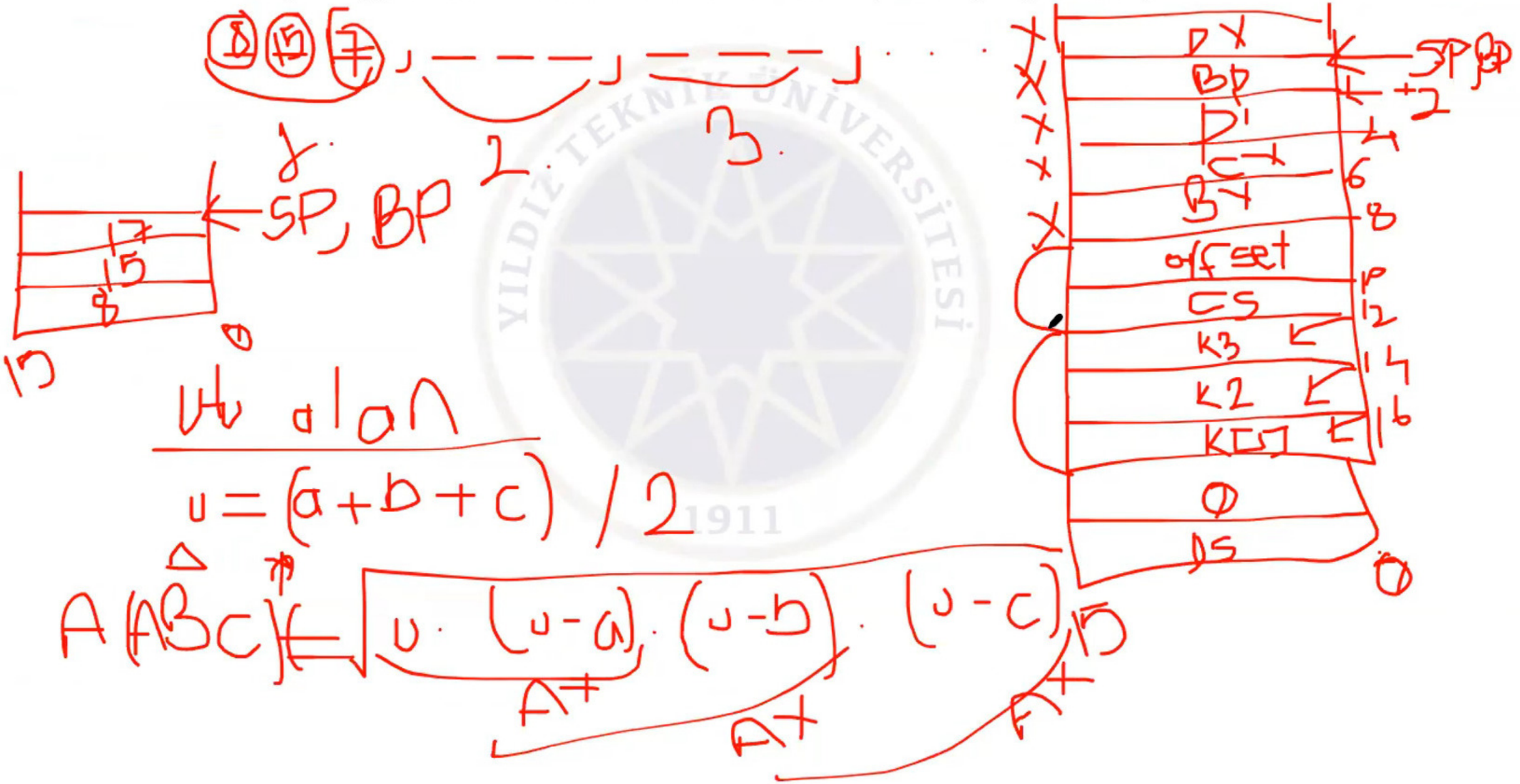
Örnek 1 – Stack Üzerinden Parametre Aktarma

Ardışık 3 elemanı üçgen kenarı olan bir dizide n tane üçgenin kenarları tutulmaktadır. Üçgen kenarlarını stack üzerinden alan ve bu üçgenin alanının karesini AX register'ı üzerinden döndüren bir harici yordam yardımıyla en büyük üçgenin alanının karesini bulan EXE tipi ASM programını yazınız.

Ardışık 3 elemanı üçgen kenarı olan bir dizide n tane üçgenin kenarları tutulmaktadır. Üçgen kenar
üzerinden alan ve bu üçgenin alanının karesini AX register'ı üzerinden döndüren bir harici yordam
en büyük üçgenin alanının karesini bulan EXE tipi ASM programını yazınız.



Ardışık 3 elemanı üçgen kenarı olan bir dizide n tane üçgenin kenarları tutulmaktadır. Üçgen kenarlarını stack üzerinden alan ve bu üçgenin alanının karesini AX register'ı üzerinden döndüren bir harici yordam yardımıyla en büyük üçgenin alanının karesini bulan EXE tipi ASM programını yazınız.



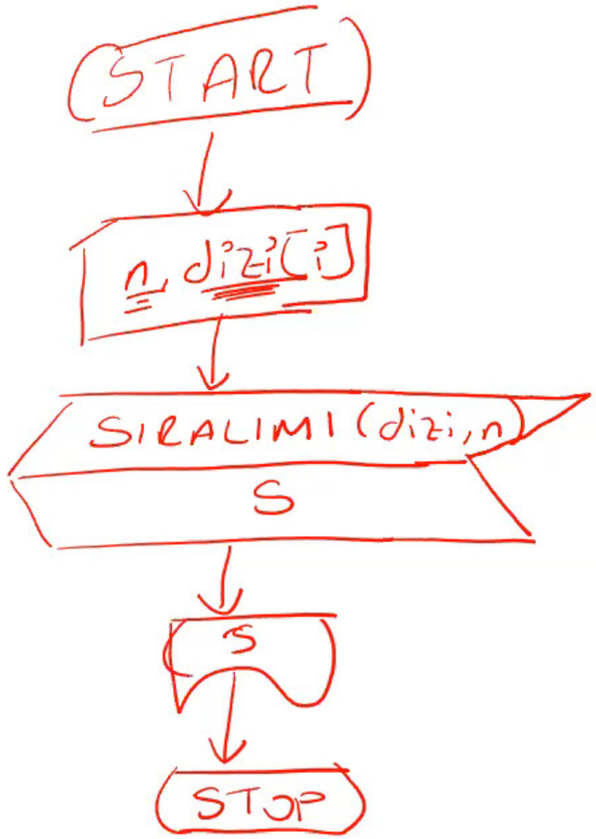
Örnek 2 – EXTRN/PUBLIC ile Parametre Aktarma

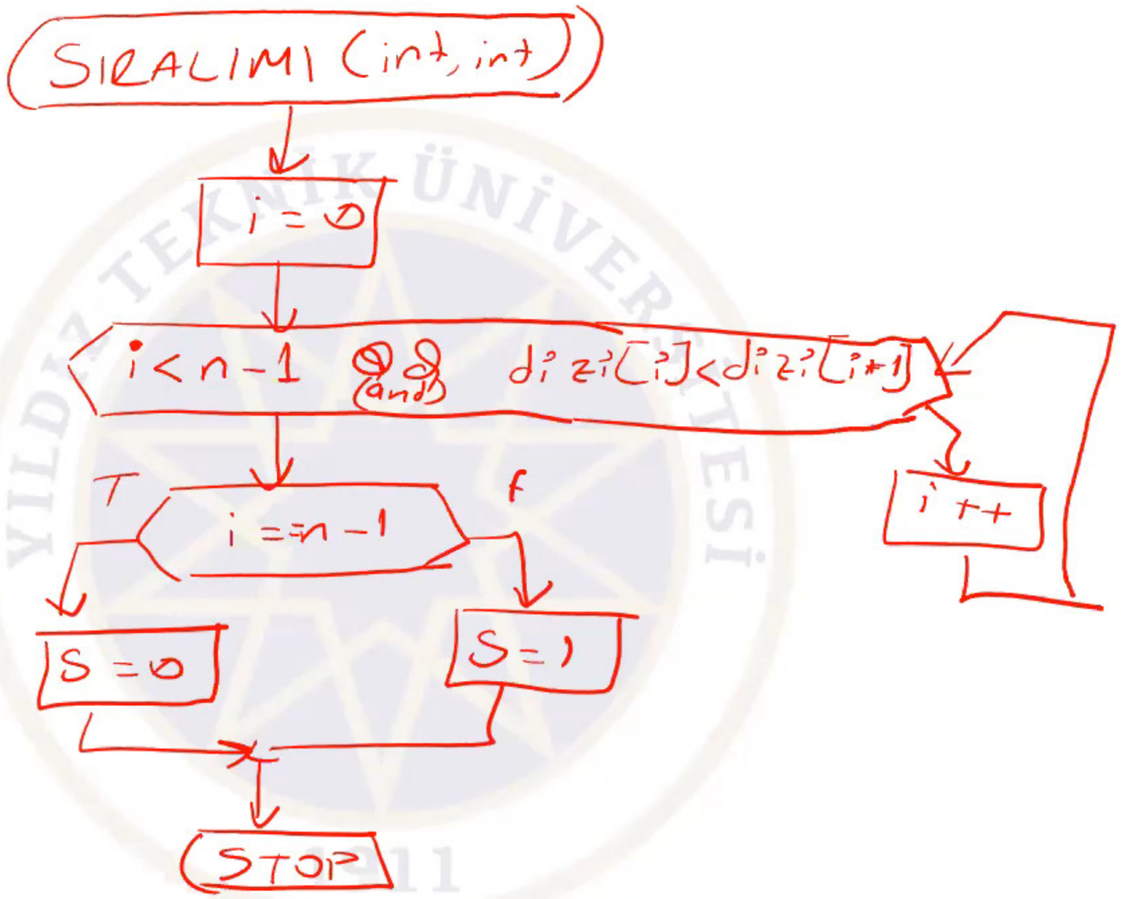
Programlama

Hafta 11

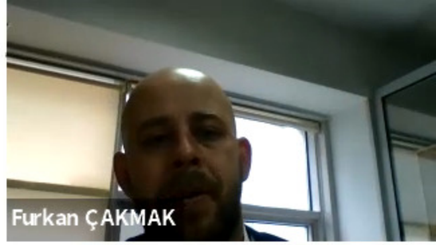
Verilen bir dizinin küçükten büyüğe sıralı olup olmadığını bulan harici yordamı yazınız. Harici yordam verilere EXTRN ve PUBLIC komutları ile erişmelidir.

Verilen bir dizinin küçükten büyüğe sıralı olup olmadığını bulan harici yordamı yazınız. Harici EXTRN ve PUBLIC komutları ile erişmelidir.





YILDIZ TEKNİK ÜNİVERSİTESİ



Ders Tanıtım Formu ve Konular

Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

Kesme (Interrupt) Nedir?

- Çeşitli kriterler ve önceliklerle program işleyişlerinin kesintiye uğratılarak eş zamanlı çalışmanın sağlanması interrupt mekanizması ile sağlanır.
- Yavaş birimler ile hızlı birimler arasındaki bağlantıyı sağlarken zaman kaybetmeyi engellemek için kullanılır.
- Çevre birimleri işlemciden daha yavaş çalışır.
  - Mouse – Klavye
  - Yazıcı
  - Monitör
- Diğer birimler
  - RAM

İşlemci ile Yavaş Birimler Arasındaki İlişki

- Busy waiting
- Polling
- Interrupt
  - Zaman kesmesi
  - Grafik kesmesi

Kesme Çeşitleri

- Temel kesmeler
  - Donanım Kesmeleri
  - Yazılım Kesmeleri
- Interrupt Handler
- Dahili kesmeler
  - Logical interrupt
  - Sıfıra bölme
  - Adım adım çalıştırma
  - Vb.
- Harici kesmeler
  - Maskelenemez Kesmeler (NMI)
  - Maskelenebilir Kesmeler (INTR)
    - IF durumu önemli !
    - 8259A bütünleşik entegresi üzerinden sürdürülür.

YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Kesme Öncelikleri

Öncelik	Kesme	Açıklama
1	Divide by Zero	Hatana neden olan program sonlandırılır. Sistem güvenliğini yitirir. Tekrar başlatılması gerekebilir.
2	INT #	Yazılım olarak numarası verilen kesmenin çağırılması
3	INTO	Aritmetik işlemin (MUL/IMUL) sonucunu değerlendirmek içindir. JO/JNO koşullu dallanma komutları da bu amaçla kullanılır.
4	NMI	INT 02H – Bellek üzerinde oluşan kritik hatalarda kullanılır.
5	INTR	8259A entegresi üzerinde oluşan donanım kesmeleridir.
6	Single step	INT 01H programın adım adım çalışmasını sağlayan kesmedir.

Kesme Oluştuğunda Yapılan İşlemler

1. Bayraklar yığında saklanır (PUSHF)
2. TF = 0
3. IF = 0 (CLI)
4. CS yazmacı yığına atılır (PUSH CS)
5. IP yazmacı yığına atılır (PUSH IP)
6. INT çalıştırılır (# x 4 sonucunda vektör tablosundaki adrese erişilir.)
   - Alınan ilk word değeri IP yazmacına,
   - İkinci word değeri CS yazmacına yerleştirilir.

- Kesme servis programını yazan;
  - Yazmaçlarla ilgili durumları değerlendirmeli,
  - IRET komutu ile kesmeden dönmelidir.

Vektör Tablosunun Görevi ve Konumu

- Kesme servis programları gerek BIOS gerekse İşletim Sistemi tarafından sağlanan değişik uzunluktaki kod parçalarıdır.
- Bilgisayar açılışında belleğe yerleştirilirler.
- Farklı bellek alanlarında bulunan bu kodlara erişmek için Vektör Tablosu kullanılır.
- Vektör Tablosunun tek görevi kesmelerin başlangıç adreslerini tutmaktır.
- 00000H – 003FFH fiziksel adresleri arasında bulunan 1024 byte'lık alanda bulunur.
- Her servisin adresi 2 word'le ifade edilir (Offset ve kesim adresi).
- Yani 1024 / 4 = 256 tane kesme vardır.
- Kullanıcılar kesme yazacaklarsa;
  - INT 60H – INT 67H arası bu iş için ayrılmıştır.
  - Mevcut olan servisin yerine yeni bir kesme yazılabilir.

Vektör Tablosunda Değişiklik Nasıl Yapılır?

- Programcı kendi kesmesini yazabilir.
- Var olan bir kesmeden hemen önce devreye girebilecek bir kesme yazabilir.
- Her iki durumda da yazılan kesme vektör tablosuna yerleştirilmelidir.
- Bunun için yapılması gereken işlemler;
  1. Kesmenin TSR ile belleğe yerleştirilmiş olması gerekir. (TSR -> Terminate and Stay Resident)
  2. Yazılan kodun adresinin (offset + kesim) Vektör Tablosunda ilgili alanlara yazılması gerekir (Kesmeler disable edilmelidir. Edilmezse ?)
  3. Değiştirilecek kesmenin adresi saklanmalıdır.
  4. INT 21H - Fonksiyon 15H (Set Interrupt Vector) kullanılarak yazılan kesme kodunun adresi vektör tablosuna yazılmalıdır.
  5. Kesmelere izin verilmelidir.
     1. Kendi kesmemizi yazmışsak IRET ile dönmeliyiz.
     2. Orijinal kesmeden önce çalışmışsak JMP ile orginal kesme adresine gitmeli ve akışa ordan devam etmeliyiz.
  6. Kullanım tamamlandıktan sonra orijinal kesmenin adresi vektör tablosuna yazılmaldır.

Interrupt Servis Programı Yazarken Nelere Dikkat Edilmelidir?

- Kesme kodu içerisinde girildiğinde kesmelere izin verilmelidir.
- Kesme içerisinde yazmaçların bir önceki değerlerinin korunması gerekmektedir.
- Kesme programı 1'den fazla kesim kullanacak ise bu değerler kesmeye gitmeden önce ilgili yazmaçlara yüklenmelidir.
- Donanım kesmesi yerine (BIOS) geçecek kesme yazılacaksa İşletim Sistemine ait (DOS) hiçbir fonksiyon kullanılmamalıdır.
- Kesme programı TSR ile belleğe yerleştirilmesi gerekmektedir.
- Yığına veri göndermişsek onları kesme içerisinde temizleyecek şekilde hareket sergilemeliyiz.

Kesmeleri Kullanırken

- 00H-1FH arasında BIOS kesmeleri,
- 20H-2FH arasında DOS kesmeleri bulunur.
- Kesmeler çağırılırken,
  - Fonksiyon numarası AH'a,
  - Eğer varsa alt fonksiyon numarası AL'ye aktarılır.
  - INT komutu kullanılır.

- Örnek;
  - MOV AH, 1H
  - INT 21H

YILDIZ TEKNİK ÜNİVERSİTESİ 1911

Ders Tanıtım Formu ve Konular

Programlama

Hafta 12

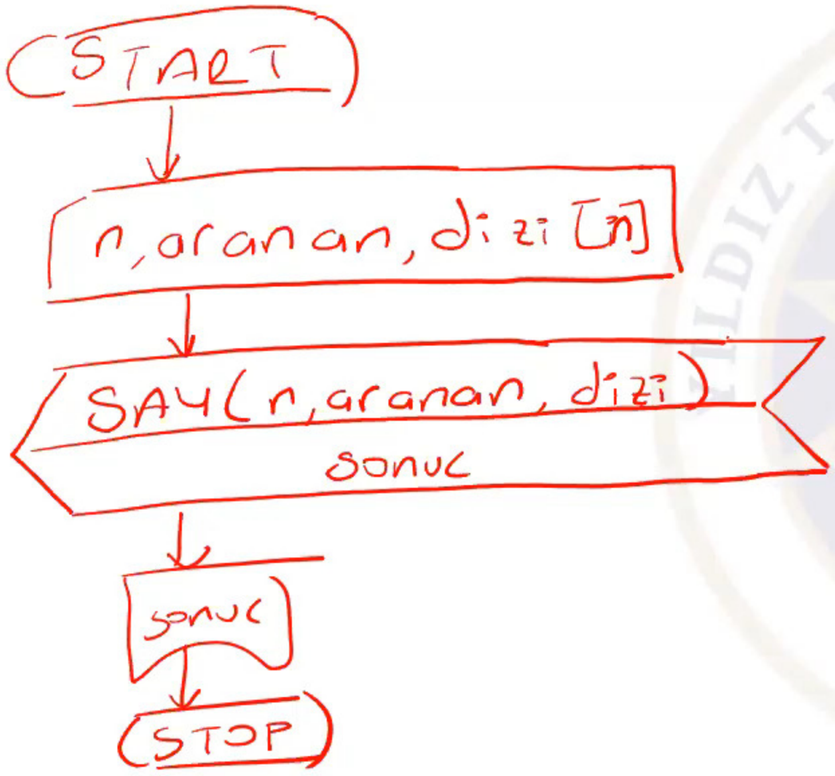
Hafta	Tarih	Konular
1	08.10.2020	Alt seviye dilinin özellikleri, sayı ve kodlama sistemleri, 80x86 ailesi işlemcileri, yazmaçları ve bayrakları ile kesim organizasyonu
2	15.10.2020	Komutlar (veri aktarımı, aritmetik ve dallanma)
3	22.10.2020	Komutlar (çevrim, bayraklar, mantıksal, öteleme, döndürme)
4	29.10.2020	Cumhuriyet Bayramı
5	05.11.2020	Komutlar (katar işlemleri, ön ekler)
6	12.11.2020	Adresleme modları, alt seviye programlama araçları, sözde komutlar
7	19.11.2020	Çalışma ortamının hazırlanması ve debug kullanımı
8	26.11.2020	EXE tipinde alt seviye programlama
9	03.12.2020	COM tipinde alt seviye programlama
10	10.12.2020	Yordam ve makro kullanımları
11	17.12.2020	Alt-programlar ve parametre aktarma yöntemleri
12	24.12.2020	Ortak kesim kullanımı ve EXTRN/PUBLIC tanımlamaları
13	31.12.2020	Kesme, vektör tablosu
14	07.01.2020	Alt seviye programlama dilinin yüksek seviyeli diller ile birlikte kullanılması

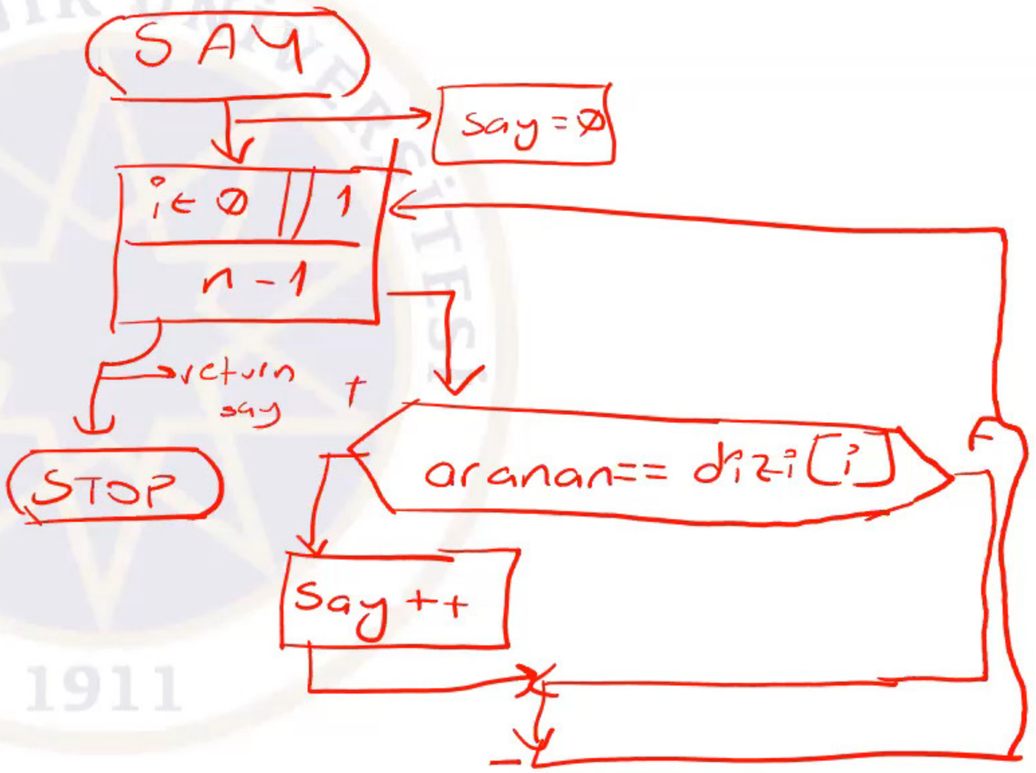
Örnek 1

300 *elemanlı word* tanımlı *bir dizi içerisinde verilen byte tanımlı bir değerin kaç kere geçtiğini bulan EXE tipinde program yazınız. Dizinin adresini, eleman sayısını ve aranan sayıyı ANA yordam STACK'e atacak ve SAY isimli yardımcı yordamı çağıracaktır. Yardımcı yordam AX register'ı üzerinden sonuç döndürecektir.*

NOT: Her iki yordam kodunun aynı kod kesiminde bulunmasını sağlayınız.

300 elemanlı word tanımlı bir dizi içerisinde verilen byte tanımlı bir değerin kaç kere geçtiğ
tipinde program yazınız. Dizinin adresini, eleman sayısını ve aranan sayıyı ANA yordam ST
ve SAY isimli yardımcı yordamı çağıracaktır. Yardımcı yordam AX register'ı üzerinden sonuç
NOT: Her iki yordam kodunun aynı kod kesiminde bulunmasını sağlayınız.





Örnek 2

Pozitif ve negatif sayılardan oluşan 5 elemanlı WORD tanımlı bir dizinin elemanlarının binary gösterimindeki 1'lerin sayını bulan ve dizinin o elemanını bulunan 1'lerin sayısına bölen EXE tipi programı yazınız. ANA yordam sırasıyla dizinin her bir elemanını STACK'e atmakta ve SAYveBOL yordamını çağırmaktadır. Bu yordam STACK'ten aldığı WORD tipindeki değerin 1'lerini sayar ve o değeri bulduğu 1'lerin sayısına böldükten sonra sonucu AX üzerinden döndürür.

Pozitif ve negatif sayılardan oluşan 5 elemanlı WORD tanımlı bir dizinin elemanlarının binary
1'lerin sayını bulan ve dizinin o elemanını bulunan 1'lerin sayısına bölen EXE tipi progra
ANA yordam sırasıyla dizinin her bir elemanını STACK'e atmakta ve SAYveBOL yordamını ç
Bu yordam STACK'ten aldığı WORD tipindeki değerin 1'lerini sayar ve o değeri bulduğu 1'l
böldükten sonra sonucu AX üzerinden döndürür.